

HEIDELBERG UNIVERSITY

DEPARTMENT OF PHYSICS AND ASTRONOMY

MASTER'S THESIS

Machine learning enhanced MHD simulations of the Interstellar Medium

Javier Palau Alegria

born in Cox, Spain

First assesor: Dr. Tobias Buck

Second assesor: Dr. Philipp Girichidis

August 2026

Abstract

In this work, two hybrid machine-learning–numerical methods are studied in the context of three dimensional magnetohydrodynamic simulations of the interstellar medium: superresolution and solvers-in-the-loop. Superresolution methods are used to recover fine scale structures from low resolution simulations using Fourier Neural Operators. We further introduce an additional parameter to the FNO model that shifts the spectral region on which the model is applied. In contrast, solvers-in-the-loop introduce learned corrections directly into the numerical simulation, allowing the core numerical methods to be maintained while improving simulation accuracy. We test curriculum learning and multiproblem approaches, with early signs of generalisation. Both approaches show potential for improving the computational efficiency and accuracy of MHD simulations while maintaining the advantages of numerical methods.

Zusammenfassung

In dieser Arbeit werden zwei hybride Machine-Learning-numerische Methoden im Kontext dreidimensionaler magnetohydrodynamischer (MHD) Simulationen des interstellaren Mediums untersucht: Superresolution und Solvers-in-the-Loop. Superresolution-Methoden werden verwendet, um feinskalige Strukturen aus Simulationen mit niedriger Auflösung mithilfe von Fourier Neural Operators zu rekonstruieren. Zusätzlich führen wir einen weiteren Parameter in das FNO-Modell ein, der den Spektralbereich verschiebt, auf den das Modell angewendet wird. Im Gegensatz dazu führen Solvers-in-the-Loop erlernte Korrekturen direkt in die numerische Simulation ein. Dadurch können die grundlegenden numerischen Methoden beibehalten und gleichzeitig die Genauigkeit der Simulation verbessert werden. Wir untersuchen Ansätze des Curriculum Learnings und des Multiproblem Learnings, wobei sich erste Anzeichen einer Generalisierung zeigen. Beide Ansätze zeigen Potenzial zur Verbesserung der Recheneffizienz und Genauigkeit von MHD-Simulationen, während die Vorteile numerischer Methoden erhalten bleiben.

Contents

1. Introduction	5
I. Theory	7
2. Machine learning	7
2.1. The smallest neural network: The perceptron	9
2.2. Multilayered perceptrons	12
2.3. Optimization of neural networks	15
2.4. Backpropagation	18
2.5. Advanced and practical training techniques	21
2.5.1. Adaptive optimisers: ADAM	21
2.5.2. Learning rate and scheduling	23
2.5.3. Data split	23
2.5.4. Physically informed neural networks	24
2.5.5. Hyperparameter optimization	25
2.5.6. Skip connections	26
2.6. Neural network architectures	27
2.6.1. Convolutional Neural Networks (CNNs)	27
2.6.2. Neural operators	29
2.6.3. Neural operators: Fourier Neural Operators (FNOs)	30
3. Fluid dynamics	32
3.1. Fluid dynamics in astrophysics	32
3.1.1. The Euler equations	32
3.1.2. Magneto hydrodynamics	35
3.2. Differentiable MHD simulations	39
3.2.1. Differentiable simulations	39
3.2.2. Astronomix: differentiable MHD interstellar simulation written in JAX	40

3.2.3. Solver-in-the-loop	41
3.3. MHD type problems	43
3.3.1. Blasts	43
3.3.2. Orszag-Tang vortex	46
3.3.3. Turbulence	48
II. Results	50
4. Post-hoc approach: Super resolution	50
4.1. Dataset	51
4.2. Considered superresolution models: Fourier Neural Operator and Enhanced Deep Residual Network	52
4.2.1. Shifted Fourier Neural Operator	52
4.2.2. Enhanced Deep Residual Network	54
4.3. Benchmarks: SFNO and EDSR models	55
4.4. Metrics study: L1 and spectral loss	56
4.5. An expansion to the SFNO model: U-Net SFNO	59
4.6. Discussion	62
5. Ad-hoc approach: solver-in-the-loop	63
5.1. Considered model: Time independent, divergence-free CNN	64
5.2. Training pipeline and stability mechanisms	65
5.3. Multiproblem API	67
5.4. Experimental evaluation	69
5.4.1. Single field output model	69
5.4.2. Curriculum learning	71
5.4.3. Biprobem multiproblem training	75
5.4.4. MHD Blast Azimuthal rotation generalization model	81
5.4.5. Scaled multiproblem training	83
5.5. Discussion	87

6. Conclusion	90
A. Training hyperparameters	103

1. Introduction

In recent years, machine learning has emerged as a new paradigm in computational physics. Exploiting the large amounts of data available from surveys, machine learning methods can extract complex relationships and enhance classic data analysis methodologies. Numerical methods might seem obsolete under the emergence of such a powerful tool; nevertheless, the power of first principle methods still holds against data based modelling due to their consistency, which ML can't provide. Nevertheless, ML can be used together with numerical methods to provide faster and better insights and results.

More recently, differentiable simulators have also emerged as a new tool for computational physics (LINX [Giovanetti et al., 2025] or JAXOPLANET [Hattori et al., 2024]). They allow exploiting the advances in machine learning within simulation setups while making use of prominent high performance computing. These advancements were possible due to the appearance of high level code that allowed for writing differentiable codes such as JAX [Bradbury et al., 2018], PyTorch [Ansel et al., 2024], or TensorFlow [Abadi et al., 2015], all of which are Python libraries. Approaches that combine classical numerical methods with modern machine learning approaches are referred to as hybrid approaches. Examples of these include:

- **Inverse problems:** Differentiable simulations allow through gradients for the optimisation of initial conditions, boundary conditions or source terms that define the initial states. These could also be attached to observations and thus provide a window to earlier stages of the system of study.
- **Simulation Based Inference:** SBI aims to infer the posterior distributions of unknown parameters from observations. These can be enhanced with differentiable simulations, as the gradient can be used as an additional term to accelerate the inference [Zeghal et al., 2022].

- **Learned solver components:** Data-based components such as neural networks can be used during the simulation calculations to provide accurate or missing physic components like closure terms. Examples include neural closure terms, solver-in-the-loop approaches, learned numerical schemes, and emulators.
- **Super resolution:** Machine learning models can learn to recover fine structures from lower resolution inputs. This allows lower resolution simulations to be run with good accuracy as the finer details ought to be recovered by SR models.

In this work, we develop and evaluate two flavours of hybrid methods within the context of the interstellar medium: superresolution and solvers-in-the-loops. Superresolution methods have proven successful in fields such as climate sciences [Najafi et al., 2026]. In this work, we investigate their application to three-dimensional simulations of the ISM using a neural operator approach.

While superresolution acts beyond the scope of the simulation, solvers-in-the-loop (SOLs) aim to interfere with it during the computation by applying learned corrections. Correcting the simulation on the fly while retaining the core numerical methods allows for longer rollouts than methods that replace them entirely, such as emulators, while improving the accuracy at little computational cost. Our base simulator for both superresolution and SOL approaches is Astronomix [Storcks, 2025], written in JAX.

The primary objective of this work is to evaluate these two approaches and characterise their capabilities and limitations in the context of MHD ISM three-dimensional simulations. Early SOL methods could enable the characterisation of which fields are most responsible to the loss of accuracy in MHD simulations such as supernova blasts. Furthermore, advancements in this field would enable detailed simulations of full galaxies, as in [Hirashima et al., 2025].

Part I.

Theory

2. Machine learning

In this section, we introduce the deep learning methods used for this work, such as convolutional neural networks (CNNs) and Fourier neural operators (FNOs).

The terms artificial intelligence (AI), machine learning (ML) and deep learning are used interchangeably in some literature and public media; here I clarify what each one refers to. AI is the broadest field as it encompasses algorithms able to showcase intelligence ^a Machine learning is encompassed within AI, and it is concerned with programs that can enhance their performance on a given task by "learning" from data. Finally, deep learning is one of the most prevalent forms of ML in recent years; this branch of ML is defined by the use of architectures known as deep neural networks (from which it receives the "deep" name), formed by different layers through which the data flows.

^aWhat is considered as showcasing intelligence goes beyond the scope of the thesis. One of the most famous formalisms of such said intelligence is the Turing Test, where a human evaluates a text that could have either originated from a machine or a human. If the machine is capable of fooling the human into thinking a human has written the text, it can be said that the machine shows some intelligent behaviour.

Humanity has long dreamed of machines capable of learning. The earliest records we have of this dream date back thousands of years to Ancient Greece; myths like Talos or Pandora are some of the first recorded mentions of artificial intelligence [MAYOR, 2018, Introduction]. Nowadays, these concepts wouldn't exist outside of fiction, and myths weren't it, but rather the rise in computational power and developed techniques since the 1950s.

The development of artificial intelligence over the years can be summarised in 4 different epochs, which are usually called summers and winters within the literature

[Toosi et al., 2021]. The birth point of AI can be traced back to 1956 at the Dartmouth Summer Research Project. This started a period of great excitement for the field known as "The first AI summer", which lasted until the late 1960s, when, due to overpromises and underdelivered results, major funding was cut, which in consequence started "The first AI winter". During the 1980s, the field reappeared under a new robust approach called "expert systems". This name relates to the idea that during the first AI summer, AI was designed to provide general solutions from basic reasoning premises, similar to how humans and other intelligent beings think and similar to what nowadays is referred to as "foundational models". These algorithms lacked the scalability and generalisation expected, and in contrast, "expert systems" focused on a narrower subset of solutions derived from domain-specific information. This second AI summer wouldn't last long, as hardware manufacturers rejected the specialised needs of such algorithms, and by the late 1980s the "Second AI winter" started. During this second winter, key developments in the field and computational science allowed the recent rise of AI that we are living in nowadays. Some of the key advancements that made AI possible are the development of the backpropagation algorithm and its application to a dataset of handwritten numbers by Yann LeCun [LeCun et al., 1989], the advancements in Big Data and overall the high availability of data due to the internet, and more broadly the increase in computational capabilities due to GPU technologies.

In recent years, machine learning (ML) has become an increasingly important tool across a broad range of scientific disciplines. The combination of growing datasets, advances in computational resources and the ability of models to identify complex relationships has enabled new approaches to data analysis, parameter inference, and numerical modelling. Thus, ML techniques have been adopted in fields such as biology for genomics and protein structure prediction [Alam et al., 2025, Jumper et al., 2021], climate science for weather forecasting and Earth system modelling [Eyring et al., 2024], particle physics for event reconstruction and anomaly detection [Karagiorgi et al., 2021], or materials science for accelerated materials discovery [Butler et al., 2018] to mention a few. Astrophysics has followed this trend, where ML is now routinely applied to problems including source classification, photometric redshift estimation, gravitational-wave detection, cosmolog-

ical parameter inference, and the emulation of computationally expensive numerical simulations [Ting, 2026, Dvorkin et al., 2022, Baron, 2019].

2.1. The smallest neural network: The perceptron

Machine learning (ML) is the field of study concerned with algorithms that improve their performance at a task through experience, represented by data [Goodfellow et al., 2016, Chapter 5]. In defining what learning means, we ended up with the needed ingredients for any machine learning program:

- **A task:** The chosen problem to solve, categorised under classification and regression.
- **A performance measure:** A quantitative value of the performance of the model on the given task.
- **The data:** The dataset used for training the model. When the data is labelled, and these labels are used to train the model, the algorithm is referred to as supervised. Whereas if the data is unlabelled, or simply not used during training, it is referred to as unsupervised.

Machine learning encompasses a broad range of algorithms, including decision trees, support vector machines, kernel methods, and neural networks. Among these, deep learning has emerged as one of the most successful approaches for learning complex, nonlinear mappings from high dimensional data.

Deep learning is the branch of machine learning focused on neural networks composed of concatenations of linear functions interleaved with nonlinear activation functions. The simplest building block for a neural network architecture¹ is known as a perceptron. A perceptron is a function that transforms inputs $\mathbf{x} = [x_0, x_1, \dots, x_p]^T$ ² into a scalar output y . This function has the following parameters: the weights $\mathbf{w} \in \mathbb{R}^{p \times 1}$ and the bias $b \in \mathbb{R}$,

¹Architecture and model both refer to neural network programs with some subtle differences. Architecture is the combination of perceptrons and different methods used that take the input to the output. In contrast, a model is an instance of such an architecture with the parameters learned from training.

²Throughout this work, we are going to use the column vector standard

and an activation function σ :

$$y = \sigma(\mathbf{w}^T \mathbf{x} + b), \quad (1)$$

a scheme of a perceptron can be seen in Figure 2.1.

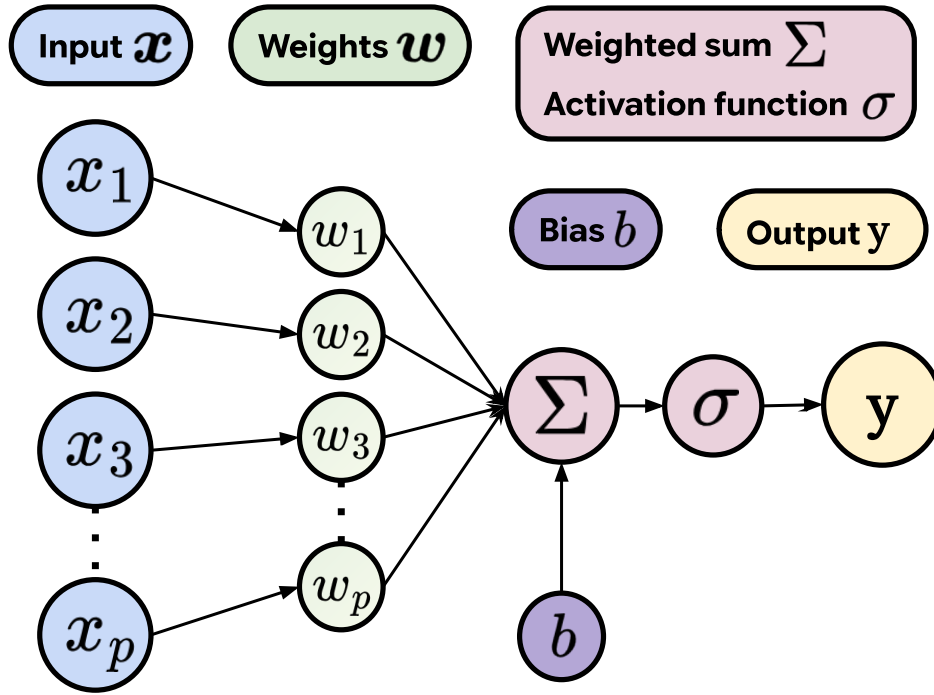


Figure 2.1: Scheme of a basic perceptron. A weighted sum is performed on an input $\mathbf{x}$ with an extra sum term, the bias. The sum output is input to a nonlinear activation function σ to obtain the output y .

When the activation function is the identity mapping $\sigma(x) = x$, the perceptron is reduced to a linear regression architecture. When σ is chosen as the sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$ the architecture receives the name of logistic regression. These two architectures are categorised in the broader categories of regression and classification, respectively. The choice of activation function σ would require work on its own. As will be shown in later sections, to leverage the ability of the models to "learn" complex and nonlinear behaviour from data, we must enforce the nonlinearity of the activation function.

Let's consider the aforementioned example of logistic regression, where the prob-

ability of the input being in one of two different classes (1 or 0) is predicted. Such a model can be written as:

$$p(y = 1|\mathbf{x}, \mathbf{w}, b) = \frac{1}{1 + e^{-\mu(\mathbf{x}|\mathbf{w}, b)}} \quad \mu(\mathbf{x}|\mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b, \quad (2)$$

where y is the label (1 or 0); as such the probability $p(y = 0|\mathbf{x}, \mathbf{w}, b) = 1 - p(y = 1|\mathbf{x}, \mathbf{w}, b)$ and μ receives the name of logit or pre-activation.

With a mathematical description of our perceptron, we are left to describe how to set the weights such that they produce accurate predictions for our problem. To achieve this, a metric to evaluate the performance of our perceptron is needed. Given a dataset $D = \{x_i, y_i\}_{i=0}^N$ of N pairs, where x_i denotes the input feature vector and y_i the corresponding target value, a performance measurement can be written. A standard choice for regression problems is the mean squared error 3:

$$MSE = \frac{1}{N} \sum_{i=0}^N (y_i - \hat{y}_i)^2, \quad (3)$$

where $\hat{y}$ is the predicted output from our perceptron. The performance measure choice depends on the problem and an in depth study is beyond the scope of this work. Standard choices are the mean squared error for regression problems, where the predicted output is in the continuous range, and the cross-entropy loss when predicting discrete labels, like in classification. Assuming the data is sampled from an underlying data generating distribution p_{data} , the performance measurement can be expressed as an expected value over the distribution, known as the risk J :

$$J(\theta) = \mathbb{E}_{(x,y) \sim p_{data}} L(f(x; \theta), y), \quad (4)$$

where L is the loss function, which quantifies the error per data point, $f(x; \theta)$ is a deep neural network and θ are the parameters of the network. This definition is written under the assumption of unregularised supervised training; therefore, the target data y is used in the loss function.

Training a neural network consists of finding the parameter vector θ that minimises the risk function. Since the expectation over the true data generating distribution p_{data} is generally inaccessible, the risk is approximated by its empirical version and computed over the available training dataset. The optimisation problem is typically solved using gradient optimisation methods, usually stochastic gradient descent (SGD) and its variants, which iteratively update the network parameters in the direction of decreasing loss.

Although a perceptron can model nonlinear relationships through its activation function, it is still limited. A single perceptron computes a single affine transformation followed by a nonlinear activation and thus can only represent relatively simple decision boundaries. Many practical problems require more complex representations than can be achieved with a single computational unit. To exploit the capabilities of the perceptron, we need to use more than one at a time, which motivates the introduction of multilayered perceptrons.

2.2. Multilayered perceptrons

Multilayered perceptrons (MLPs) are organised in layers, with each layer composed of several individual perceptrons, where the perceptrons of the previous layer are all connected to the perceptrons of the next layer. These layers are called fully connected layers. Each layer receives as input the output of the previous layer (the input in the case of the first layer, also known as the input layer). Any layer that is not the input or output layer is known as a hidden layer. Each layer is composed of $d^{(l)}$ perceptrons where l signifies the layer (starting from 0 at the input all the way to the total number of layers L). A scheme of a multilayer perceptron can be seen in Figure 2.2. The mathematical description of an MLP can be written as

$$\begin{aligned} \mathbf{h}^{(0)} &= \mathbf{x} \\ \mathbf{h}^{(l)} &= \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \quad l = 1 \dots L \\ \mathbf{h}^{(L)} &= \mathbf{y} \end{aligned} \tag{5}$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{d^{(l)} \times d^{(l-1)}}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^{d^{(l)} \times 1}$. Here we assume σ is the same throughout the network; this is not necessarily true, as depending on the expected output, a design choice or inductive bias could be introduced through the last activation function. For example, in the case of working with physical quantities where non-negativity applies, such as pressure or density, a suitable choice of final activation function could be a Rectified Linear Unit (ReLU), which is defined by:

$$\text{ReLU}(x) = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{if } x \leq 0, \end{cases} \quad (6)$$

the ReLU is one of the most used activation functions in machine learning for its stability as we will see later in the backpropagation section.

The MLP is one of the earliest developed deep neural architectures³. Its expressive power lies in the nonlinearity of the activation function, as without it there would be no need to use the layered structure, as the network would reduce to a simple linear transformation. These nonlinearities make the optimisation problem of training a deep neural network highly non-convex. In classical optimisation problems, second order methods such as Newton's method can be applied, these methods can't be applied to neural networks due to the large number of parameters, typically in the millions. Instead, training is performed with first order gradient optimisation methods.

³MLPs might seem less prominent nowadays due to more modern architectures such as Diffusion models or Transformers. It's important to note that within transformer blocks, a fully connected layer is used afterwards, signifying the importance this architecture entails.

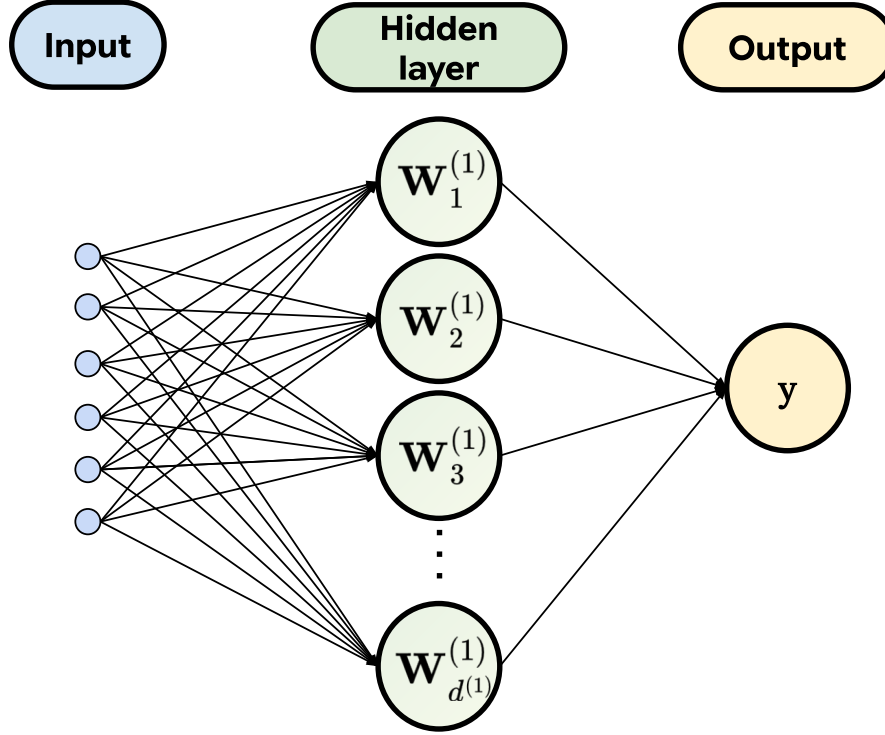


Figure 2.2: Scheme of a multilayer perceptron with 1 hidden layer. For visual simplicity, the linear weight multiplication, the bias, and the activation function are grouped in the perceptron. The architecture maps an input vector $\mathbf{x} \in \mathbb{R}^{d^{(0)} \times 1}$ to an output $y \in \mathbb{R}$ through a hidden layer composed of $d^{(1)}$ perceptrons, corresponding to the weight matrices $\mathbf{W}^{(1)} \in \mathbb{R}^{d^{(1)} \times d^{(0)}}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{1 \times d^{(1)}}$, which in the image $\mathbf{W}^{(2)}$ is embedded in the output perceptron y

Lastly, it is worth motivating the use of neural networks through the theoretical framework. This is done through the Universal Approximation Theorem. The theorem has several variations, and here we present the original formulation, which deals with a network consisting of a single hidden layer of arbitrary width. Given such a network and a sigmoid activation function, the network can approximate up to precision ϵ any arbitrary function. Written formally, the theorem looks like:

Theorem 1 [Hornik et al., 1989] *Let $\mathbb{X}$ be any compact subset of $\mathbb{R}^n$ and let σ be any*

sigmoid activation function. Then the finite sum of the form

$$f_{\text{NN}}(x) = A_2 \sigma(A_1 x + b_1) = \sum_{j=1}^{m_1} a_{2j} \sigma(A_{1j} x + b_{1j}) \quad (7)$$

is dense in $\mathbb{X}$.

In other words, given any function $f : \mathbb{X} \rightarrow \mathbb{R}$ and any $\varepsilon > 0$, there exists a finite sum f_{NN} of the form in Eq. (7) such that

$$|f(x) - f_{\text{NN}}(x)| < \varepsilon \quad \text{for all } x \in \mathbb{X}.$$

Further in [Leshno et al., 1993], the theorem was proven for any non-polynomial arbitrary activation function. This theorem is also studied in the direction of arbitrary depth (arbitrary number of layers) with a single neuron instead of arbitrary width. These theorems give theoretical motivation to the use of neural networks to approximate any unknown function. For further reading on universal approximation theorems, we refer to [Augustine, 2024].

2.3. Optimization of neural networks

The loss landscape of a deep neural network is highly non-convex; traversing it to find optimal parameters for a given model is the most difficult and central task in neural network training. The parameters should yield accurate predictions on data outside of the training distribution. These optimal parameters are given by minimisers of the risk:

$$\hat{\theta} = \arg \min_{\theta} J(\theta) = \arg \min_{\theta} \mathbb{E}_{(x,y) \sim p_{\text{data}}} [L(f(x; \theta), y)] \quad (8)$$

Since p_{data} is unknown, the risk can't be computed analytically. Therefore, we need to substitute the expected value with the empirical version:

$$\hat{J}(\theta) = \mathbb{E}_{(x,y) \sim \hat{p}_{\text{data}}} L(f(x; \theta), y) = \frac{1}{N} \sum_{i=1}^n L(f(x_i; \theta), y_i), \quad (9)$$

where the expected value was substituted with an averaged sum over all the individual losses through the data $D = \{x_i, y_i\}_{i=0}^N$, this formula is known as the cost function or the empirical risk.

The empirical risk function and the MSE are motivated by the maximum likelihood estimation (MLE) method. The maximum likelihood estimate of θ is the value that maximises the likelihood of the parameters given the data points :

$$\mathcal{L}(\theta) = \prod_{i=1}^m p_{\text{model}}(y_i | x_i; \theta), \quad (10)$$

the best estimators for the parameters are therefore given by:

$$\hat{\theta} = \arg \max_{\theta} \mathcal{L}(\theta) = \arg \min_{\theta} [-\log \mathcal{L}(\theta)] = \arg \min_{\theta} \left[-\sum_{i=1}^m \log p_{\text{model}}(y_i | x_i; \theta) \right]. \quad (11)$$

This last form is known as negative log likelihood (NLL) and is preferred over the original form due to numerical underflows: as probabilities go from 0 to 1, if the sample data is large enough, multiplying them would result in a number impossible to represent with 64-bit architectures. To connect back to the MSE, if the conditional model distribution is assumed to be Gaussian, the NLL takes the form (up to constants) of the MSE.

Due to the high dimensionality of θ and non-convexity of the hypersurface defined by $\hat{J}(\theta)$, analytical solutions are non-existent. Therefore, we need to rely on first-order methods; the main method used in machine learning is Stochastic Gradient Descent (SGD), which is a stochastic approach to gradient descent. Gradient descent methods are iterative first order optimisation methods in which the parameters are optimised in the negative direction of the gradient in the hypersurface defined by $\hat{J}(\theta)$.

To make use of gradient descent, the cost function must be differentiable. Under that

assumption, the gradients are given by:

$$\nabla_{\theta} \hat{J}(\theta) = \frac{1}{N} \sum_{i=1}^n \nabla_{\theta} L(f(x_i; \theta), y_i), \quad (12)$$

In later sections, we will comment on how these gradients are calculated through the forward computation. Given the gradients, the parameters of the neural network can be updated following the gradient descent formula:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t), \quad (13)$$

where t signifies the iteration index, also known as epoch number in machine learning, and η the learning rate. The parameters are updated in the opposite direction of the gradient as to decrease the cost function. The stopping mechanism is then determined by the maximum number of epochs set beforehand or more sophisticated methods like plateau detection, known in the machine learning literature as early stopping.

Stochastic gradient descent adds stochasticity to gradient descent by not training on the full set of training data in each step, but instead choosing a reduced batch and training on it. The stochasticity introduced by optimisation dynamics adds noise to the gradient updates. This noise can help the optimiser escape local minima and go towards wider minima that generalise better over unseen data. Another advantage of SGD is merely computational: the reduced data reduces the cost per iteration from $\mathcal{O}(N)$ to $\mathcal{O}(n')$ where N is the dataset size and n' is the batch size, making this approach easier to scale for larger datasets.

In this subsection, the basic pieces for a machine learning training framework were introduced. Together, these define an algorithm written in 1, note that we wrote the SGD version as it is more commonly used than basic GD.

Algorithm 1 Stochastic Gradient Descent for Empirical Risk Minimisation

Require: Dataset $D = \{(x_i, y_i)\}_{i=1}^N$, model $f(x; \theta)$, loss L , learning rate η , batch size n' , number of epochs T

- 1: Initialize parameters θ_0
- 2: **for** $t = 0, \dots, T - 1$ **do**
- 3: Sample a mini-batch $B_t \subset D$ with $|B_t| = n'$
- 4: Compute predictions $\hat{y}_i = f(x_i; \theta_t)$ for $(x_i, y_i) \in B_t$
- 5: Compute mini-batch loss:

$$\mathcal{L}_t = \frac{1}{n'} \sum_{(x_i, y_i) \in B_t} L(\hat{y}_i, y_i)$$

- 6: Compute gradient $g_t = \nabla_{\theta} \mathcal{L}_t$
 - 7: Update parameters $\theta_{t+1} = \theta_t - \eta g_t$
 - 8: **end for**
 - 9: **return** $\hat{\theta} = \theta_T$
-

2.4. Backpropagation

Neural network training relies on the computation of derivatives of the empirical risk function to optimise the networks. The methods for derivative computations can be aggregated in 3 classes, [Baydin et al., 2018]:

- **Symbolic differentiation:** The derivative is computed analytically via the well known differentiation rules. There are computational algorithms that perform these computations (i.e., Matlab, Mathematica, etc.). There are two main drawbacks to this approach in a neural network setting: first, the need to write your model as a closed-form expression, thus limiting the use of control flow. Secondly the complexity and intermediate values required for the final result grow as the networks grow, therefore it poses a limit on the scale of the networks, this is known as *expression swell*.
- **Numerical differentiation:** Through exploiting the definition of derivative, numerical methods have been developed to allow for the computation of approximate gradients, such as finite difference methods (forward, backwards or central). The main drawback of these methods is their inaccuracy due to their approximate

nature and sensitivity to the step size. In particular, for neural networks, such a computational framework would require two forward passes for each parameter. Therefore, this limits the scalability of the algorithm to large datasets and networks.

- **Automatic differentiation:** Provided a computational graph, the derivative is computed by traversing this graph in either the forward or reverse direction. In doing so, the intermediate partial derivatives (tangents in forward mode and adjoints in reverse mode) at each node of the computational graph are computed with respect to the connected nodes (the children for the reverse mode and the parents for the forward mode). The derivative value is then obtained by combining the computed intermediate values following the chain rule. This method provides an accurate gradient (up to the floating point precision) and high computational performance.

Automatic differentiation (AD), although not adopted at first in machine learning, as shown in [Domke, 2009], has proven to be the most effective gradient computational method, as it provides the same accuracy as symbolic differentiation while achieving better performance than symbolic and, in some cases, numerical differentiation.

Further going into detail of AD, the computation of the derivative of the output y with respect to the inputs x_i can be performed in the forward or reverse modes (in this example we are showing how AD works with a scalar output as in ML this is mainly the case, this method is easily extendable to the case where the output lives in a larger space).

If now we want to compute the derivative of y with respect to x_i in forward mode AD computation, we must start by assigning each of the intermediate values v_i a value:

$$\dot{v}_i = \frac{\partial v_i}{\partial x_1}, \quad (14)$$

known as the tangent. Then, by use of the chain rule, we can write the tangents as:

$$\dot{v}_i = \sum_{j \in \text{parents}(i)} \dot{v}_j \frac{\partial v_i}{\partial v_j}, \quad (15)$$

where parents means the predecessors of the node v_i in the computational graph. By computing the tangents along the computation, the final derivative can be computed. In this case, the derivative starts from the input towards the output and has to be done for each input.

In the case of reverse mode, the intermediate value assigned to each node differs from the forward mode and now takes the shape,

$$\bar{v}_i = \frac{\partial y}{\partial v_i}, \quad (16)$$

these are known as adjoints, and by using the chain rule we can define them in terms of their children nodes:

$$\bar{v}_i = \sum_{j \in \text{children}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}. \quad (17)$$

The calculation is then done in a two-step fashion: first, the forward pass of the graph has to be done before the derivative calculation (instead of in forward AD, where they are done simultaneously). In doing so, the intermediate values at each node have to be stored, these are called primals. On the second step, starting now from the output, the derivatives are calculated via propagating the adjoints in the reverse direction (hence the name).

Comparing both methods, we can first comment on the shortcomings of forward mode compared to reverse for its application in machine learning. The main drawback lies in the need to perform the computation for each input, whereas in reverse mode this is done once per output. Thus, reverse mode is significantly less costly time wise, although highly costly memory wise compared to forward, as the primals must be stored during the forward pass. To alleviate the memory constraints, methods such as checkpointing or rematerialisation are used. In these methods, not all the primals are saved during the forward pass. Still, only a selected few are saved during the backward pass, the missing intermediate values are computed using the selected few stored primals. In this way, we alleviate memory constraints while trading it off for computational time.

To enable gradient computation, several computational frameworks have been developed

over the years, such as PyTorch, TensorFlow, and Jax. These libraries allow the user to obtain derivatives from automatic differentiation by constructing the computational graph of a given mathematical operation. This abstraction relieves the user from manually deriving and implementing the gradient expressions, while also enabling optimised implementations on CPUs and GPUs.

Lastly, it should be noted that when expanding either gradient computation equation, forward or backwards, what ends up appearing is the product of a long series of derivatives. These derivatives can either be larger or smaller than 1. If enough derivatives have magnitude larger than 1, the final value will explode; conversely, enough derivatives smaller than 1 in magnitude can make the gradient vanish. This poses a challenge for deeper neural network architectures, in which gradients tend to vanish/explode. To overcome this, several techniques to regularise training are used, such as activation function selection, skip connections, normalisation, or gradient clipping. One of them is the selection of activation functions, for example, the ReLU was introduced in Equation 6, whose derivative is equal to one for positive inputs, which helps preserve gradients during backpropagation and alleviates the vanishing gradient problem.

2.5. Advanced and practical training techniques

In the previous section, we introduced the basic elements that constitute neural network training. In this section, we go into detail on certain advanced topics of machine learning training that are used across most training frameworks nowadays or that are used in our project.

2.5.1. Adaptative optimisers: ADAM

While simple gradient descent takes a first-order approximation, the optimiser introduced in [Kingma and Ba, 2017] adapts the learning rate by estimating the first and second moments of the gradients. Hence the name adaptive moment estimation, ADAM. The algorithm differs from standard gradient descent by using the gradient (m_t) and the

squared gradient (v_t) moving averages. These follow the formulas:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \end{aligned} \tag{18}$$

where two new hyperparameters were introduced β_1 and β_2 which determine the decay rate of the moving averages and g_t^2 indicates the element wise square ($g_t \odot g_t$).

As the moving averages are initialised to 0s, the updates are heavily biased at the early stages of training. To correct the bias, the moving averages are weighted in the following way:

$$\begin{aligned} \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t}. \end{aligned} \tag{19}$$

Putting everything together, the Adam parameter update formula is:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \tag{20}$$

where ϵ is a constant to ensure safe division. Optimisers like Adam and AdamW are the most common in machine learning due to their proven performance in experiments. The physical intuition behind it is a boulder rolling down a hill. Where optimisers like basic gradient descent didn't account for accumulated steps in the same direction (the physical momentum of the boulder in our analogy), Adam, by keeping two moving averages, can account for those and retain a stable and directed gradient direction towards wider minima. Despite their rapid convergence and robustness, Adam and AdamW have been observed to produce models that generalise worse than SGD [Reddi et al., 2019]. Although its stochastic variant, stochastic Adam, has recently been found to achieve the same or better performance than SGD [Tang et al., 2025]. Thus, to achieve better performance, it is recommended to either use a stochastic version of Adam or use Adam at the early stages of training, then switch to SGD at later stages.

2.5.2. Learning rate and scheduling

Fixed learning rates are undesirable as the motion on the loss landscape can either overshoot or get trapped in local minima. There are several techniques that, during training, modify the learning rate value depending on the dynamics of the loss, such as the prior introduced SGD and ADAM. Another flavour of learning modifiers are schedulers, which change the learning rate value across training through a fixed formula. Typical shapes for schedulers include decaying schedulers like exponential or cosine decays, or cyclical schedulers such as cosine annealing [Loshchilov and Hutter, 2017]. For a further analysis of learning rate schedulers, we refer the reader to [Defazio et al., 2024].

2.5.3. Data split

It is assumed that the training data has been independently and identically distributed from the data generating distribution p_{data} . With access to sufficiently large amounts of data, the empirical distribution approaches the underlying data generating distribution. As this is not typically the case and we have to work with limited data, be it due to computational or complexity constraints, or lack of data, the training data represents a subset of the original data distribution and, as such, could be biased. A consequence of training with limited data is the possibility of overfitting or underfitting. Overfitting means the model adapts too closely to the training data. On the contrary, underfitting occurs when the model is not capable of capturing the underlying relations in the training data. Both lead to poor performance on unseen data, i.e., poor generalisation.

This issue can not be circumvented in trivial ways and has to be solved through smart architecture and training decisions, as well as other techniques such as data augmentation. One key consequence of such data limitations is how the available data is treated and how the trained models are evaluated against it. A naive approach is to train on the full dataset and then test performance on the same data; this leads to meaningless performance measurements, as the network is evaluated on the same data it trained with. Therefore, the common practice is to partition the available dataset into three subsets:

- **Training:** Used for and during training to optimise the network parameters. Could be stochastically sampled in methods like SGD. Typically represents around 80-70 % of the total available data.
- **Validation:** Used during training to measure the performance in real time against never-used during training data. The performance measurement obtained with this subset already represents a more valid measure than with the training data. Thus such a performance measure is used during real time training to evaluate the evolution of the network's performance and can be used in combination with early stoppers. Typically represents 10-15 % of the total available data.
- **Test:** Used after training to give a final and more definite performance measure than with the validation subset, as the validation subset was used in some way during training and although it consists of never-seen data it still could be bias optimised through methods such as early stoppers. Therefore, the test dataset should only be seen by the network once to provide an unbiased estimate of the model's performance. Typically represents 10-15 % of the total available data.

Due to convenience or data limitations, machine learning frameworks sometimes use training data that originates from simulations or different domains than the ones aimed to predict. Assuming that, after training, the final model performs as expected on the training domain, this does not ensure similar performance on the target domain, because the underlying distributions may differ. Thus, the model has to undergo what is known as transfer learning [Zhuang et al., 2020], where the model is finetuned to adapt to the target domain instead of the source domain on which it was trained.

2.5.4. Physically informed neural networks

Physical systems are constrained by conservation and symmetry laws encoded by partial differential equations. Training a neural network on such a system leads to physically aware neural networks with sufficient data. As this is typically not the case, we must enforce some of these constraints within the network or the training procedure. Such

networks are known as physically informed neural networks (PINNs) and achieve data-driven and physically informed networks through feature engineering, model constraints and extra residual terms to the loss [Farea et al., 2024] :

- **Feature engineering:** Knowledge of the physical quantities that rule the study domain is useful to extract and selectively choose the features the model should take as input to avoid multicollinearity. This approach can also be done the other way: we could create new features as a combination of others that convey greater physical meaning. Either way, feature engineering allows for physics informed data even before the model has seen it. ⁴
- **Model constraints:** Model architectures entail their own constraints by design, be it translation as in the case of convolutional neural networks (CNNs) or rotation in the case of graph neural networks (GNNs). By choosing the right architecture, physical constraints can be addressed from the outset.
- **Extra residual terms:** Where a standard choice for loss function L in machine learning, like the mean squared error, might work for most problems, it does not capture the constraints and physical meaning of a system. To do so, a widely used flavour of PINNs is to add an extra term to the loss function which encodes some physical constraint, usually in the form of a PDE or a conservation law.

2.5.5. Hyperparameter optimization

In the prior sections, several methods and techniques have been introduced to train a neural network. It should come to the attention of the reader that such a framework depends highly on the values of the parameters that affect the training before it even started (known as hyperparameters). Hyperparameters include the learning rate and ADAM parameters, the model used and its parameters, the early stopping patience, the learning rate scheduler, the mini-batch size... It's a balancing effort to set the values of the parameters correctly for stable training. Standardised values are a good starting

⁴Although not explicitly feature engineering, data augmentation methods that aim to enforce other constraints (i.e. augmenting a dataset of images with rotated images, as we know the model should be rotation invariant) could be seen as a way of embedding constraints within the data.

point, still, methods to finetune further include grid searching, random searches, or Bayesian-based methods such as Gaussian processes or Tree-Structured Parzen methods.

Grid search explores the hyperparameter space in a grid fashion, such an approach scales prohibitively with the number of parameters and may underperform against most advanced methods in convergence speed. Similarly, random searches, although useful as a first approach for a small hyperparameter space, may be outperformed in convergence speed by more advanced methods. On the contrary, Bayesian optimisation methods offer a more efficient traversal of the parameter space, although their design and application are considerably more complex. These methods, together with pruning strategies for training instances that don't seem promising, are widely used in modern training frameworks and are grouped under software libraries such as Optuna [Akiba et al., 2019]. Optuna provides an easy-to-use API and support for hyperparameter optimisation within Python; other alternatives exist, such as Ray Tune [Liaw et al., 2018], which specialises in parallelisation.

2.5.6. Skip connections

Skip or residual connections alleviate the vanishing gradient problem by introducing an identity mapping that provides a direct path for the gradient and the computation; they were popularised by the ResNet architecture introduced in [He et al., 2015]. As an example, consider a network composed of 2 differentiated parts f_1 and f_2 , connected through a skip connection:

$$f_{NN} = f_2(x + f_1(x)), \quad (21)$$

applying the chain rule yields

$$\frac{\partial f_{NN}}{\partial x} = \frac{\partial f_2}{\partial (x + f_1(x))} \left(\frac{\partial f_1}{\partial x} + I \right). \quad (22)$$

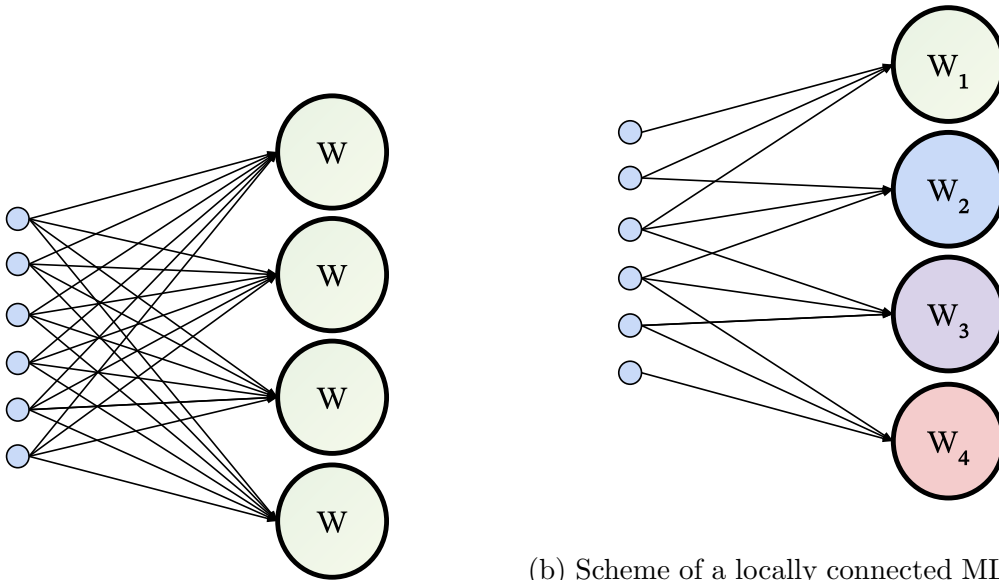
The identity term I provides a direct path for the propagation of the gradient. Therefore, even if the Jacobian of f_1 becomes very small, the gradient can still propagate through

the skip connection, mitigating the vanishing gradient problem while also allowing information to flow more easily through the network.

2.6. Neural network architectures

In the following, we introduce a few neural network architectures that are used throughout this work.

2.6.1. Convolutional Neural Networks (CNNs)



(a) Scheme of a weight sharing MLP, the same weight matrix is used for each of the perceptrons.

(b) Scheme of a locally connected MLP; each perceptron is only connected to 1 input from the previous layer and its neighbours. The number of neighbours depends on the kernel size, which in the figure is 3.

Figure 2.3: Schemes of both a weight sharing and locally connected MLP. By combining these two approaches, the number of parameters required is highly reduced, and the network applies a convolution at the input from the previous layer, thus receiving the name Convolutional Neural Network.

Multilayered perceptrons consist of fully connected layers, these layers are expensive in parameter count, as each one requires $N = (n_{in} + 1) \times n_{out}$. One way to reduce

the number of parameters is by using locally connected NNs or weight sharing NNs. Locally connected NNs reduce the number of parameters by connecting 1 input and its neighbours to an output. In a 1 dimensional case it would be 3 inputs (1 and 2 neighbours) for 1 output. Weight sharing NNs apply the same weight to all the perceptrons in a layer. Both architectures are represented in Figure 2.3. By combining both approaches, the number of parameters is reduced to $N = 3 + 1$ including the bias. Thus, the network applies a convolution to the input from the previous layer, thus the name Convolutional Neural Networks (CNNs).

It should be noted that, although not an explicit requirement, this architecture is order dependent and thus it's usually best applied when the problem at hand has a natural ordering, as is the case of 2D images. Focusing briefly on the 2D case, as it's the most intuitive and most used, we can see in Figure 2.4 the application of the kernel. Given an image I of pixel values $I(i, j)$ and kernel K of values $K(l, m)$ and size $K_h \times K_w$ the resulting output C at the point (x, y) is given by:

$$C(x, y) = \sum_{l=0}^{K_h-1} \sum_{m=0}^{K_w-1} I(i+l, j+m) K(l, m) \quad (23)$$

Widely extended kernel size choices for 2D images are 3x3 or 5x5, although any size can be used. Another important parameter in CNNs is the stride, which defines the step size of the kernel application; together with padding, it defines the size of the output layer. For example, if the input image is 128x128 and we use a kernel of size 5x5 with stride 2 and no padding, the output will result in an image of size 62x62, given by the formula $\lfloor \frac{N-K+2P}{S} \rfloor + 1$. When applying CNNs to 2D images, a per channel approach is used; therefore, a kernel is defined for each channel output. Furthermore, more channels can be created, increasing the dimensionality in the hidden layers. Lastly, it's important to note that, as the convolution operation is shift equivariant, the whole model is. Consequently, CNNs are not only useful due to their ease of memory but also due to their enforced architectural constraint.

CNNs have been successful in applications such as image recognition and classification [He et al., 2015], medical image analysis [Ronneberger et al., 2015], super-resolution

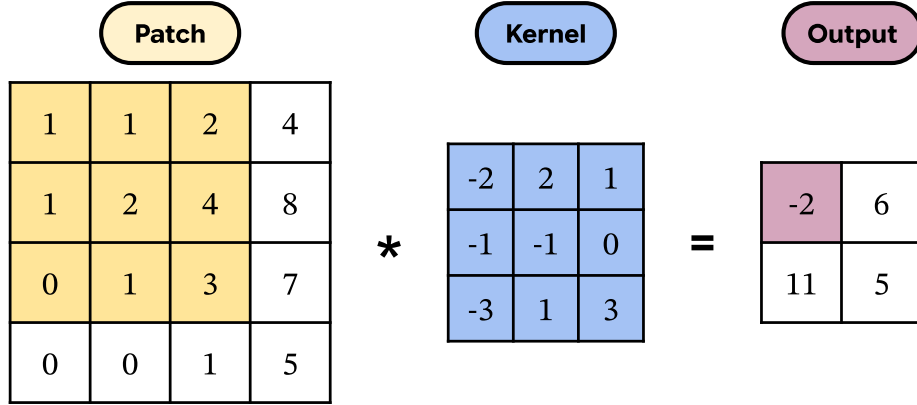


Figure 2.4: Example of a convolution application in the 2D case.

[Zhang et al., 2018] or astronomical analysis [Hayat et al., 2021]. In recent times, CNNs have been widely outperformed by visual transformer architectures. Nevertheless, CNNs still pose a useful solution for certain applications due to their lightness and good performance with small datasets [Pasupuleti, 2025].

2.6.2. Neural operators

The neural networks introduced up to this point act as approximators for discrete functions between finite dimensional Euclidean spaces. This approach, although proven useful, is not mathematically grounded and depends on the discretisation at hand. In [Lu et al., 2021], the neural operator architecture was proposed, where, instead of applying the network to discretised inputs and outputs, the network learns an operator between two function spaces, therefore removing the limitation of discretisation. Examples of operators are PDEs on dynamical systems, where, given an initial condition function, the operator represents the map to the solution.

The main distinction lies in the final output: in standard neural networks, the output is given in a discretised fixed Euclidean space. In neural operators, the output is not fixed to a discretisation and therefore the network behaves as an operator acting on the input function. Typically, the input function can't be accessed, and therefore we have to discretise the data. Thus the architecture has to bridge the gap between finite-dimensional

data and infinite-dimensional function spaces [Kovachki et al., 2024]. This is done by modifying the non-operator layer architectures to allow for infinite-dimensional outputs.

2.6.3. Neural operators: Fourier Neural Operators (FNOs)

Fourier Neural Operators (FNOs) (introduced in [Li et al., 2021]) transform the convolutional layer to the operator network space by transforming it to a spectral convolution. Rather than acting on a discretised grid space, the spectral convolution acts on the Fourier space, thus making the operation resolution invariant. The motivation for the spectral convolution is taken from the spectral convolution theorem (the following description is taken from [Duruiseaux et al., 2026]), where given a convolution kernel $\mathcal{K}$ applied on a function parametrised by a k_θ :

$$(\mathcal{K}v)(x) = \int_D k_\theta(x, y)v(y)dy = \int_D k_\theta(x - y)v(y)dy = k_\theta * v, \quad (24)$$

where in the second equality the fact that $\mathcal{K}$ is a convolution was used. This operation, by the spectral convolution theorem, can be written as

$$(\mathcal{K}v)(x) = \mathcal{F}^{-1}(\mathcal{F}(k_\theta) \cdot \mathcal{F}(v))(x) = \mathcal{F}^{-1}(R_\theta \cdot \mathcal{F}(v))(x), \quad (25)$$

where $\mathcal{F}$ is the Fourier transform and $\mathcal{F}(k_\theta)$ was substituted by R_θ , a learnable complex tensor, which, instead of learning the kernel, learns the Fourier coefficients of the kernel. R_θ acts as a linear map between channels for each Fourier mode k , given $v : D \subset \mathbb{R}^{d_{in}} \rightarrow \mathbb{R}^{d_{out}}$, then $R_\theta(k) \in \mathbb{C}^{d_{out} \times d_{in}}$, thus acting as a mixing operator on a given scale size. The collection of matrices $R_\theta(k)$ is truncated to an upper limit on the wave number $|k| < k_{max}$, this is motivated by the higher wave numbers typically being dominated by noise or small fluctuations that shouldn't mix across the whole physical domain. The remaining frequencies are reconstructed through skip, nonlinear, or other mixing layers. In practice, it is discretised, and therefore the Fourier transform can be computed efficiently via the Fast Fourier Transform.

The full architecture is then comprised of, together with the spectral convolutions, point-

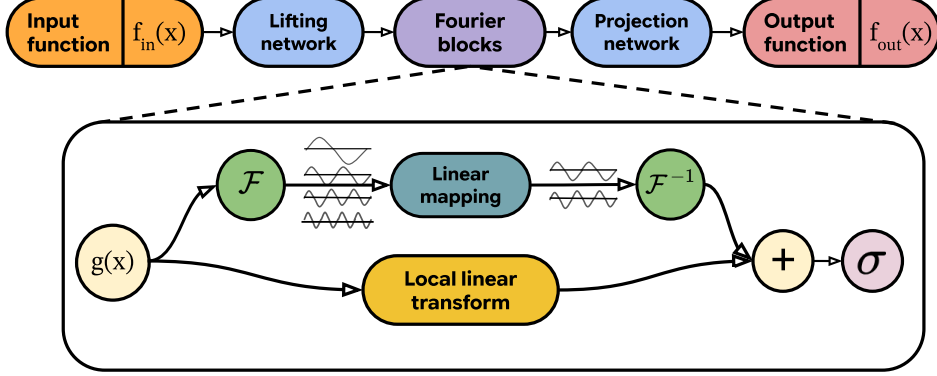


Figure 2.5: FNO full architecture. The discretised input is lifted by the **projection network**. The **spectral blocks** are applied, where a convolution in Fourier space is performed together with a linear transformation. Following the spectral blocks, the state is brought back to its original dimensionality through the **projecting network**.

wise lifting and projecting layers, which raise the low dimensional input into a higher dimensional space and, at the end, lower it back to the original space, respectively, these are typically implemented as MLPs. After the P projection, the FNO blocks are applied, which consist of a spectral convolution together with a pointwise linear transform, usually modelled as convolution with a 1×1 pixel kernel. The output of these layers is then added together and passed through an activation function σ . A scheme of the full architecture can be seen in Figure 2.5. The original FNO architecture has been expanded, for instance as a diffusion model [Liu and Tang, 2025], to general geometries [Li et al., 2024], factorised R_θ matrices [Tran et al., 2023] or with extra operations in the Fourier block such as the application of an integral or local kernels [Liu-Schiaffini et al., 2024].

3. Fluid dynamics

3.1. Fluid dynamics in astrophysics

Fluid dynamics is one of the main areas of study in astrophysics. It describes phenomena such as accretion disks, star formation, galactic dynamics or the interstellar medium.

Because of the computational and mathematical impossibility of studying the behaviour of the ensemble of particles that constitute fluids, fluid dynamics eases the study of such systems by studying the macroscopic behaviour of the fluid such that meaningful statistics can be recovered. Thus, fluids are defined by continuous fields of density, velocity and pressure, which are defined everywhere. This approximation works under the assumption that the mean free path of the particles that constitute the fluid is much smaller than the characteristic scale, $\lambda_{mfp} \ll L$. If this is not the case, a theory of kinematics is needed, given by Boltzmann's formula. In the following sections, the basic equations of hydrodynamics in the case of an inviscid fluid and no thermal conductivity are obtained and justified within the context of the interstellar medium.

3.1.1. The Euler equations

Let's first consider an arbitrary closed volume V with a defined surface S which contains a fluid of mass M with a velocity $\mathbf{v}$ defined at each point of the volume. By considering the amount of mass trapped by the volume V , M , and the mass flowing out of this volume, which is defined by $\rho \mathbf{v} \cdot d\mathbf{S}$ and relating them through the divergence theorem, we can write the continuity equation 26:

$$\partial_t \rho + \nabla \cdot (\mathbf{v} \rho) = 0. \quad (26)$$

This equation is interpreted as the conservation of mass within a certain volume. The change in the density depends on the amount of mass entering and leaving the volume.

Assuming a Newtonian and isotropic fluid, the total force applied to the fluid volume,

defined by the closed surface S , is given by the integral

$$F_{total} = - \oint_S p \, d\mathbf{s}, \quad (27)$$

of the pressure taken over the whole surface. By transforming the surface integral to a volume integral via the divergence theorem, we obtain that the force exerted on a volume element of fluid $d\mathbf{V}$ is given by $-\nabla p$. Forward writing Newton's 2nd law on a unit volume of fluid, we get

$$\rho \frac{D\mathbf{v}}{Dt} = -\nabla p, \quad (28)$$

having introduced notation for Lagrangian or material derivatives $\frac{D}{Dt}$, this derivative is not evaluated at a fixed point in space but rather follows a parcel of fluid. The derivative at a fixed point in space is referred to as the Eulerian derivative ∂_t and is related to the Lagrangian derivative through $\frac{D}{Dt} = \partial_t + \mathbf{v} \cdot \nabla$. Since both are related by a simple equation, the entirety of hydrodynamics can be written in either of the frameworks. In the present work, we employ grid-based methods where, rather than tracking parcels, the parcel's location is fixed. Therefore, the Eulerian view is more convenient. Thus, we rewrite equation 28 in the Eulerian derivative framework and obtain

$$\rho \frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} = -\nabla p. \quad (29)$$

The equations for the conservation of mass 26 and momentum 29 are referred to as Euler's equations, both written in the Eulerian derivative fashion. Together with the equation of state, they define the evolution of a fluid with zero viscosity and no thermal conductivity. These equations were derived with no external source terms assumed, like gravity in the case of the momentum equation. These terms can be added to the RHS of each equation to incorporate extra physics into our model.

Our system so far lacks closure, to do so, we add an equation of state to it. A typical choice of equation of state is the ideal gas equation of state

$$P = (\gamma - 1) \rho \mathcal{U}, \quad (30)$$

where $\mathcal{U}$ is the internal energy per unit mass and γ is the adiabatic index given by $5/3$ in the case of monatomic gases. As a new field was introduced, the internal energy, we must add another equation to complete the system. The conservation of energy equation

$$\frac{\partial \rho e}{\partial t} + \nabla \cdot [(\rho e + P)\mathbf{v}] = 0, \quad (31)$$

which can be derived from the same arguments as equations 26 and 29, given that now the conserved quantity is the total energy per unit mass e given by $e = \mathcal{U} + \frac{1}{2}\mathbf{v}^2$. Lastly, with this equation we've given closure to the hydrodynamical model. The list of assumptions considered while obtaining these equations and their validity in modelling the astrophysical interstellar medium is as follows ⁵:

- **Hydrodynamic approximation:** Given the Hydrogen number density $n_H \sim 0.6 \text{ cm}^{-3}$ the mean free path is $\lambda_{mfp} \sim 2 \times 10^{15} \text{ cm}$. Whereas the characteristic length $L \sim 1 \text{ pc} = 3.09 \times 10^{18} \text{ cm}$ is 3 orders of magnitude larger than λ_{mfp} . Thus justifying the hydrodynamic approximation.
- **Inviscid:** The inviscid assumption can be validated by comparing the viscous diffusion and dynamic timescales. Following [Carraro, 2021][Chapter 4.1], for WNM conditions the viscous diffusion timescale is $\tau_\nu \sim 2.5 \times 10^7 \text{ yr}$ whereas the dynamical timescale is $\tau_{dyn} \sim 1.2 \times 10^5 \text{ yr}$. Then $\frac{\tau_\nu}{\tau_{dyn}} \sim 200 \gg 1$ meaning that viscous momentum transport occurs much more slowly than the dynamical evolution and can therefore be neglected.
- **Negligible thermal conductivity:** Similarly, the thermal conductivity assumption can be validated by comparing characteristic times. Again, following [Carraro, 2021], for WNM values the dynamic thermal timescale $\tau_t \sim 1.4 \times 10^8 \text{ yr}$, thus $\frac{\tau_t}{\tau_{dyn}} \sim 10^3 \gg 1$. In consequence, the thermal conductivity can be neglected as the dynamical evolution of the gas dominates.
- **Ideal gas:** The ISM is a highly diffuse gas. The hydrogen number densities $n_H \sim$

⁵We take the values from the warm neutral medium WNM from [Draine, 2011][Table 1.3] with $n_H \simeq 0.6 \text{ cm}^{-3}$ and $T \simeq 5000 \text{ K}$. This phase constitutes around 40% of the ISM volume, thus being representative of the fluid dynamics considered.

$10^{-3} - 10^6 \text{ cm}^{-3}$, thus collisional interactions between particles are scarce, and the ideal gas assumption is well motivated.

A comment on simulations Equations for the mass 26, momentum 29 and energy 31 conservation together with the energy and the equation of state give closure to the model. To simulate such a fluid, the conserved quantities are evolved over time by defining a system of conserved equations

$$\partial_t S + \nabla \cdot F(S) = 0, \quad (32)$$

where S is the conserved state vector and F the flux rank 2 tensor. With the equations described prior, S and F are

$$S = \begin{pmatrix} \rho \\ \rho \mathbf{v} \\ \rho e \end{pmatrix}, \quad F(S) = \begin{pmatrix} \rho \mathbf{v} \\ \rho \mathbf{v} \mathbf{v}^\top + P \mathbf{I} \\ (\rho e + P) \mathbf{v} \end{pmatrix} \quad (33)$$

where $\mathbf{I}$ is the identity tensor. To write these equations, the conservative form of the momentum equation was used. For a derivation of such an equation, see [Sormani, 2022, Section 1.8]. The first component corresponds to the mass flux, the second to the momentum flux and the last one to the energy flux. Written in this conservative form, the Euler equations become a system of hyperbolic conservation laws, a formulation well suited for finite volume methods commonly employed in computational fluid dynamics.

3.1.2. Magneto hydrodynamics

As the interstellar medium is weakly ionised, we must consider the electromagnetic fields that are coupled to the hydrodynamics equations. The theory that describes such an ionised fluid is magnetohydrodynamics (MHD). In the case of the ISM, the ionisation fraction $x_e = n_e/n_H$ varies across the considered areas. In dense molecular clouds, the gas is almost neutral, $x_e \lesssim 10^{-6}$, whereas in HI or HII regions the ISM is partially to

fully ionised, $x_e \sim 10^{-3} - 1$ [Draine, 2011, Chapter 13].

Maxwell's equations that model the EM fields are

$$\nabla \cdot \mathbf{E} = 4\pi\rho_e \quad (34)$$

$$\nabla \cdot \mathbf{B} = 0 \quad (35)$$

$$\nabla \times \mathbf{E} = -\frac{1}{c} \partial_t \mathbf{B} \quad (36)$$

$$\nabla \times \mathbf{B} = \frac{4\pi}{c} \mathbf{J} + \frac{1}{c} \partial_t \mathbf{E}, \quad (37)$$

first assuming the fluid considered has a net charge of zero and therefore there's almost no net force exerted by $\mathbf{E}$, we can discard the time derivative of the electric field and write $\nabla \times \mathbf{B} = \frac{4\pi}{c} \mathbf{J}$. Thus, writing the Lorentz force $F_{EM} = \frac{1}{c} \mathbf{J} \times \mathbf{B}$ ⁶. Taking the induction equation and substituting $\mathbf{J}$, the Lorentz force can be written in a conservative form and only in terms of the magnetic field

$$F_{EM} = \frac{1}{4\pi} (\nabla \times \mathbf{B}) \times \mathbf{B} = \frac{1}{4\pi} \left[-\frac{1}{2} \nabla B^2 + (\mathbf{B} \cdot \nabla) \mathbf{B} \right] = \frac{1}{4\pi} \nabla \left[-\frac{1}{2} B^2 + \mathbf{B} \mathbf{B}^\top \right], \quad (38)$$

where on the first equality the following vector identity was used $(\nabla \times \mathbf{B}) \times \mathbf{B} = -\frac{1}{2} \nabla B^2 + (\mathbf{B} \cdot \nabla) \mathbf{B}$ and on the second one $\nabla \cdot (\mathbf{B} \mathbf{B}^\top) = (\mathbf{B} \cdot \nabla) \mathbf{B} + \mathbf{B} (\nabla \cdot \mathbf{B}) = (\mathbf{B} \cdot \nabla) \mathbf{B}$ by the Gauss magnetic field equation. We can insert the Lorentz force equation in the momentum equation 29 and obtain:

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} \right) = -\nabla \left(P + \frac{B^2}{8\pi} \right) + \frac{1}{4\pi} (\mathbf{B} \cdot \nabla) \mathbf{B}. \quad (39)$$

Using the last form of 38 and the momentum part from 33, the conservative form of the ideal MHD momentum equation can be written

$$\frac{\partial \rho \mathbf{v}}{\partial t} + \nabla \left[\rho \mathbf{v} \mathbf{v}^\top + \left(P + \frac{B^2}{8\pi} \right) \mathbf{I} - \frac{1}{4\pi} \mathbf{B} \mathbf{B}^\top \right] = 0. \quad (40)$$

⁶This result is also obtained by assuming an almost net charge of zero and no relativistic effects as done in [Sormani, 2022]. The third equation in 34 shows that $E \sim \frac{v}{c} B$ and therefore, when compared with the LHS of the fourth equation, the term $\partial_t \mathbf{E}$ can be neglected. Depending on the author one assumption is mentioned over the other, whichever is chosen is important to notice that by neglecting $\partial_t \mathbf{E}$ Maxwell's equations and the Euler equations are both Galilean invariant.

In order to ensure the closure of the model, the induction equation under the assumption of a perfect conductor ($\sigma \rightarrow \infty$)

$$\frac{\partial \mathbf{B}}{\partial t} = \nabla \times (\mathbf{v} \times \mathbf{B}) \quad (41)$$

is needed. Using this equation, by studying the flux that goes through a given surface, it can be shown, that the field lines are "frozen" in the fluid, as shown in [Draine, 2011, Chapter 35.4]. We take from the same reference the motivation to set the assumption of a perfect conductor in the case of an ionised medium. The authors find the time scale for magnetic field decay on an ionised gas with sizes $L \gtrsim 10^2 \text{AU}$ to be larger than the age of the Universe, hence motivating the assumption for large structures, whereas on short scales this approximation no longer applies.

Lastly, we must consider the conservation of energy equation, considering the magnetic energy

$$\partial_t \left(\rho e + \frac{B^2}{8\pi} \right) + \nabla \cdot \left[\left(\rho e + P + \frac{B^2}{8\pi} \right) \mathbf{v} - \frac{1}{4\pi} (\mathbf{v} \times \mathbf{B}) \times \mathbf{B} \right] = 0, \quad (42)$$

where e is given by the internal and kinetic energy. For a derivation of this equation, see [Galtier, 2016, Section 3.3].

All the equations considered in these last two subsections are aggregated in Table 1.

Table 1: Hydrodynamic and ideal MHD equations in conservative form. Here ρ is the density, $\mathbf{v}$ the velocity field, P the thermal pressure, $\mathbf{B}$ the magnetic field, and $e = \mathcal{U} + \frac{1}{2}v^2$ the total hydrodynamic energy per unit mass.

Equation	Hydrodynamics (HD)	Ideal MHD
Continuity	$\partial_t \rho + \nabla \cdot (\rho \mathbf{v}) = 0$	$\partial_t \rho + \nabla \cdot (\rho \mathbf{v}) = 0$
Momentum	$\partial_t(\rho \mathbf{v}) + \nabla \cdot (\rho \mathbf{v} \mathbf{v}^\top + P \mathbf{I}) = 0$	$\partial_t(\rho \mathbf{v}) + \nabla \cdot \left[\rho \mathbf{v} \mathbf{v}^\top + \left(P + \frac{B^2}{8\pi} \right) \mathbf{I} - \frac{\mathbf{B} \mathbf{B}^\top}{4\pi} \right] = 0$
Energy	$\partial_t(\rho e) + \nabla \cdot [(\rho e + P) \mathbf{v}] = 0$	$\partial_t \left(\rho e + \frac{B^2}{8\pi} \right) + \nabla \cdot \left[\left(\rho e + P + \frac{B^2}{8\pi} \right) \mathbf{v} - \frac{1}{4\pi} (\mathbf{v} \times \mathbf{B}) \times \mathbf{B} \right] = 0$
Eqn. of state	$P = (\gamma - 1) \rho \mathcal{U}$	$P = (\gamma - 1) \rho \mathcal{U}$
Induction	—	$\partial_t \mathbf{B} = \nabla \times (\mathbf{v} \times \mathbf{B})$
Constraint	—	$\nabla \cdot \mathbf{B} = 0$

3.2. Differentiable MHD simulations

To simulate the Euler equations presented in Table 1 a number of proposed methods exist, these can be split into two groups, particle and grid based. Particle based methods track fluid particles (which are not the same as the physical particles that constitute matter) position and the macroscopic fluid quantities are obtained through kernel density estimation or any other field estimation method. On the other hand grid based methods divide the simulation box into a grid (either uniform or nonuniform). These cells can be either considered as a finite volume over which an average fluid field quantity value is assigned, known as finite volume methods, or, the centre of each cell can be considered a fixed point on which the fluid is evaluated, known as finite difference methods.

Magnetohydrodynamics simulations pose an extra challenge to the simulation, as a discretised form of the free magnetic field divergence has to be ensured throughout the simulation. If this is not the case, the simulation could lead to nonphysical results such as negative pressure or density, as proven in [Wu, 2018] Theorem 4.3. Thus, ensuring the divergence free (DF) condition is key to a physically stable simulation, a plethora of methods exist to ensure DF such as constrained transport, projection methods or divergence cleaning. For a more detailed explanation of each of the methods, we refer the reader to the introduction of [Hopkins, 2016].

3.2.1. Differentiable simulations

End-to-end differentiable simulations have appeared recently as a new paradigm of simulations leveraged by the advancements in automatic differentiation codes such as JAX or PyTorch. Consider an initial discretised state determined by a set of parameters θ , $s_0(\theta)$ of a dynamical system. $s_1(\theta)$ is then obtained by applying the discretized PDE operator $\tilde{\mathcal{P}}$, $s_1(\theta) = \tilde{\mathcal{P}}(s_0(\theta))$. By iteratively applying $\tilde{\mathcal{P}}$, the final state s_n can be obtained

$$s_n(\theta) = \tilde{\mathcal{P}} \circ \tilde{\mathcal{P}} \dots \circ \tilde{\mathcal{P}}(s_0(\theta)) = \tilde{\mathcal{P}}(\tilde{\mathcal{P}}(\dots \tilde{\mathcal{P}}(s_0(\theta))\dots)) = \tilde{\mathcal{P}}^n(s_0(\theta)) \quad (43)$$

where the n exponential signifies repeated application. Assuming a differentiable simulation and given a scalar loss function, $\mathcal{L}(s_n, s_n^*)$ where s_n^* is a target final state, we can

compute $\partial\mathcal{L}/\partial\theta$ and apply any gradient based optimisation method to optimise for θ^* . Examples of applications of these methods are:

- Inverse problems: Given an initial state parametrized by a set of parameters θ and a desired final state we can optimize the parameters such that the initial state evolves to the target final state, such as in [Yang et al., 2025] where a differentiable simulation was used together with a robot arm to infer physical parameters of materials.
- Trajectory optimisation: Typically used in simulations that consist of objects and entities with parameters of their own (e.g. simulations of robots), where the path of such entities is optimised by reducing a cost function. Such as in [Du et al., 2021] where robots were taught to walk on their four legs by minimising a cost function through the differentiable simulation.
- Neural network augmented simulations: Hybrid AI-simulations can be obtained with differentiable simulations where a neural network is embedded with the simulation as a means of improving either performance, accuracy or both, an example of such a neural network integration was used in [Um et al., 2021] where by intertwining steps of the simulation and a corrector neural network more accurate physical statistics could be recovered from low resolution simulations.

For a further review of applications of differentiable simulations, we refer the reader to the [Newbury et al., 2024] review. Differentiable simulations examples can be found in material sciences ([Drysck et al., 2025], [Daviet et al., 2025], [Daubner et al., 2025], [Mahlau et al., 2026]), quantum and particle sciences ([Kaiser et al., 2024], [Guilmin et al., 2025]), astronomy and cosmology ([Horowitz et al., 2025], [Vermariën et al., 2025], [List et al., 2025], [Viterbo and Buck, 2025]) or biophysics [Neymotin and Hazan, 2025] to mention some examples.

3.2.2. Astronomix: differentiable MHD interstellar simulation written in JAX

The simulator used in this work is the MHD simulation code Astronomix [?] (formerly known as jfluids [Storeks and Buck, 2024]). Astronomix implements finite volume sim-

ulations with Lax-Friedrichs, HLL and HLLC Riemann solvers and recently finite difference simulations. Astronomix has a focus on astronomical applications and implements physical modules such as self-gravity, cosmic rays or stellar winds. Completely written in JAX, Astronomix allows for differentiable simulation applications like the previously discussed ones.

JAX [Bradbury et al., 2018] is a Python library which allows for computations with arrays (with NumPy [Harris et al., 2020] like syntax) and allows for Just-In-Time (JIT) compilation. JIT allows for Open XLA compilation of any pure function written in JAX (pure signifies the lack of side effects like print statements), such compilation allows for high speed functions that can run on CPUs, GPUs or TPUs. JAX also allows for automatic differentiation through any pure function, which allows for high performance differentiable simulation pipelines.

3.2.3. Solver-in-the-loop

One common machine learning application on simulations within the literature consists of training a network to emulate the simulation, thus skipping the whole PDE applications, like in [Hao Wu, 2026] where an operator model is trained as a surrogate for turbulence simulations. Such an application, although proven useful, has its shortcomings, especially under long simulations where the errors will grow, making the emulated simulation unphysical. The solver-in-the-loop approach, proposed by Um et al. in [Um et al., 2021], instead, doesn't substitute the simulator but adds an extra term in between applications of the PDE. The model interacts only with the local structures not captured by the simulator and thus acts as a residual term. Consequently, the learning objective is reduced to a drift term in the simulation space towards the target state, as shown in the schematic 3.1. This target state is established at training, and during inference we expect the model to have learned the discrepancies between the simulated and target dynamics.

A simulation can typically be described as an iterative application of the discretised PDE operator $\mathcal{P}$ to an initial state $s_{t=0}$. Given an initial state with resolution HR $s_{t=0|HR}$, if we

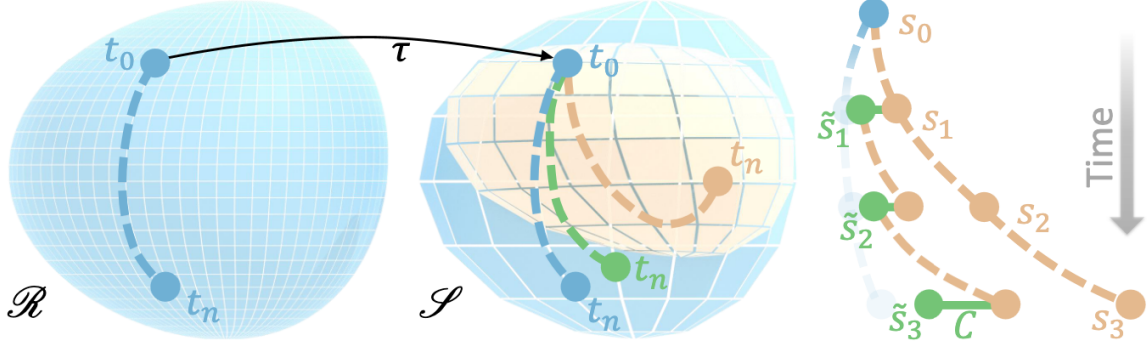


Figure 3.1: Solver-in-the-loop phase space diagram. The left sphere represents the HR phase space; by downscaling the initial state with transformation τ , the state is moved to the LR phase space on the right. Evolving this state results in the low resolution trajectory in orange. Correcting now in between applications of the PDE with corrector $\mathcal{C}$ the green trajectory is obtained. Figure taken from [Um et al., 2021].

want to get the state at time T we can apply the PDE n times ⁷ $s_{t=T|HR} = \mathcal{P}^n(s_{t=0|HR})$, where the exponent signifies iterative application. If now through a nonaffine operation τ we reduce the resolution of the state to LR $\tau(s_{t=0|HR}) = s_{t=0|LR}$ and apply the same time evolution to this state $s_{t=T|LR} = \mathcal{P}^n(s_{t=0|LR})$ we will find that $\tau(s_{t=T|HR}) \neq s_{t=T|LR}$. This is typically expected, and $\tau(s_{t=T|HR})$ might not even be initially accessible in phase space for the low resolution simulation, even more so in chaotic systems the low resolution simulation would be further from the high resolution resolved one in phase space. These can all be attributed to the numerical errors introduced when discretising, and as such it is assumed that the higher the resolution, the closer we are to the actual physical solution of the system.

We can then introduce a corrector term in between applications of the PDE, $\mathcal{C}$, such that the iterative simulation is $(\mathcal{P}\mathcal{C})^n s_{t=0} = \tilde{s}_{t=T}$. The corrector, a neural network, learns the inherent numerical error introduced by the discretisation and changes the trajectory of the simulation in phase space towards the highly resolved downsampled states, as shown in Figure 3.1 green line. Thus $\mathcal{C}$ is tasked with learning a drifting term in the state space only through residual terms. In this way, the model has less responsibility, and

⁷Here we are not going to make the distinction between fixed or variable timestep

we can exploit its full expressive power in achieving better accuracy for a fraction of the performance. Lastly it's worth commenting on how these models are used and trained, when treating the PDE simulation as an output system (from the point of view of the corrector), the training procedure resembles the one from a Recurrent Neural Network [Schmidt, 2019], where the model iteratively gets an input which in our case gets modified afterwards by the discretised PDE operator $\mathcal{P}$, thus known training strategies and overall knowledge from RNNs can be exploited to train solver-in-the-loop (SOL) models.

3.3. MHD type problems

In the following we introduce 3 MHD type problems used to test simulation capabilities: complex magnetic fields structures, wave shocks, and chaotic systems.

3.3.1. Blasts

Shock waves occur in the ISM through a lot of different mechanisms (e.g. supernova explosions, expanding HII regions, accelerated gases due to gravity). Shock waves are disturbances that propagate faster than the sound speed of the medium and originate due to a discontinuity. The conditions under which a discontinuity in the medium turns into a shock wave are given by the Rankine–Hugoniot conditions [Galtier, 2016, Section 6.1].

We are concerned with MHD Blasts as they provide challenging MHD simulations due to the high Reynolds number and plasma β . Blast waves typically occur in supernova explosions both in the initial collapse and afterwards the revived shock. Thus, it's important to understand this problem as it determines the outward flow of mass and energy from supernova explosions.

We take the initial conditions for a MHD Blast from [Mignone and Del Zanna, 2021]. The state is set on a $[0, 1]^3$ box, the initial density and velocity are constant with $\rho_0 = 1.0$ and $\mathbf{v}_0 = 0$, the pressure is constant everywhere $p_0 = 1$ except for a spherical region in the center of the simulation box $p_{center} = 100.0$ with radius $r_0 = 0.125$. The magnetic field is given by $\mathbf{B} = B_0 (\sin \theta \cos \phi, \sin \theta \sin \phi, \cos \theta)$ with $B_0 = 10.0$ and $\theta = \pi/2, \phi =$

$\pi/2$, therefore the ambient plasma $\beta = 0.02$, resulting in a highly magnetic dominated environment. Depending on the implementation a soft gradient from the high density sphere to the low density environment is also implemented.

$$p(r) = \begin{cases} p_0 + (p_{\text{center}} - p_0) \frac{r_1 - r}{r_1 - r_0}, & r_0 < r < r_1 \\ p, & r \leq r_0 \text{ or } r \geq r_1 \end{cases} \quad (44)$$

where p is the initialised pressure before implementing the gradient and $r_1 = 0.1375$. We evolve the system till $t = 0.2$. An example of an MHD blast setup described here is given in Figure 3.2.

The chosen parameters don't fully represent a supernova setup, rather a numerical setup by emphasizing the magnetic effects. Although the setup doesn't represent an accurate supernova progenitor, we capture the basic structure of an expanding supernova blast, a localised high pressure region in a magnetised environment. The resulting shock interacts with the magnetic field to test the magnetohydrodynamic capabilities of the code, as the ambient is highly magnetised compared with a typical supernova ambient.

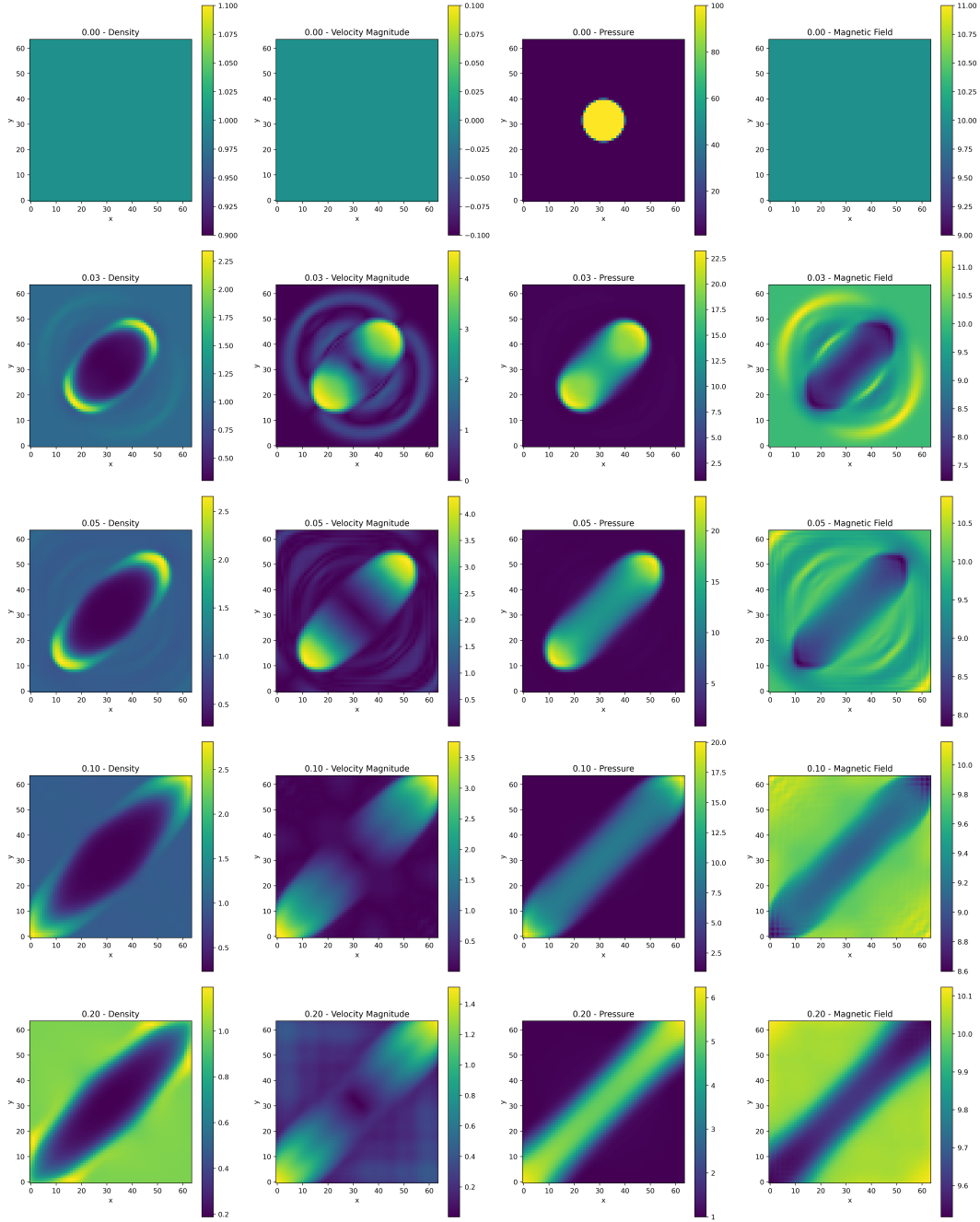


Figure 3.2: 3D MHD Blast solved with Astronomix. Resolution is 64^3 , solver mode is finite volume, and boundary conditions are open. The times shown are $[0.00, 0.03, 0.05, 0.10, 0.2]$, with each time shown in a separate row, to clearly see the initial shock waves. Note that the color scalings are different on each row. The rest of the setup details are given in section 3.3.1.

3.3.2. Orszag-Tang vortex

A key problem in testing the transition to supersonic MHD turbulence is the Orszag-Tang vortex introduced in [Orszag and Tang, 1979]. It also tests the DF condition, MHD shocks and shock-shock interaction [Lopes et al., 2018]. The original problem was written for 2 dimensions, here we expand it to 3, following [Helzel et al., 2011]. The initial conditions are given by a constant density and pressure, $\rho_0 = \gamma^2$, $p_0 = \gamma$, where γ is the adiabatic index, a variable magnetic field $\mathbf{B} = (-\sin(y), -\sin(2x), 0)$, and a velocity field given by 2. The computational domain is $[0, 2\pi]$ in each axis of the box, with a final time of $t = \pi$. An image comparing the initial and final state of the OT vortex can be seen at 3.3.

Table 2: Orszag-Tang vortex: initial condition for velocity field.

	2D	3D
u_x	$-\sin(y)$	$-[1 + \varepsilon_p \sin(z)] \sin(y)$
u_y	$\sin(x)$	$[1 + \varepsilon_p \sin(z)] \sin(x)$
u_z	0	$\varepsilon_p \sin(z)$

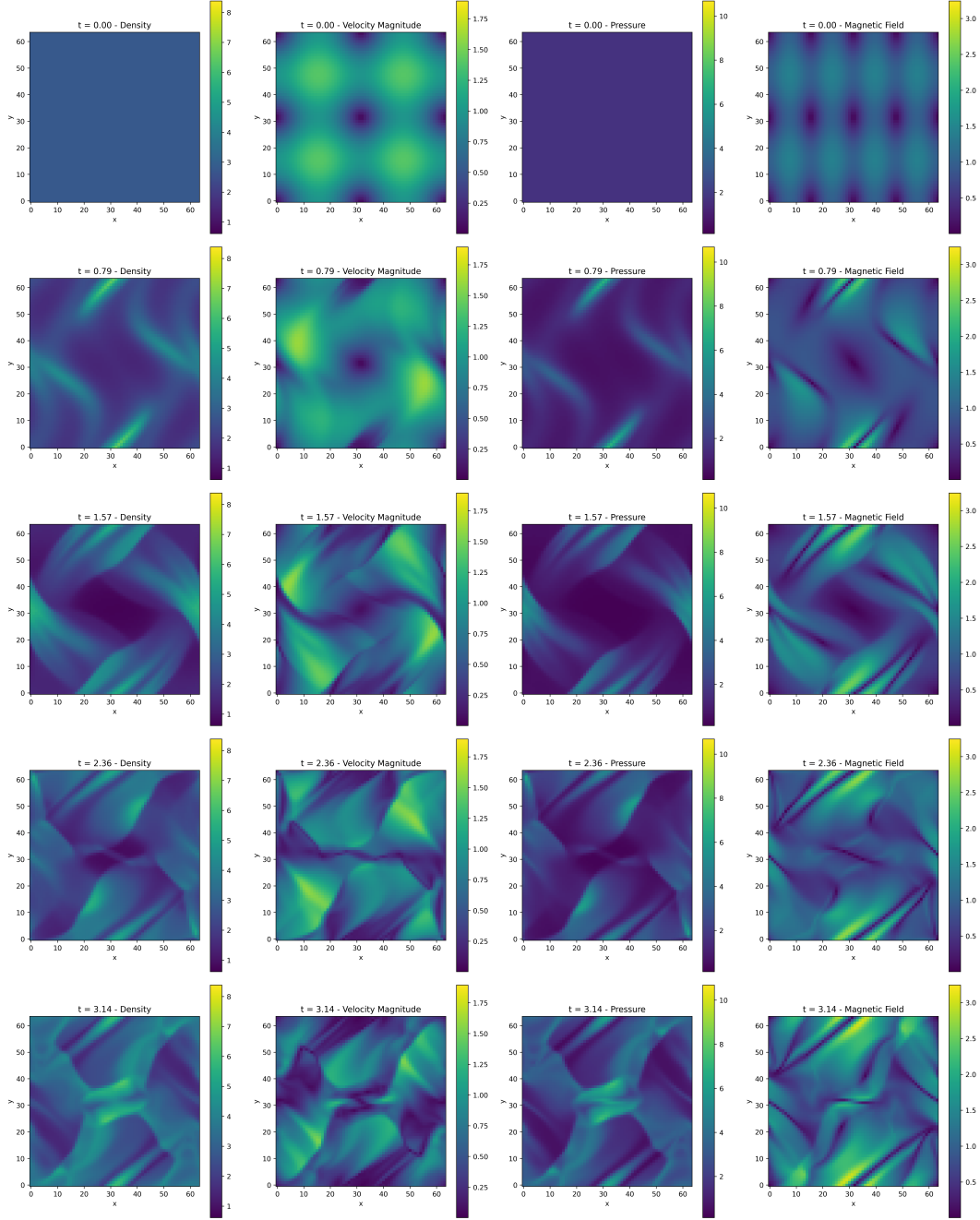


Figure 3.3: Orszag-Tang vortex state evolution at $t = [0, \pi/4, \pi/2, 3\pi/4, \pi]$, with each time shown in a separate row, computed in Astronomix with $\gamma = 5/3$. The simulation was computed in finite difference mode with periodic boundaries and a resolution of 64^3 .

3.3.3. Turbulence

Flow regimes are broadly categorised into two: laminar and turbulent. Considering the Navier-Stokes equations, where the Euler equations are the inviscid approximation of them, turbulence appears when the inertial nonlinear effects of the Navier-Stokes equations dominate over the viscosity effects. The kinematic viscosity of the fluid ν appears in the equation as a term $\nu \nabla^2 v$. The competition between this term and the nonlinear velocity term $v \nabla v$ is what creates turbulence. This competition is parametrised by the Reynolds number $Re = uL/\nu$, where u is the flow velocity, and L is the characteristic length. For a given geometry and viscosity, low enough Reynolds number values correspond to a laminar regime, whereas a high enough value corresponds to a turbulent regime⁸.

Turbulent flows contain a hierarchy of eddies spanning a wide range of scales, from large energy-containing motions to small dissipative structures. The large scales usually contain the most kinetic energy and are anisotropic, a consequence of the external forces that caused the turbulence, and define the characteristic length of the fluid L . The smaller scales are where the diffusion effects take place, this scales are characterized by the Kolmogorov scale η which denotes the smallest scales before the viscosity dominates and the kinetic energy is diffused. Hence there exists an energy cascade from the larger scales to the smallest ones, till the Kolmogorov scale is reached at which point diffusion takes effect and the kinetic energy is transformed into thermal.

Within the inertial range ($\eta \ll r \ll L$, where r is the separation scale), the diffusion effects are negligible, and therefore we can assume a mean energy cascade rate ϵ through the range, which is equal to the diffusion rate and the supply energy rate. Kolmogorov's hypothesis states that velocity fluctuations in the inertial range depend only on r and ϵ , resulting in $\delta u(r) \propto (\epsilon r)$. Hence, so does the energy spectrum $E(k)$,

$$E(k) = C_K \epsilon^{2/3} k^{-5/3}, \quad (45)$$

⁸The exact value under which the fluid transitions from one regime to the other highly depends on the geometry of the problem. In [Mukund and Hof, 2018] they give a value for this transition around 2000 for a pipe geometry.

where C_K is the Kolmogorov constant with a value of $1.5 \sim 1.7$. Therefore, recreating the correct energy spectra is important in numerical simulations of turbulence, as it represents whether the model is able to capture the correct rate of energy cascade across all scales.

In simulations governed by the Euler equations, an explicit physical viscosity term does not exist. Nevertheless, the discretisation of the PDEs introduces numerical diffusion, which acts as a subgrid scale model for energy diffusion at unresolved scales. Hence, we also observe turbulent regimes in these simulations, although the viscosity value is dependent on the simulation's resolution and scheme.

Two approaches exist for modelling subgrid scale diffusion in simulations. In implicit large eddy simulations (ILES) the effect is governed by the numerical diffusion, thus in this simulations the energy cascade and diffusion rate are both physically correct but the scale at which the diffusion happens to occur is wrong [Semenov, 2025]. The alternative is known as Explicit LES, where the sub grid scale diffusion is modelled through a closure term this can be done in an analytical form like eddy diffusivity models, mean dissipation formulations or dynamic procedures (for a more in depth analysis of analytical subgrid diffusion models we refer the reader to [Moser et al., 2021]) or modelled after data based models [Choi et al., 2025].

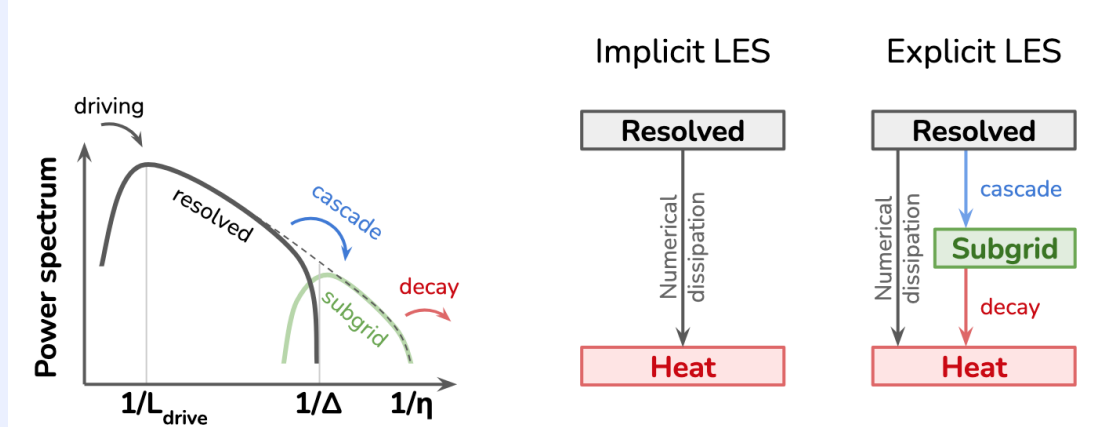


Figure 3.4: Scheme of the energy cascade and diffusion in Explicit and Implicit LES. Image taken from [Semenov, 2025].

Part II.

Results

In this part we are going to describe the computational and analytical methods developed. The main goal is to improve the accuracy of different MHD simulation setups without a significant tradeoff in computational cost. This was done in 2 different approaches:

- **Post-hoc:** In this approach a superresolution model is applied to the final state of a low resolution simulation in order to try to recover statistics such as power spectra and meaningful structures.
- **Ad-hoc:** In this approach a neural network is intertwined with the simulator, learning how to steer it in the right direction in order to get close to a high resolution simulation baseline downscaled to the low resolution grid.

4. Post-hoc approach: Super resolution

Super resolution models aim to transform a low resolution input, typically 2 dimensional 3 channel images captured by cameras, to a higher resolution image that reconstructs the lost information in the downsampling operation. These downsampling operations are typically caused by technical constraints. In the case of traditional cameras their sensors are made of squared light receptors, the density of which determines the final resolution of the image.

In our case the technical constraint is by mere experiment design as the actual resolution of the simulation is chosen by the experimenter at the early design stages and it can be compromised by factors such as time, memory or energy. Thus low resolution simulations are preferred costwise. From a numerical analysis point of view, given a stable and consistent numerical method, the global error will converge at the same rate as the truncation error, the error introduced in a single simulation step. Thus if the truncation

error is reduced as the temporal or grid resolution ($\Delta t, \Delta x$) are reduced ($\Delta t \rightarrow 0, \Delta x \rightarrow 0$), so will do the global error⁹ [LeVeque, 2002, Chapter 8.2]. Therefore higher resolution simulations will be more accurate to the physical model, governed by the PDE, than lower resolution simulations.

Super resolution is an ill defined inverse problem due to the non affinity of the down-sampling operation, this is, an infinite amount of high resolution states could correspond to its low resolution pair. The solution to this ill defined problem is defining priors over the high resolution states, in the case of machine learning this means that our model has to understand the dynamics that result in the HR states and therefore understand the underlying physical model. Therefore we develop neural operator models with physically informed strategies to superresolve turbulent slices. Our approach is a proof of concept for neural operators within the context of turbulence superresolution and it is not intended to be a valid methodology but rather a study of feasibility.

4.1. Dataset

The dataset used consists of 500 pairs of high and low resolution turbulence simulations split into training (400 pairs) and validation (100 pairs) sets, each state of the simulation consists of a 5 channel (pressure, 3 velocity fields and density) 3 dimensional state. Each initial velocity state is obtained by building a random phase Gaussian field with a predefined power spectrum shape in Fourier space (following Kolmogorov's law) between two cutoff wave numbers, to then apply the inverse Fourier transform to it. The velocity field is then scaled to correspond to a predefined mean velocity, in our case this corresponds to 50 km/h.

The pressure and density fields are initialized as constant and uniform with values $\rho_0 = 2m_p \text{ cm}^{-3}$ and $p_0 = (3 \times 10^4 \text{ K}) k_B \text{ cm}^{-3}$, where m_p is the proton mass and k_B Boltzmann's constant. The end time is 10000 years and the physical box is 3 parsecs long in each direction. The integration was done on the finite volume mode of the simulation with a CFL value of 0.4. The high resolution that will be used as the targets for the loss

⁹ $\Delta t, \Delta x$ are related through the Courant–Friedrichs–Lewy condition as $C = u\Delta t/\Delta x$ which for many discretizations is a needed condition to ensure stability

calculation is 128^3 , whereas the low resolution is 32^3 . To store and load the states we use the python library h5py which allows for a convenient and organized storage.

Using the speed of sound formula for an ideal gas $c_s = \sqrt{\gamma P / \rho}$ and $\gamma = 5/3$ we get a value of $c_s = 14.4$ km/s, given the mean velocity we obtain a Mach number of $M = U_{rms}/c_s \approx 3.5$, which corresponds to a highly supersonic regime.

4.2. Considered superresolution models: Fourier Neural Operator and Enhanced Deep Residual Network

Two models were considered and compared, a Fourier neural operator (FNO) similar to the one in section 2.6.3 adapted to superresolution in [Yang et al., 2023] and a ResNet based superresolution model based on [Lim et al., 2017]. We introduce an extra term to the FNO which shifts the Fourier modes acted on, hence we call this model Shifted-FNO, or SFNO.

4.2.1. Shifted Fourier Neural Operator

Starting from the SFNO, the operator G_θ acting on a state $s \in \mathbb{R}^{d_{LR}^3}$ is built as

$$G_\theta(s) := \mathcal{F}_\theta(T^{-1}(f_\theta(s))) \in \mathbb{R}^{d_{HR}^3}, \quad (46)$$

where f_θ is the initial lifting operator, here implemented as a ResNet, T^{-1} is a discretization inversion operator, here implemented as a trilinear interpolation, and $\mathcal{F}_\theta$ is functional operator, here implemented as an FNO following the original paper. The discretization inversion operator is chosen as an interpolation with the assumption that the FNO is complex enough to learn the proper mapping. Going into the detail of each of the parts:

- **Lifting network f_θ :** modelled with a set of standard ResNet blocks with 3x3x3 convolutions, with a ReLu as activation function. Transforms the input to a higher dimensionality space given by the number of hidden channels C_{hidden} , thus $f_\theta(s) \in \mathbb{R}^{C_{hidden} \times d_{LR}^3}$

- **Discretization inversion operator** T^{-1} : interpolates $f_\theta(s)$ in latent space to the target resolution with a variable scaling factor, $T^{-1}(f_\theta(s)) \in \mathbb{R}^{C_{hidden} \times d_{HR}^3}$. Thus, as the interpolation is done on arbitrary target grids without being fixed to a integer scale factor as in the EDSR architecture, the architecture is resolution agnostic and we can say we are talking about a neural operator.
- **Functional operator** $\mathcal{F}_\theta$: modelled as an FNO, learns the mapping from the interpolated high dimensional $T^{-1}(f_\theta(s))$ to the intended dimensionality (3 in our case). It holds the majority of the responsibility in the upsampling operation and as such it has to be expressive enough. The FNO is defined by a series of Spectral Blocks which apply both a spectral convolution and a 1x1x1 kernel convolution which get joined at the end.
- **Projection layer**: a final layer maps back to the input channels, this is done by a 3x3x3 convolution together with a softmax constraint layer from [Harder et al., 2024] to ensure mass and energy conservation. It also allows for a ReLu pass on the density and pressure channel to preserve positivity.
- **Final (optional) skip layer**: an optional skip layer with the initial input upsampled by an interpolation can be applied, delegating the FNO to compute not the whole image but the residual distinctions with the interpolation.

We introduce in this work an extra parameter to the spectral convolution, which controls the region of the Fourier space the learned parameters act on. Typically in a spectral convolution this is not controlled and the parameters act from the 0th wave number to a predefined cutoff value k_{max} , by introducing an extra parameter called k_0 or shifting modes, we modify the region of the Fourier space acted on to $[k_0, k_{max} + k_0]$ and $[-k_{max} - k_0, -k_0]$. The motivation for the introduction of such a parameter lies in the unresolvedness of the finer details in Fourier space, rather than the larger scales. By shifting the action of the network to this finer scales we aim for the network to be able to predict the smaller scales lost in the low resolution simulation. At the same time the coarser scales are preserved by the piecewise section of the Spectral blocks. As we are shifting the region of application of the FNO layer, we call this architecture Shifted-FNO

(SFNO).

4.2.2. Enhanced Deep Residual Network

The CNN model used is based on the Enhanced Deep Residual Network (EDSR) model from [Lim et al., 2017]. Here the network was adapted to allow for 3 dimensional inputs whereas in the original implementation it was for 2 dimensional inputs. The network consists of 3 distinct blocks, the lifting block, feature extraction block and upsampling block, applied in such order. Going on detail of each of the blocks:

- **Lifting block:** A single 3x3x3 convolution lifts the 5 channel 3D input to a state with n hidden channels.
- **Feature extraction:** A stack of residual blocks, each of the residual blocks consists of two 3x3x3 convolutions with a ReLu between them and an identity skip connection. After applying this block another skip connection is applied with the output of the lifting block.
- **Upsampling:** Done by the application of n PixelShuffle layers [Shi et al., 2016], where n is given by the scale factor $s = 2^n$. A PixelShuffle application consists of a 3 dimensional convolution that multiplies the number of channels by 8, and then a pixel shuffle operation which rearranges the expanded channels into 2x2x2 spatial blocks, effectively doubling the resolution. Finally a 3x3x3 convolution brings down the number of channels to the desired one.

Comparing both architectures, the EDSR has 2 clear shortcomings against the FNO. Firstly the EDSR is forced into a scaling factor from the definition, whereas the FNO allows for arbitrary scale factors. Secondly the EDSR is a pure local convolution architecture, whose effective field of interaction is given by the number of residual blocks (2 per residual block application), on the other hand the FNO has a global interaction field due to working in Fourier space which can be exploited for turbulent fields with long correlations.

4.3. Benchmarks: SFNO and EDSR models

After running an Optuna optimisation we find the best parameters for the models considered which are given in table 3. We observe how the best shifting modes found correspond to 8, favouring high wave numbers and ignoring the low wave numbers which are captured by the residual connections. The metrics table for both models and interpolation as a benchmark can be seen in Table 4; the MSE per channel, on the energy spectra and on the vorticity are presented. We observe how the SFNO outperforms the EDSR slightly. The models were trained for 500 epochs and 250 epochs in the case of SFNO and EDSR respectively, each epoch corresponding to computing the whole training dataset on shuffled batches of 64. All the training hyperparameters are presented in appendix A.

Table 3: Model hyperparameters for the SFNO and EDSR architectures.

Parameter	SFNO	EDSR
Input channels	5	5
Feature channels	32	64
Residual blocks	3	16
Operator blocks	2	–
Fourier modes	16	–
Shifting modes	8	–
Kernel size	–	3
Skip connection	True	True
Number of parameters	33.5 mill	5.5 mill

The loss evolution curves of both the SFNO and EDSR model can be seen in Figure 4.2. The SFNO does not show a substantial improvement to the performance of the EDSR, one factor that may contribute to the similar performance is the similarity between the training and evaluation data. Since the data was generated with the same parameters and just changed the initial state velocity distribution, the performance on the evaluation data does not represent the ability of the model to generalize to other regimes, therefore, the potential improvement of the SFNO may not be reflected in this evaluation. We observe a slight overfitting for the SFNO, this can be explained due to the high amount of parameters compared with the EDSR. In order to observe a larger difference between the two models the dataset should span a larger range of parameters and the validation

Table 4: MSE metrics for $\times 4$ upsampling.

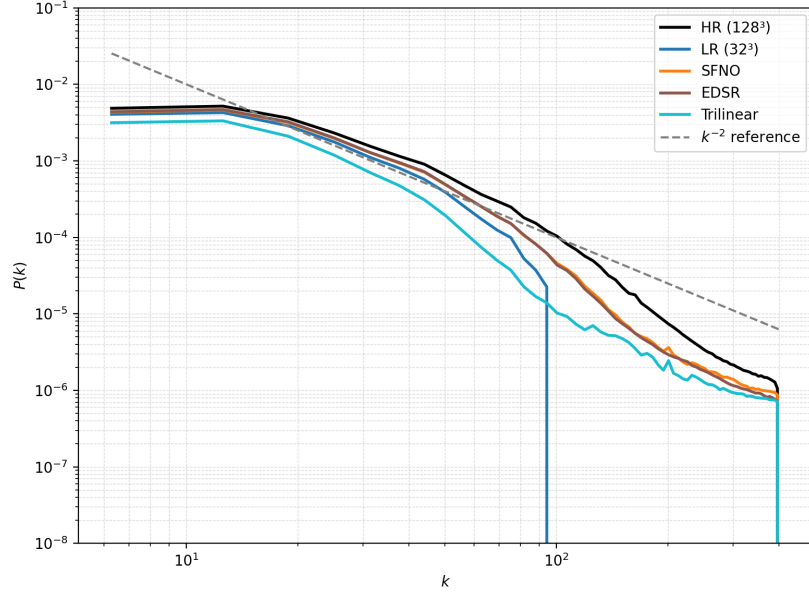
Metric	EDSR	SFNO	Trilinear
MSE	0.008 47	0.00808	0.037 16
MSE Pressure	0.000 31	0.00028	0.001 63
MSE Density	0.038 15	0.03643	0.162 08
MSE v_x	0.001 28	0.00121	0.007 33
MSE v_y	0.001 30	0.00124	0.007 38
MSE v_z	0.001 29	0.00124	0.007 40
MSE $\ v\ $	0.001 48	0.00141	0.008 16
MSE Vorticity	0.001 07	0.00100	0.002 84
MSE Energy Spectra	0.066 39	0.04987	0.217 37

data contain simulations generated with unseen parameters. Nevertheless we still see improvement from the SFNO to the EDSR. A snapshot and energy spectra comparison for both models and interpolation can be seen in Figure 4.1.

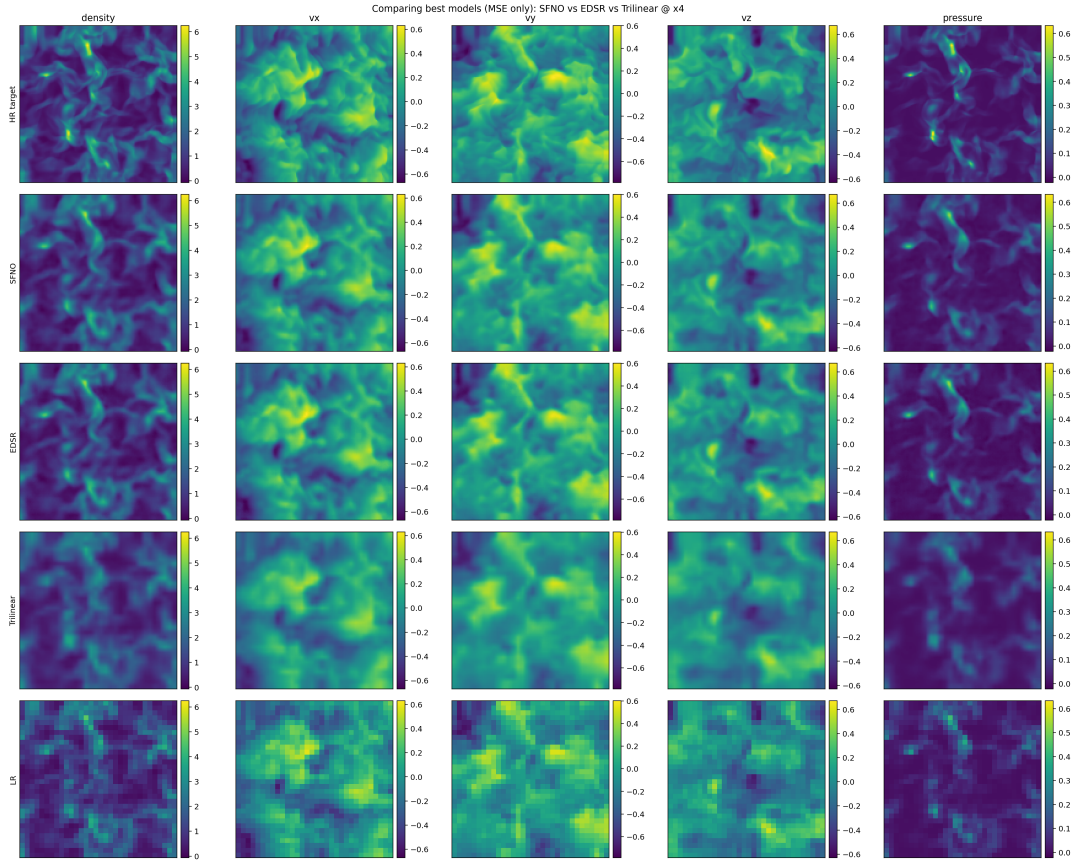
4.4. Metrics study: L1 and spectral loss

In the previous section the models were trained with an MSE loss, although suitable for most machine learning problems in the case of superresolution MSE averages out the details and doesn't allow the model to learn sharper edges. Thus we consider two different metrics to train on: the L1 loss and a spectral MSE from the velocity spectra. This last one is physically motivated from section 3.3.3 and the Kolmogorov spectra. The L1 loss is motivated by [Zhao et al., 2018] in which it is claimed that in order to achieve perceptually better and sharper images L1 losses are better posed than L2 based losses.

We test these additional losses by adding each as an extra term to the MSE with a weight that represents 10% of the MSE. 3 models were trained, 2 with MSE and each of the extra losses (L1 and Spectral) and one with the 3 losses. The results can be seen in table 5. The combination of MSE and L1 seems to yield the best MSE value by a small margin whereas the MSE and spectral loss combination yields similar values to the other models while improving the performance on the spectral MSE. This can be

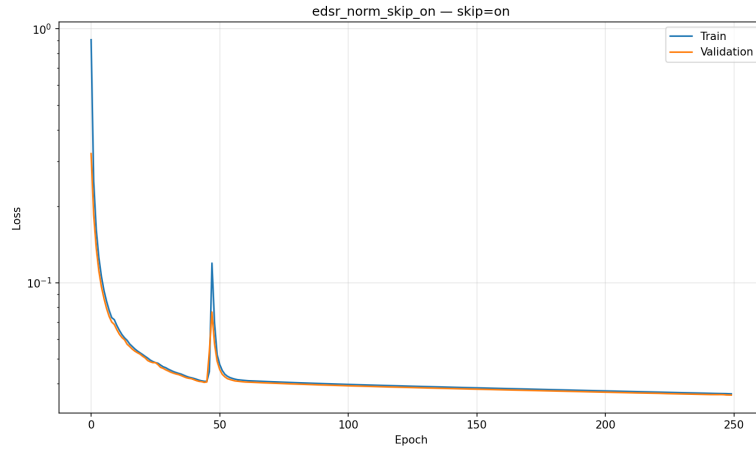


(a) Spectra comparison of a high resolution end state and low state and superresolved states from interpolation, an EDSR model and an SFNO model.

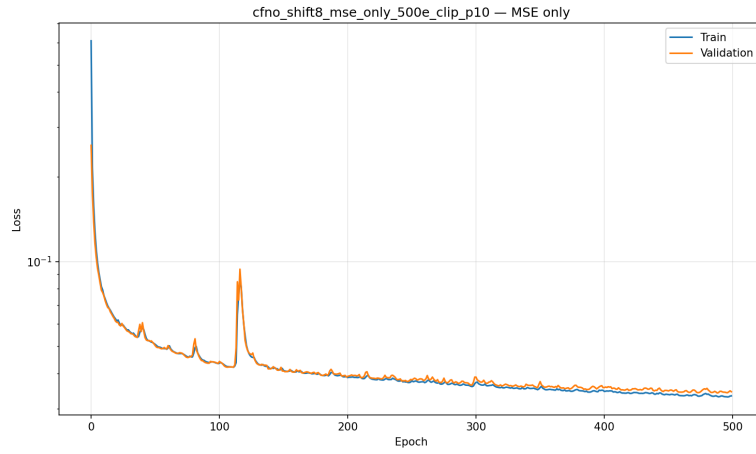


(b) Snapshot slice of a turbulent superresolved state with interpolation, an EDSR model and an SFNO model.

Figure 4.1: Spectra and snapshot comparison of SFNO and EDSR superresolution models.



(a) Loss evolution of EDSR model trained with parameters from table 3



(b) Loss evolution of FNO model trained with parameters from table 3

Figure 4.2: Loss evolution of both FNO and EDSR models trained for superresolution of turbulent states.

Table 5: Comparison of loss functions for the SNO model with $\times 4$ upsampling. Lower is better.

Metric	MSE	MSE + Spectral	MSE + L1	MSE + Spectral + L1
MSE	0.008 08	0.008 20	0.00799	0.008 50
MSE Pressure	0.000 28	0.000 29	0.00028	0.000 30
MSE Density	0.036 43	0.036 92	0.03609	0.038 27
MSE v_x	0.001 21	0.001 26	0.00118	0.001 32
MSE v_y	0.001 24	0.001 25	0.00120	0.001 31
MSE v_z	0.001 24	0.001 28	0.00120	0.001 31
MSE $\ v\ $	0.001 41	0.001 41	0.00136	0.001 47
MSE Vorticity	0.001 00	0.001 00	0.00099	0.001 03
MSE Spectra	0.049 87	0.03274	0.046 71	0.040 42

observed in Figure 4.3, although the effect is rather light. In the following we train on the MSE + Spectral loss as for a little trade off in MSE the velocity spectra improves considerably which is one of the most important statistics we would like to recover.

4.5. An expansion to the SFNO model: U-Net SFNO

We test another architecture from [Wen et al., 2022] in which an extra path to the spectral block is added, this time in the shape of a U-Net, hence the name UFNO. In the original proposal both spectral blocks and U-Net spectral blocks were used, in our case as our model consists of 2 spectral blocks we substitute both. A scheme for a UFNO can be seen in Figure 4.4, we illustrate the shifting of the modes in the spectral blocks, resulting in the architecture we refer to as U-SFNO.

The model is trained following the same parameters as the SFNO models from the previous section. The loss used is MSE + Spectral. The resulting metrics on the evaluation dataset can be seen in table 6. The U-SFNO model outperforms the two other models in every metric, when comparing a slice of a snapshot and its energy spectra, shown in figures 4.5 and 4.6. Despite the improved performance of the U-SFNO, all three models considered preserve the overall structure of the flow while struggling to accurately recover the sharpest peaks and fine scale features.

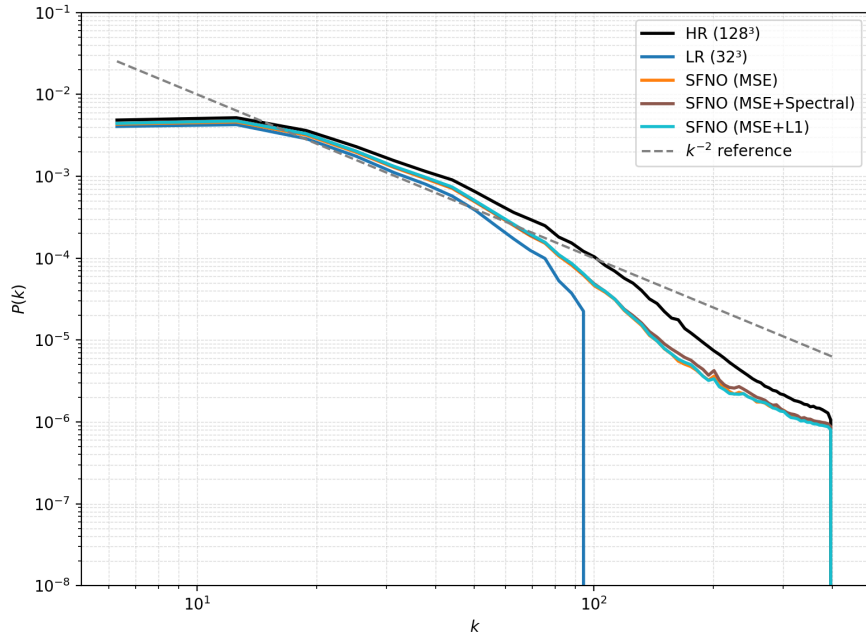


Figure 4.3: Spectra comparison of SFNO models trained with different losses, MSE was used as basis and 2 extra terms are studied. An spectra based loss to physically inform the model and an L1 loss to recover sharper edges.

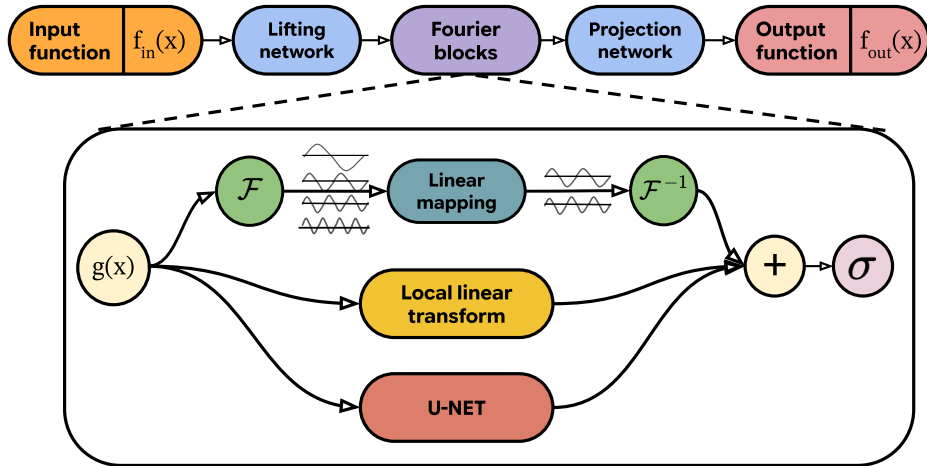
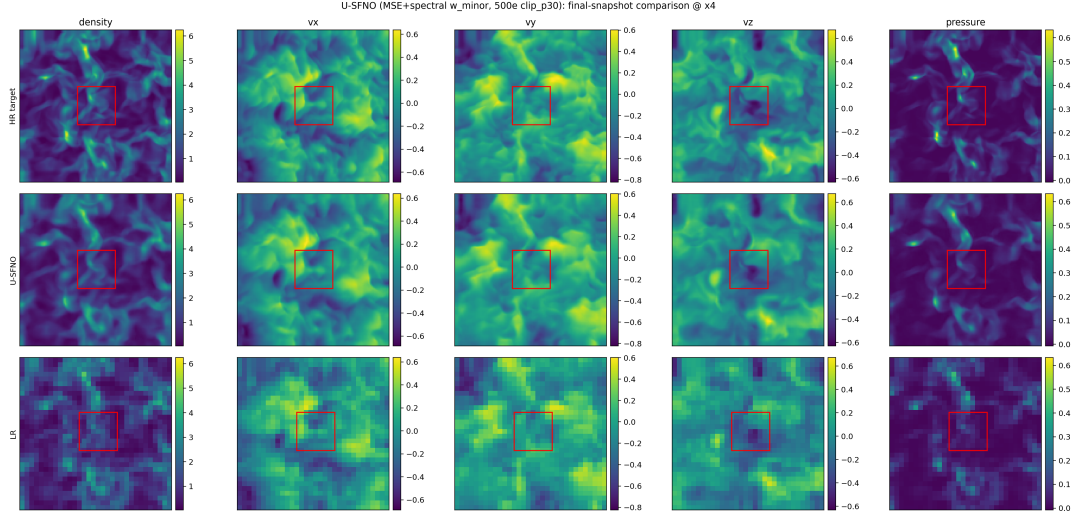
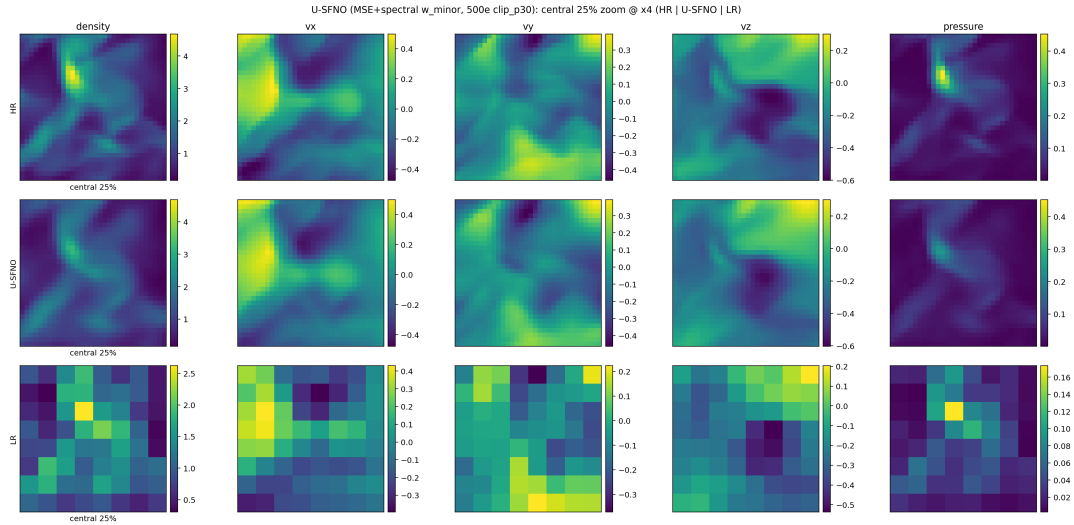


Figure 4.4: U-SFNO full architecture. Similar to a FNO architecture, the state is lifted to a higher dimensional space, followed by the application of the spectral blocks and later transformed back to the original dimensionality. The difference lies in the spectral blocks, which now incorporate an extra path encoded by a U-Net.



(a) Turbulent snapshot slice of a high resolution, low resolution and superresolved states with the U-SFNO model. The red rectangle represents the zoomed in region shown in the figure below.



(b) Zoomed in slice of a turbulent snapshot. Presented here are the high resolution simulation, superresolved and low resolution input of the SR model, in this case a U-SFNO.

Figure 4.5: U-SFNO application on a slice of a turbulent snapshot. In both plots HR (top row), U-SFNO augmented (middle row) and LR (bottom row) are presented.

Table 6: Comparison of spectral loss for operator networks with $\times 4$ upsampling. Lower is better.

Metric	SFNO MSE	SFNO MSE + Spectral	U-SFNO MSE + Spectral
MSE	0.008 08	0.008 20	0.00613
MSE Pressure	0.000 28	0.000 29	0.00020
MSE Density	0.036 43	0.036 92	0.02767
MSE v_x	0.001 21	0.001 26	0.00090
MSE v_y	0.001 24	0.001 25	0.00094
MSE v_z	0.001 24	0.001 28	0.00093
MSE $\ v\ $	0.001 41	0.001 41	0.00104
MSE Vorticity	0.001 00	0.001 00	0.00074
MSE Spectra	0.049 87	0.032 74	0.02407

4.6. Discussion

Superresolution for turbulent hydrodynamical setups has been tested on a dataset covering a limited amount of parameters with succesful results. Overall the tests presented here are rather reduced and a full fledge test would need generalization results and a larger dataset. Still as an early test super resolution has shown to work under a turbulent setup for interstellar medium applications. In climate sciences super resolution driven methods, referred as downsampling, have been proven to work under the chaotic system that is the earth’s climate [Najafi et al., 2026]. More recently super-resolution methods have also been developed for turbulent setups with NO approaches [Oommen et al., 2025] and [Abid and San, 2026]. Thus as a first approach such an application to the ISM was expected to work. A full comparison with emulator based models is also needed to ensure which of the two methods gives better results on long simulation times.

Commenting on the overall performance, we’ve seen a well known problem with super-resolution setups, the lack of finer structure. Although we tried to circumvent it with the FNO and the shifted fourier modes it still appeared due to using MSE based losses. Further study and optimisations are needed to the model and training procedure. In the model aspect diffusion and transformer based models have been proven to recover finer details and sharper edges than traditional networks or neural operators. Still staying in

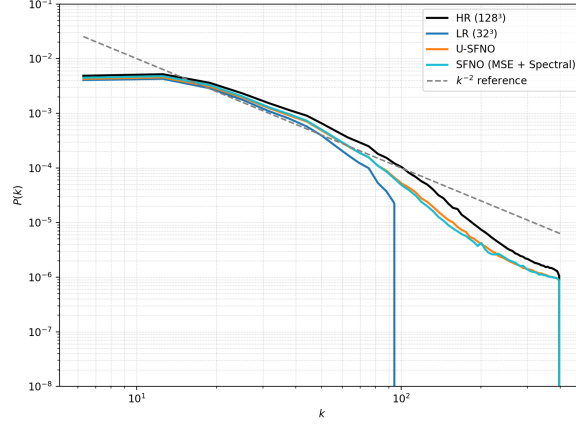


Figure 4.6: Energy spectra comparison of a high resolution and low resolution turbulent states with the U-SFNO model and the SFNO model both trained with an MSE + Spectral combined loss.

the neural operator regime, other approaches using FNOs exist like in [Qin et al., 2024] or [Abid and San, 2026] both multiscale approaches. Other techniques to be studied are the use of gradient based losses or other combinations of losses that might help recover the finer details or adversarial training strategies as in GANs.

As to the effectiveness of a the neural operator against a normal neural network architecture we can comment on the fact that such a model allows for other increments of superresolution beyond the ones it was trained on. Thus one could train a model for 4x superresolution and use it for 2x out of the box or fine tune it even more, which would allow for multiscale methodologies, although this needs further testing. It was also shown the efficiency of the shifted fourier modes which allow the model to capture finer details in fourier space which are key for turbulence.

5. Ad-hoc approach: solver-in-the-loop

Following section 3.2.3, we develop solver-in-the-loop models for different MHD type problems.

5.1. Considered model: Time independent, divergence-free CNN

The model considered throughout this section consists of a simple CNN with inductive biases motivated by physical and simulation constraints. Depending on the solver mode, finite volume or finite difference, the input state contains either 8 or 11 channels. The state is passed through the network, which consists of a series of convolutional layers intertwined with activation functions. Afterwards, the state is lifted again to the original number of channels. Further, the magnetic field correction is transformed by applying the curl operation, ensuring the magnetic field divergence is 0. The correction is then multiplied by the timestep, making the model time independent by learning the time derivative of the correction. Finally, the resulting correction is added to the initial state. A scheme of the model can be seen in Figure 5.1.

Commenting on the solver modes, Astronomix implements both finite difference and volume. Depending on which is used, the model acts in slightly different ways:

- **Finite volume:** in the finite volume mode, 8 channels are used (density, pressure, 3 velocity fields and 3 magnetic fields), and all of them are used during the correction and loss calculation.
- **Finite difference:** in the finite difference mode, Astronomix saves an extra vector field to the state. This vector field stores the intermediate cell values to compute fluxes between differences later. The field is obtained from the magnetic field centre values; therefore, the intermediate fluxes are neglected when computing the correction and are afterwards computed from the magnetic field centre values. Equally, the fluxes are not considered when computing the losses.

The CNN model simplicity is deliberate; in this fashion, we minimise the performance decrease of the simulation. More complex architectures could be considered, but a thorough evaluation of their performance is beyond the scope of this work. It is also important to note that the inductive biases introduced into the model reduce its expressiveness [Cazenavette and Lucey, 2021]. In this case, three constraints are imposed on the space of possible corrections. First, the magnetic field correction is computed as the curl of the network output, ensuring that it is divergence-free. Second, ReLU activation

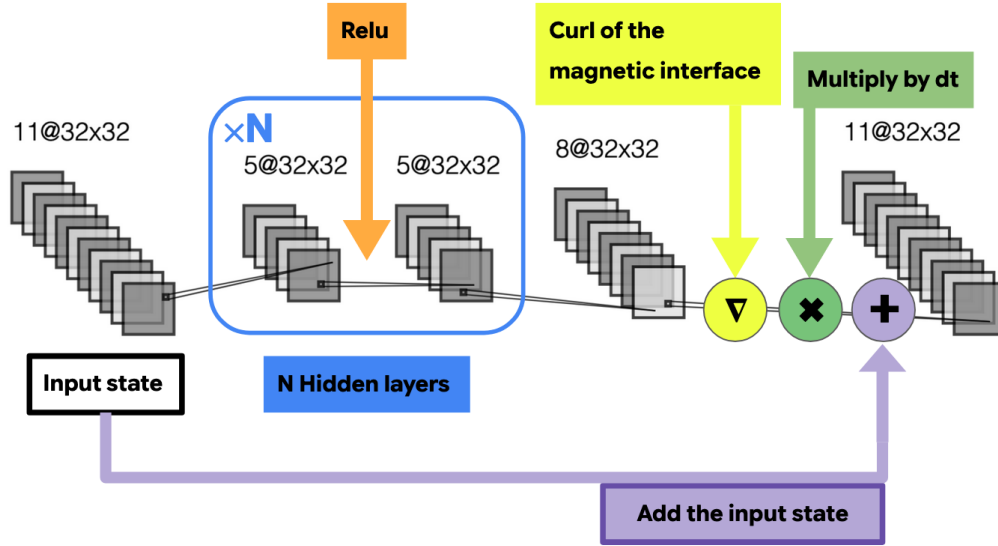


Figure 5.1: The model considered for solver-in-the-loop applications. The model predicts a divergence-free and time independent correction to the state by computing the curl of the magnetic field correction and predicting the time derivative of the correction.

functions are used for the density and pressure fields, enforcing positivity and preventing nonphysical negative values. Finally, predicting the correction time derivative further constrains the space of possible corrections. Nevertheless, considering the scale of the network in this case, the biases encode key information not only for the network but also to ensure the simulations' stability.

5.2. Training pipeline and stability mechanisms

Following the notation introduced in section 3.2.3, we described the basic training procedure used in the following sections. Commenting first on the technical aspects, the simulation runs in JAX, and its API allows for the gradients and models to be fully written in JAX aswell. Nevertheless, a series of wrappers which implement neural network capabilities with similar APIs to those of established neural network libraries such as PyTorch exist, examples of such wrappers are Haiku [Hennigan et al., 2020] (which was replaced by Flax [Heek et al., 2024], both developed by Google) or Equinox

[Kidger and Garcia, 2021]. Out of these, we use Equinox, which implements PyTorch like APIs and advanced techniques to manage PyTrees.

Given an initial state, a high resolution simulation is computed that will be used as a reference. The downscaling transformation τ is then applied to both the initial state $\tau(s_{t=0|HR}) = s_{t=0|LR}$ and the reference states $\tau(s_{t=T|HR}) = r_{t=T}$. Then, during training, the low resolution initial state is used to initiate the solver-in-the-loop enhanced simulation, thus obtaining the final low resolution enhanced state $\tilde{s}_{t=T|LR}$. The loss is then computed between the reference state and the final low resolution enhanced state. Exploiting now the differentiable nature of the simulation, the loss is backpropagated through the simulation to obtain the gradients with respect to the network parameters. This procedure is showcased in Figure 5.2.

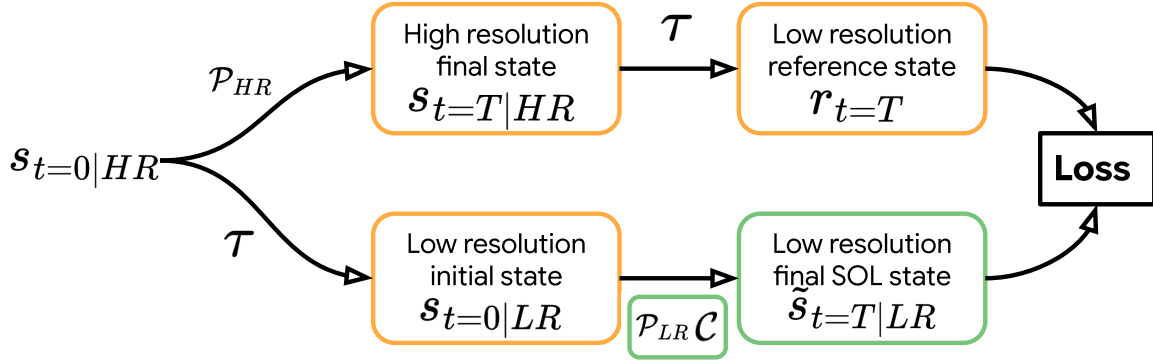


Figure 5.2: solver-in-the-loop training procedure. An initial high resolution state $s_{t=0|HR}$ is both propagated forward in time $\mathcal{P}_{HR}^n(s_{t=0|HR}) = s_{t=T|HR}$ and downsampled to a lower resolution $\tau(s_{t=0|HR}) = s_{t=0|LR}$. The final high resolution state is then downsampled and used as the reference state during the loss calculation $\tau(s_{t=T|HR}) = r_{t=T}$. Finally the initial low resolution scale is used to initiate the low resolution SOL corrected simulation $(\mathcal{PC})_{LR}^n(s_{t=0|LR}) = \tilde{s}_{t=T|LR}$ and the loss is computed and backpropagated to optimise the network weights.

One important caveat of training SOL models is ensuring the stability and physicality throughout the simulation. Since any unphysical realisation of the network during training invalidates the corresponding training run, several measures are taken to promote stable training. The architecture used, introduced in the previous section, promotes

stability by incorporating safeguards and inductive biases. Still, gradients can explode and lead the simulation quickly to unphysical results. Hence, the model has to be initialised with rather small weights, in our case, we scale the initial weights (typically randomly assigned) by a factor of 0.02. In addition, the learning rate scheduler follows an initial linear warm up followed by cosine decay to ensure a well-posed and stable loss environment. Lastly, we apply extra noise to the initial state to ensure regularisation. Nevertheless, most of the training stability depends on the problem chosen to train on, for instance, noise-based regularisation strategies are not suitable for turbulent setups due to their chaotic nature.

A normalised mean squared error is used as the de facto loss for the following experiments. The normalisation is done on a per state basis using the reference state. In our experiments, we tried other losses, such as physically informed losses like total energy losses or, in particular for turbulence, energy spectra or vorticity-based losses. However these alternatives didn't improve the model's performance due to training instabilities and complexity. For multiproblem experiment setups, the loss formulation was kept simple. As each of the initial states, when run through the training pipeline, returns a gradient, thus making this a multiobjective problem already. Further studies on losses with solver-in-the-loop models are needed.

5.3. Multiproblem API

We develop an API that supports iterative model training across multiple problems and facilitates the implementation of different type problems with custom parameters. The API works by defining an abstract class `BaseProblem`, which specifies the common interface that every simulation problem must implement. It defines the following properties and methods:

- **name** (abstract property): a property returning the unique name of the problem, used to identify it within the problem catalogue.
- **t_end** (abstract property): a property specifying the final simulation time.
- **generate_initial_state(SimulationConfig, SimulationParams)** (abstract

method): generates the initial simulation state together with the corresponding simulation configuration, parameters, and registered variables.

- `get_hyperparams()` (abstract method): returns a dictionary containing the problem-specific hyperparameters used for configuration and deterministic naming.
- `filename` (computed property): returns a deterministic filename from the problem name and parameters, allowing for unique identification of each problem’s instance.

Each problem instance is then represented by a `ProblemDescriptor` object, which stores its name, parameters and other fields that facilitate easy training and iterative experimental design. The `ProblemDescriptors` objects are aggregated into a list, which is later used to define an instance of the `ProblemManager` class. `ProblemManager` generates the high resolution - low resolution training pairs, provides methods for storing and loading lists of `ProblemDescriptor`, obtains evaluation data and allows for random problem sampling.

The `BaseProblems` that will be used in the following are:

- MHD Blast: following section 3.3.1, the parameters that define a blast instance in the API are `B_0` and `B_direction` which determine the magnitude and direction of the magnetic field, and `r_0` which defines the radius of the high pressure sphere at the centre of the simulation box. In the following, we characterise `B_direction` by the spherical coordinate angles (θ, ϕ) as the code normalises the vector.
- OT vortex: following section 3.3.2, the parameters that determines the OT vortex are ϵ , which defines the perturbation in the field perpendicular to the vortex plane, `vortex_axis`, which defines the axis perpendicular to the vortex plane, and `parity`, which defines the rotation direction of the vortex.

Each problem considered during training returns an optimisation gradient. Therefore, when considering multiple problems within an epoch, the task becomes a multiobjective, which requires gradient accumulation strategies. In the following, we use the `CONFIG` multiobjective method [Liu et al., 2025], which ensures conflict-free gradient updates via

positivity enforcement of the dot product between the final gradient update and each of the individual gradients.

5.4. Experimental evaluation

In this section, we present the experimental evaluation of solvers-in-the-loop across five setups: a single field output baseline, a temporal generalization study using curriculum learning, and three multiproblem experiments consisting of a dual-problem setup, a rotation-generalization study and a six-problem suite.

5.4.1. Single field output model

In this experiment a single field correcting SOL model is trained and further optimised. This experiment was initially designed as a lightweight SOL test, thus low and high resolutions 16^3 and 32^3 are used, further in the following 4 experiments the resolutions are increased. A single problem is used to train on: an MHD blast with the same specifications as in 3.3.1: $B_0 = 10.0$, $\theta = \pi/2$, $\phi = \pi/4$ and $r_0 = 0.125$. Noise injection is applied at the initial state for regularisation. As a learning rate schedule, we use the cosine warmup learning rate with an initial value of 8.5×10^{-5} , a peak learning rate value of 4×10^{-3} and an ending value of 3×10^{-5} , with the warm up regime occupying 25% of the training. The target state is defined at $t_{end} = 0.2$.

We obtain the best hyperparameter combination by running an Optuna optimisation with 100 studies. The best objective results found are a magnetic field only output model with 24 hidden channels and 2 hidden layers. The training loss evolution can be seen in Figure 5.3. In Figure 5.4 the diagonal profiles at the training target time can be seen. The evolution of the MSE over simulation time can be seen in figure 5.5, here two different phenomena can be observed:

- Observing the dashed red line, which is the error of the low resolution simulation (without solver-in-the-loop) when initialized from the last state used to train the SOL model ($t = 0.2$). It shows a better performance throughout the simulation than the SOL-enhanced simulation. This suggests that the SOL model does not

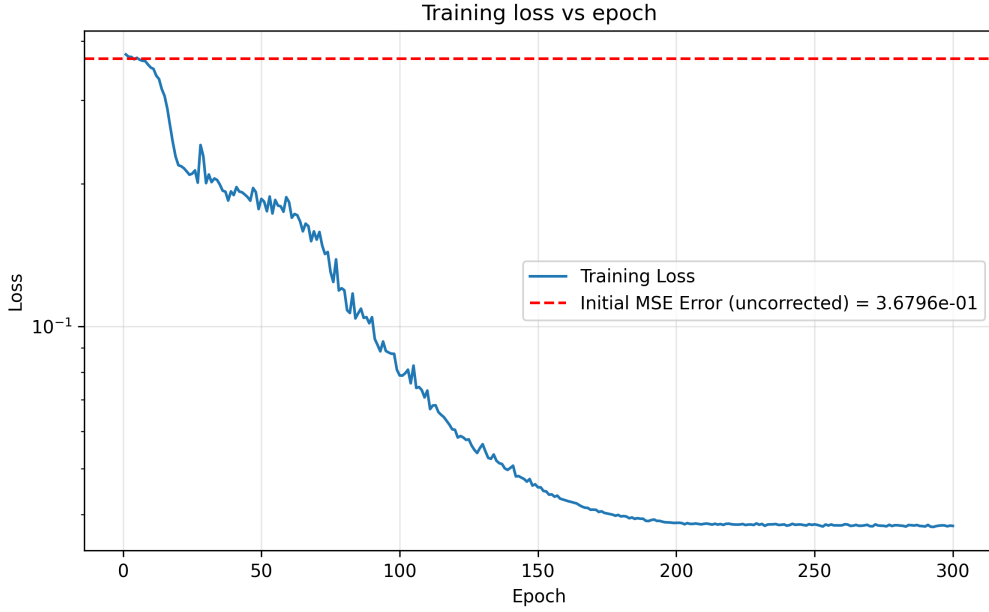


Figure 5.3: Loss epoch evolution of the magnetic field only output model trained on a single MHD blast initial state with low resolution 32^3 and high resolution 64^3 .

generalise well onto unseen simulation states. This comparison is typically omitted from SOL simulation loss over time plots. However here the results highlight the including it of showing as it might reveal a lack of generalisation on to unseen simulation states.

- An abrupt increase in the error of the SOL simulation can be observed at the early stages of the simulation. One possible explanation is overfitting in both phase space and state space. Within the phase space, given a non-chaotic problem such as the MHD blast, where the initial state strongly determines later states, the model learned to modify mostly the initial state. In this case, the stability of the problem allows for such a procedure. Besides, as the model is trained on one single initial state, the early stages of the simulation, remain relatively similar across epochs. As a result these states are encountered more frequently by the network during training. Thus, it is the easiest path of optimisation for the network. Both these effects combined could explain the sharp peak at the beginning of the simulation. Hence, this signifies the need to augment the problem and initial state dataset

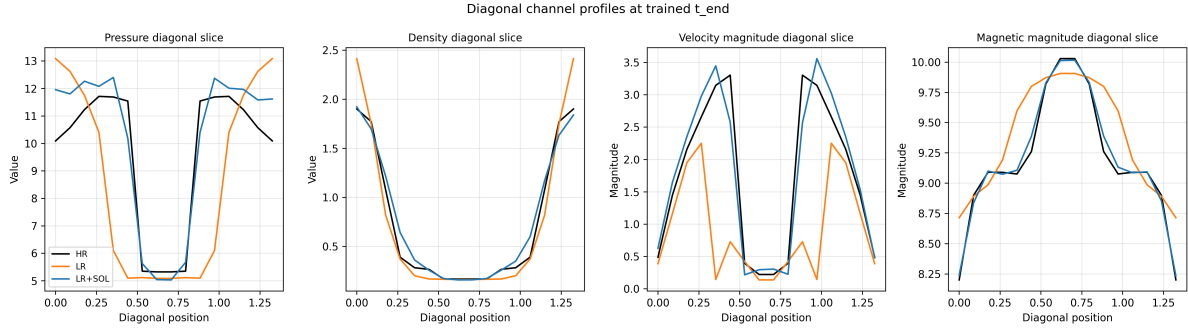


Figure 5.4: Diagonal profiles from the lower left corner to the upper right corner in the x,y field with a fixed z at the middle of the simulation box at target time $t = 0.2$. The black line is the high resolution state, the orange line is the low resolution and the blue line is the SOL enhanced low resolution. The model successfully captures the high resolution structures.

pool, as well as other training strategies.

The model was trained on an Nvidia RTX 2080, and the whole training took 25000 seconds (7 hours), approximately 70 seconds per epoch. We note the time complexity that training such models entails, especially in 3D scenarios. Furthermore, the resolutions used (32^3 and 16^3) aren't representative of real scientific scenarios due to their lack of resolution and accuracy. Nevertheless, from the loss training evolution 5.3, it's apparent that an early stopper would have reduced the number of epochs to 250, however, the discussion still holds. We extract that solver-in-the-loop models are resource costly and need further memory and time optimisation techniques.

Following section 3.3.1 it was expected to obtain a model that prioritizes the magnetic field correction as the blast setup used was designed to test the magnetic field capabilities of the simulation, thus a large portion of the error originates from the unresolved magnetic field. In the following sections, we explore the same setup with full models to observe which other field (beyond the magnetic field) is corrected the most.

5.4.2. Curriculum learning

Curriculum learning is a machine learning technique in which the difficulty of the training set increases as the training progresses. To apply such a technique, a measure of

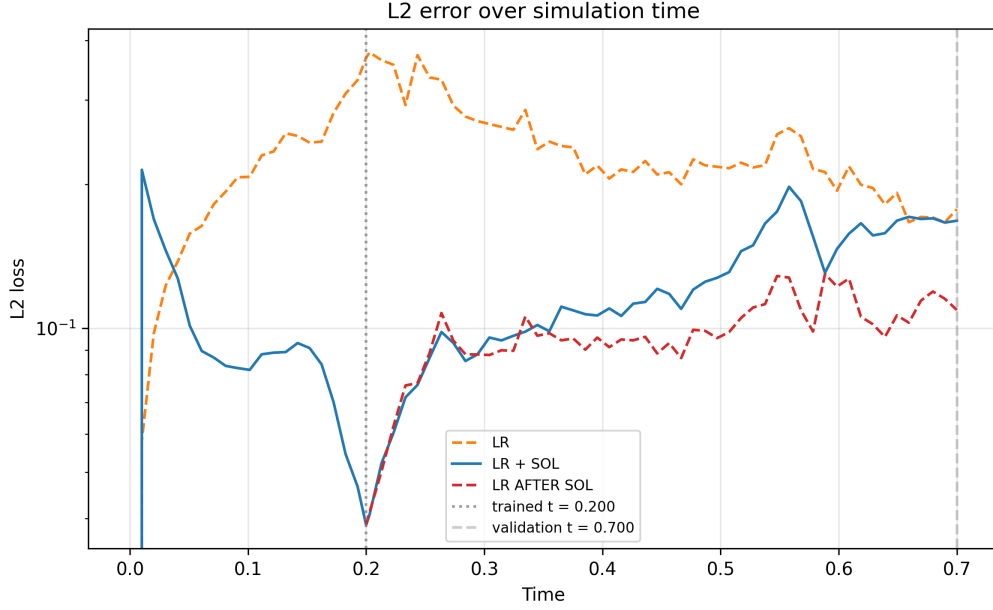
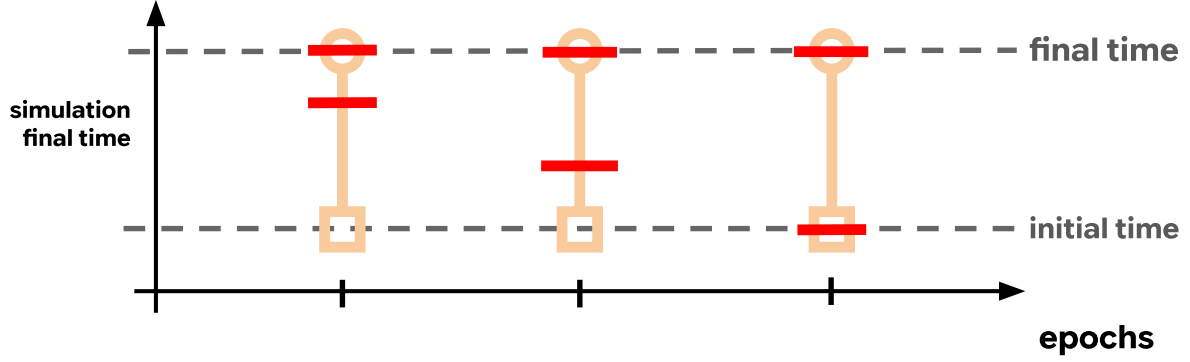


Figure 5.5: MSE error over simulation time of the magnetic field only output model for an MHD blast problem. The orange line is the error of the low resolution model, the blue line is the error of the low resolution SOL enhanced simulation, and the red line is calculated by taking as the initial state the last state at which the SOL was trained and simulating without the SOL. Hence, if the blue line goes over the red line, it signifies that the model is not able to generalise over unseen regimes.

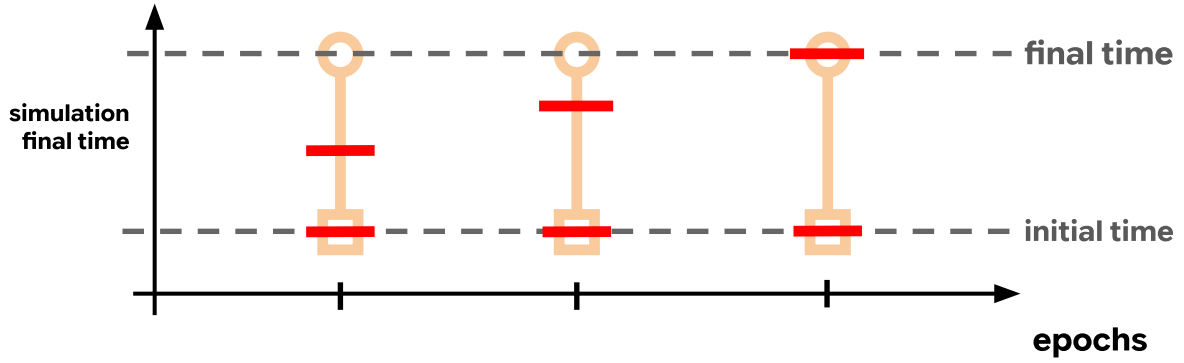
”difficulty” is needed that works on a case by case basis. In our case, as the problem we are training on (MHD Blast) is rather stable, we are going to increase the simulation time as the training progresses. This can be done in 2 fashions, which are represented in Figure 5.6:

- Front to Back: Keeping the initial time constant and moving the end time further away.
- Back to Front: Keeping the end time constant and moving the initial time earlier.

We train 3 models on an MHD Blast with low resolution and high resolution, 32^3 and 64^3 . These resolutions are twice of those used in the previous section, 16^3 and 32^3 . Besides from the curriculum learning, the rest of the training setup is taken from the previous section, including learning rate values and problem setup. The different curriculum learning strategies used are:



(a) Back to Front curriculum learning approach. The end time is kept fixed and the initial time is moved earlier as the training progresses.



(b) Front to Back curriculum learning approach. The initial time is kept fixed and the final time is moved later as the training progresses.

Figure 5.6: Curriculum learning solver-in-the-loop approaches. The red lines signify the portion of the simulation used to train on.

- No curriculum learning (No CL): Our baseline model, training with the whole simulation length through the whole training.
- Front to Back (FTB): Split into 3 different regimes, with a fixed starting time and 3 different end times $t_{end} = (0.1, 0.15, 0.2)$ on these epochs = (150, 150, 200) respectively.
- Back to front (BTF): Split into 3 different regimes, with a fixed end time and 3 different starting times $t_{initial} = (0.1, 0.05, 0.0)$ on these epochs = (150, 150, 200) respectively.

The loss curves for each of the models can be seen in Figure 5.7, we kept the 500 total epochs and no early stopping for a more direct comparison. Using an early stopper, we would have seen an earlier convergence (in training time) of both the BTF and FTB, as when training with less simulation time the backpropagation step length is reduced significantly.

The error evolution over simulation time can be seen in Figure 5.8. Commenting first on the target point, we can see that both FTB and No CL outperform BTF, which signifies the reduced number of epochs the BTF saw the target time. Afterwards, we can look at the initial error. Interestingly the FTB has the lowest initial error introduced, as it saw the initial state for fewer epochs; in that way, the FTB poses as a good strategy to avoid the overfitting in phase space present in each of these models. Considering the error tail beyond the target time, we observe that the No CL model produces little to no correction of the simulation error. This is further supported by the model outputs shown in Figure 5.9, where the magnitude of the corrections produced by No CL is approximately one order of magnitude smaller than those produced by the FTB and BTF models. Consequently, while both BTF and FTB actively attempt to correct the simulation beyond the target time, these corrections worsen its accuracy, whereas the much smaller corrections from No CL have a negligible effect.

BTF and FTB pose as suitable strategies to circumvent overfitting, which is to be expected from the proposed solution of [Mikhaeil et al., 2022]. Where, in the case of chaotic problems for RNNs, it is proposed to sparsely substitute the inputs of the network for the real values while training. BTF and FTB are similar to this strategy with some caveats, we change the inputs in between epochs, whereas the proposed methodology involves changing the input state during the forward pass of the network. Besides, although the referenced work studies chaotic systems, here we can invert the logic: instead of chaotic systems, we can think of RNNs overfitting to stable problems and apply the same solution, although in the case of curriculum learning this is done in a discretised fashion. Still, the No CL by avoiding to correct the state beyond the target time actually performs better on the unseen times than the 2 other models. Hence more regularization strategies are needed to achieve better generalization and performance

across the complete time range.

In Figure 5.9, the relative model output over simulation time is shown. The three models behave similarly, applying larger corrections early in the simulation, after which the magnitude of the model output remains relatively constant. Returning to the discussion in Section 5.4.1, where we studied a model predicting corrections for a single component, this section considers the full corrector model. This allows us to examine the relative importance of each field in the predicted correction.

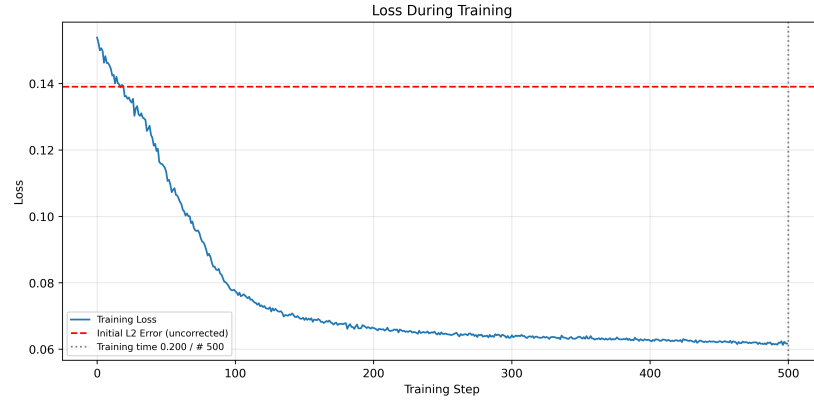
When interpreting these results in the context of supernova simulations, however, we should exclude the magnetic field from this comparison. The MHD blast setup used in this section was specifically designed to test the simulation’s magnetic field capabilities and therefore does not represent a typical supernova setup, in which the magnetic field is expected to play a comparatively smaller role. Including the magnetic field in this comparison would therefore give it a disproportionate importance relative to the other fields. Excluding it allows us to examine the relative importance of the hydrodynamic variables more representative of a supernova blast. With this consideration, the corrector is observed to prioritise the velocity components, suggesting that the velocity field is one of the most important fields when simulating supernova blasts. This is consistent with the role of velocity in transporting both energy and mass, meaning errors in the velocity field can propagate to the density and pressure fields.

5.4.3. Biprobem multiprobem training

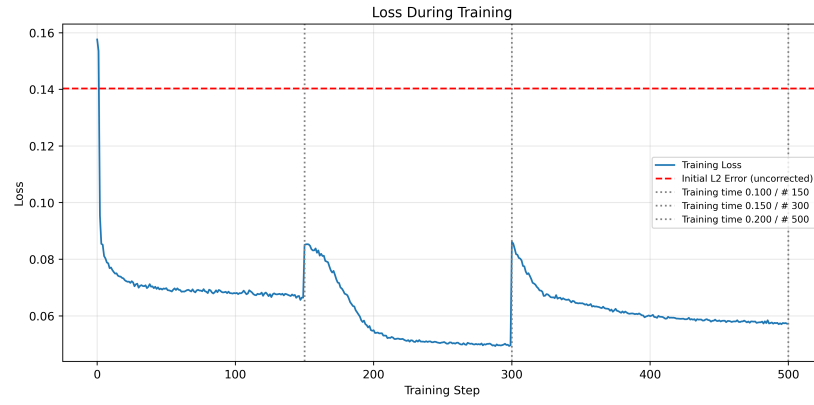
We train a model on the following two problem instances to test the multiprobem API and generalisation capabilities of a model trained on a reduced set of problems:

- **MHD blast:** With $B_0 = 10$, $\theta = \pi/2$, $\phi = \pi/4$ and $r_0 = 0.125$.
- **OT vortex:** With $\epsilon = 0.2$, `vortex.axis` = z axis and `parity` = -1 .

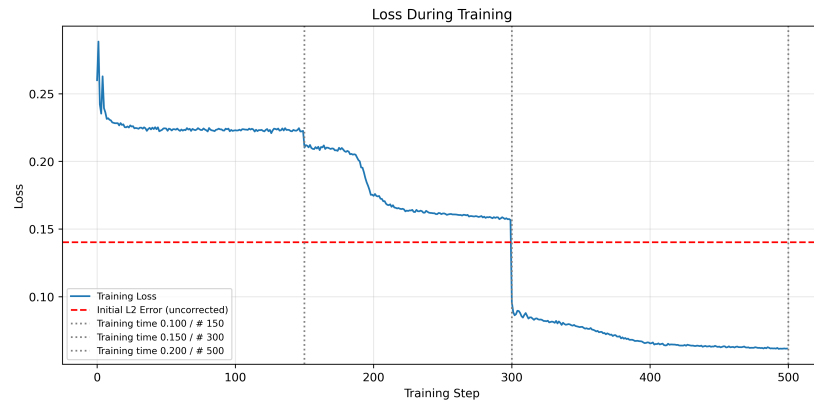
The training loss evolution is presented in Figure 5.10. Overall, we observe the model converge on the 2 problems simultaneously, showing that the network can improve on both without observable conflicts. The OT vortex objective is weakly dependent on the



(a) Benchmark model loss evolution

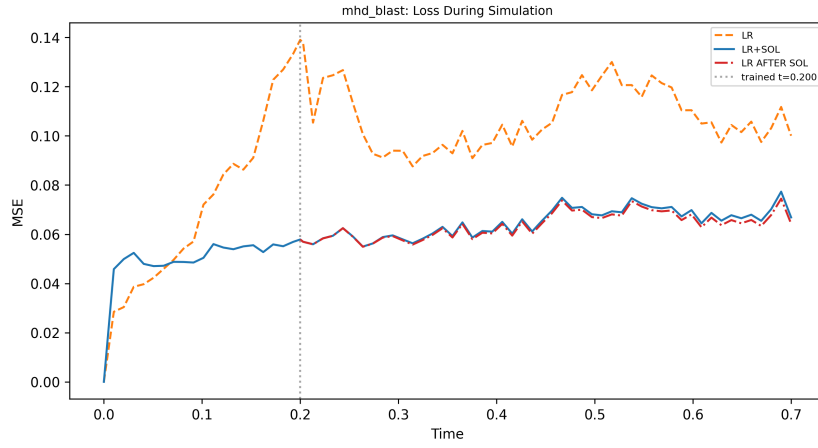


(b) Back-to-front model loss evolution

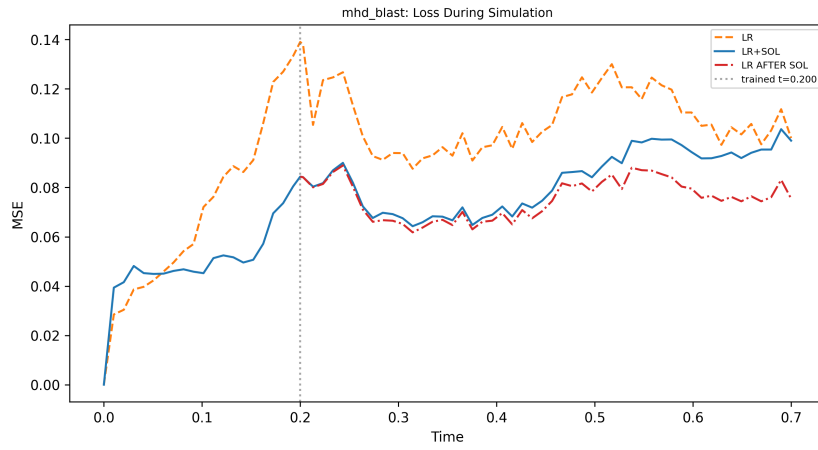


(c) Front-to-back model loss evolution

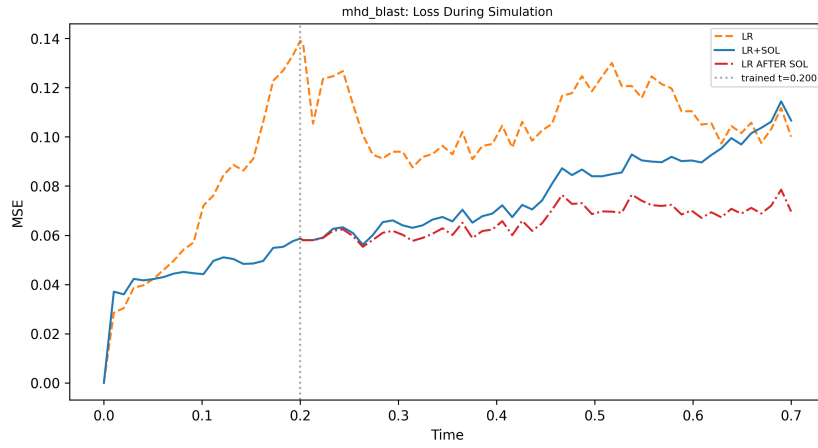
Figure 5.7: Training loss evolution for the different curriculum learning strategies.



(a) Benchmark model loss blast MSE

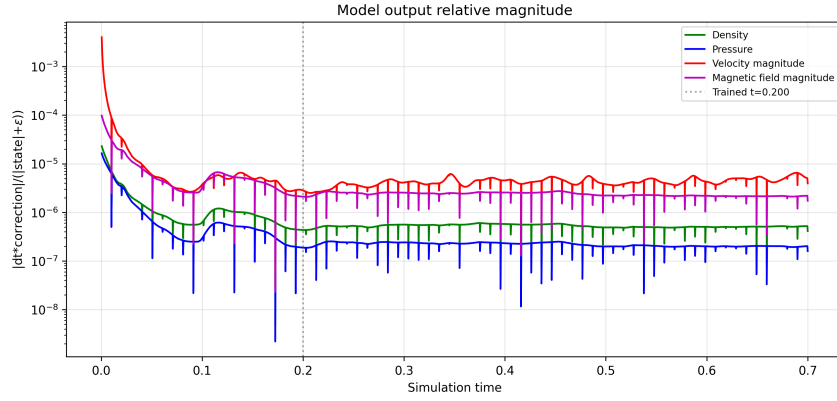


(b) Back-to-front model loss blast MSE

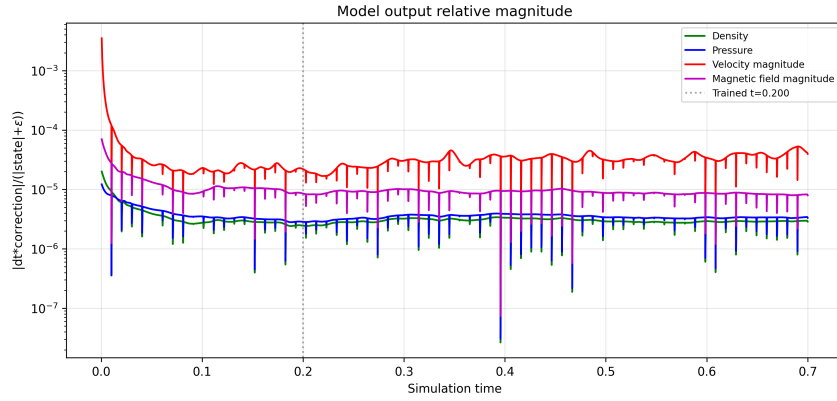


(c) Front-to-back model loss blast MSE

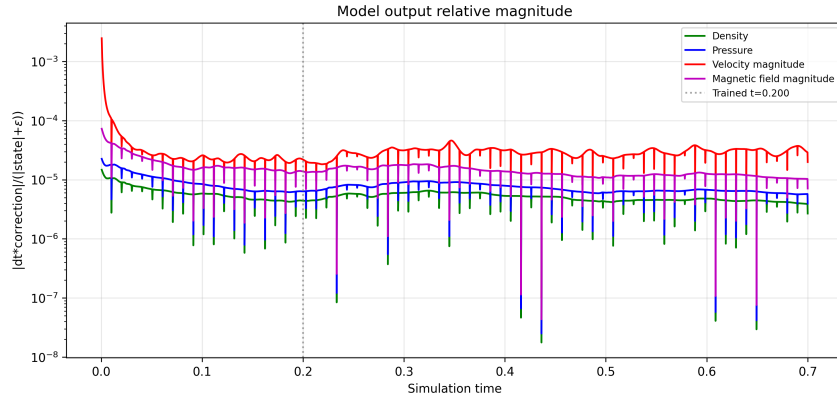
Figure 5.8: Simulation loss on an MHD blast for the different curriculum learning strategies.



(a) Benchmark model relative output



(b) Back-to-front model relative output



(c) Front-to-back model relative output

Figure 5.9: Relative output of each of the considered curriculum learning trained models. The metric presented is $|\Delta t \times \text{correction}|/(|\text{state}| + \epsilon)$ where ϵ is a small constant to ensure numerical stability. The sharp peaks are due to the adaptative time stepping and don't reflect the behaviour of the model.

resolution when compared with the MHD blast, thus acting as a physical consistency objective for the model. This observation then suggests that the addition of weakly dependent resolution problems may help ensure physically self-consistent models while maintaining the same performance.

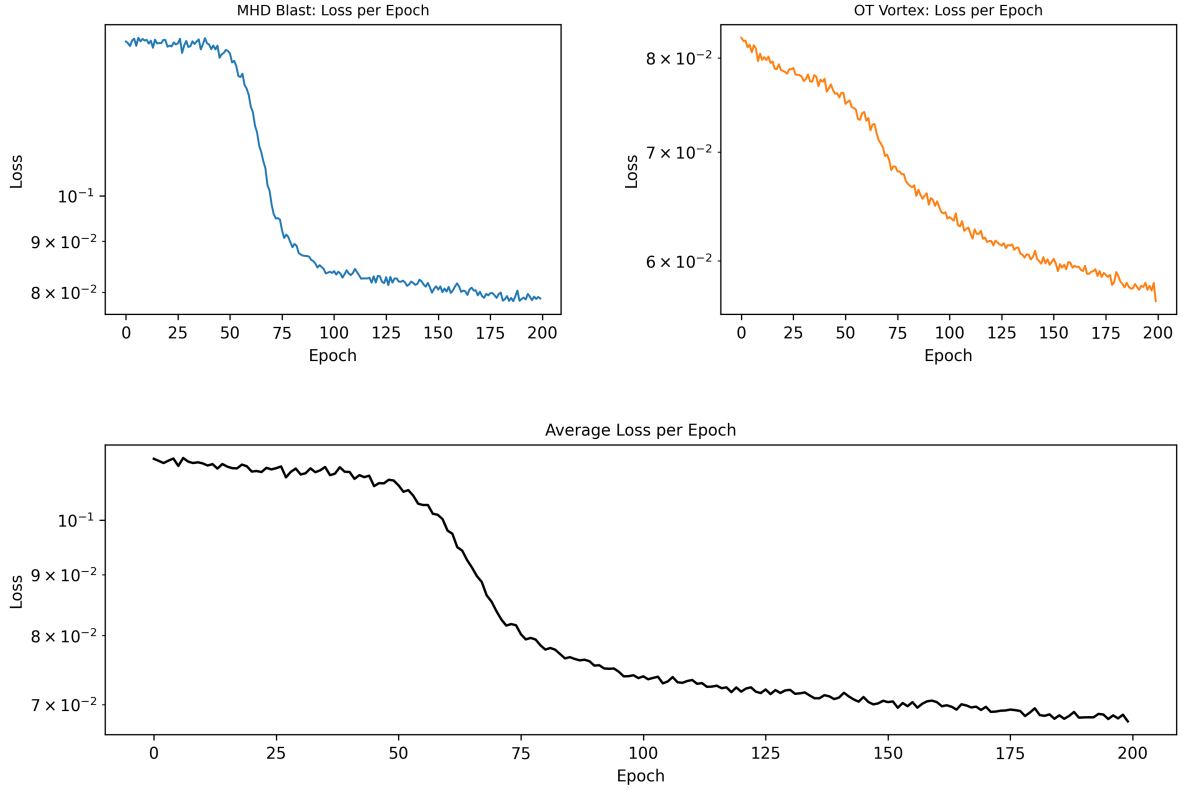
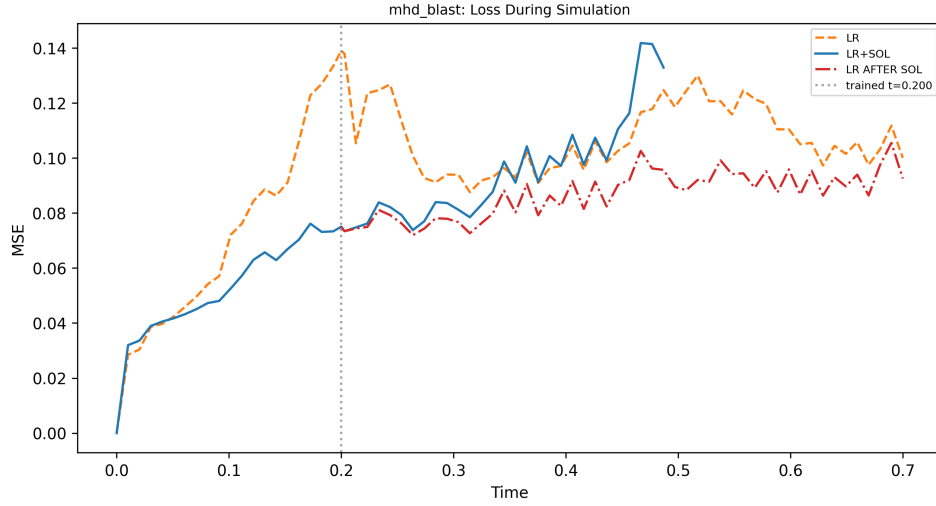
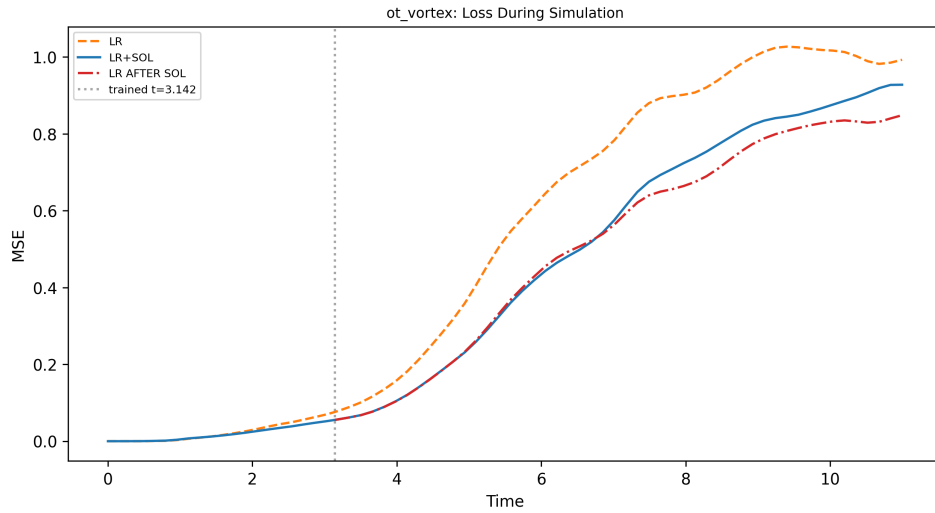


Figure 5.10: Loss training evolution over a set of 2 problems, an MHD blast and an OT vortex.

The loss against simulation time for both problems is presented in Figure 5.11. On the later part of the MHD blast problem the model quickly transforms the simulation unstable and crashes. In particular, the corrector predicts a non-physical correction, which, regardless of the safeguards imposed, runs the simulation rapidly into crashing. Therefore, although the addition of the OT vortex problem did not prevent loss convergence or stable training, the resulting final model is less robust during long simulations onto unseen phenomena. This may be attributed to several factors, including the limited number of effective training epochs compared with the previous model which was



(a) Loss simulation evolution over the MHD blast.



(b) Loss simulation evolution over the OT vortex.

Figure 5.11: Loss simulation evolution for the MHD blast and OT vortex. The MHD blast pure SOL run crashes during the simulation, thus, its line does not reach the far right end.

trained on 500 epochs, the small dataset of 2 problems, or the gradient aggregation strategy which promotes conflict-free updates while and might degrade the individual performance on each of the problems. Although the loss was still decreasing at the end of training, its rate of decrease had become relatively small. Consequently, extending the training could potentially provide further improvements, while the use of multiple curriculum learning strategies may have limited the effective training time available for learning the MHD blast dynamics.

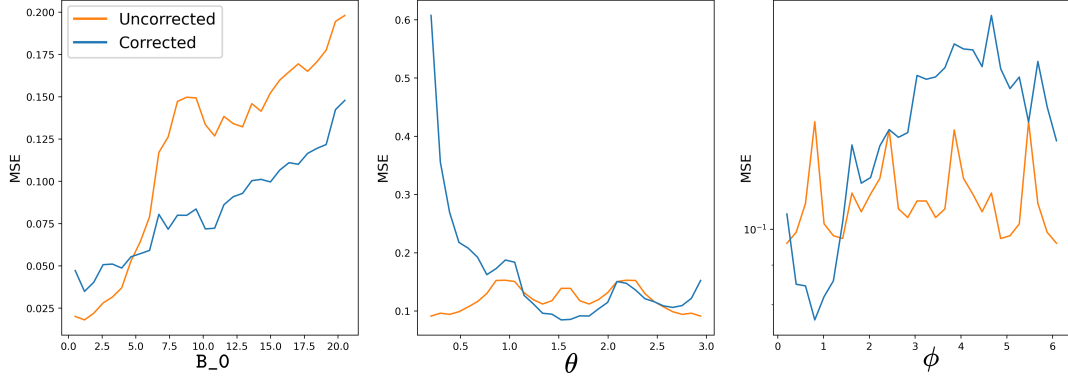
Lastly, we examine how this model generalises over unseen parameters. We plot the MSE loss at the target time while varying the problems parameters. Although a more thorough examination would be needed to ensure real generalisation, the current performance is already too poor. Therefore, improving the performance at the target time is a prerequisite to continue looking further in a broader study. The figures are presented in 5.12. The improved performance appears only in the regions around the trained parameters, this phenomenon is extreme for the OT vortex where the only combination of parameters that appears to improve the baseline is the trained one. This observation, together with the lack of angle generalisation in the MHD blast plots, may be attributed to CNN models not being rotation invariant. A typical strategy to achieve rotation invariance is data augmentation, which we try in the following.

5.4.4. MHD Blast Azimuthal rotation generalization model

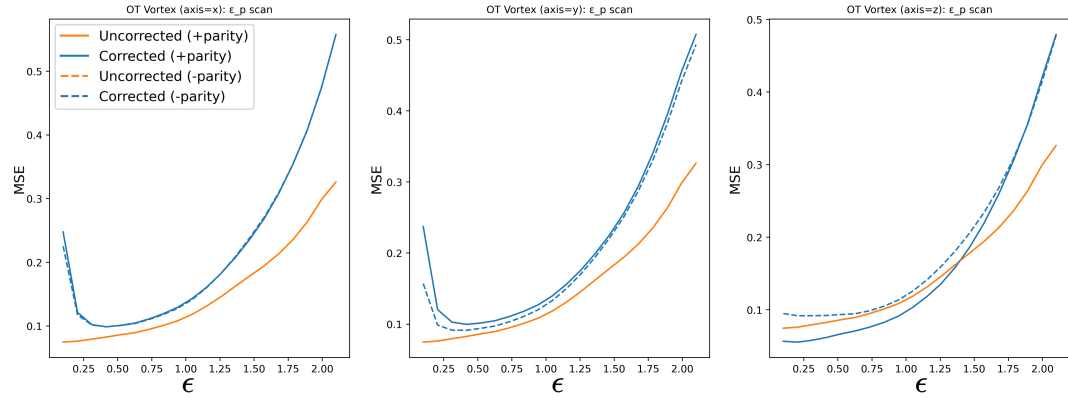
We train a model on the following 3 problem instances to test for angle generalisation of the CNN in the azimuthal direction:

- **MHD blast:** With $B_0 = 10$, $\theta = 0.8$, $\phi = \pi/4$ and $r_0 = 0.125$.
- **MHD blast:** With $B_0 = 10$, $\theta = \pi/2$, $\phi = \pi/4$ and $r_0 = 0.125$.
- **MHD blast:** With $B_0 = 10$, $\theta = 2.3$, $\phi = \pi/4$ and $r_0 = 0.125$.

The training loss evolution is presented in Figure 5.13. The model converges on the 3 problems simultaneously with no apparent issue. The generalisation over the parameters is presented in 5.14. In the middle plot, the study over θ can be seen; comparing



(a) Loss parameter study of the MHD blast. The blast radius is fixed in all panels. From left to right, (i) the magnetic field strength is varied while the field direction is fixed at $\theta = \pi/2$ and $\phi = \pi/4$; (ii) the polar angle θ is varied while the magnetic field strength and azimuthal angle ($\phi = \pi/4$) are fixed; (iii) and the azimuthal angle ϕ is varied while the magnetic field strength and polar angle ($\theta = \pi/2$) are fixed, restricting the field direction to the x - y plane



(b) OT vortex parameter study. The ϵ parameter, which controls the perturbation amplitude in the plane perpendicular to the vortex axis, is varied. From left to right, different vortex orientations are studied by varying the axis perpendicular to the vortex plane, together with the effect of parity.

Figure 5.12: Generalization study of the model trained simultaneously on an MHD blast and an OT vortex. The parameter studies evaluate the model's ability to generalize to configurations beyond those encountered during training. The parameters used during training are $B_0 = 10$, $\theta = \pi/2$, and $\phi = \pi/4$ for the MHD blast, and **axis**= z , $\epsilon = 0.2$, and positive parity for the OT vortex.

it now with 5.12, the overall generalisation can be seen in work. Although overall performance decreased, especially when looking at the rightmost plot, the difference between the corrected loss and target loss at the trained point $\phi = \pi/2$ is reduced. Thus we can extract an overall tradeoff between performance and generalisation, which is to be expected, as before the model was overfitting the problem and now, through data augmentation, we are weakly enforcing biases such as rotation invariance. Of course this was done at a small scale, and larger tests are needed to ensure these claims.

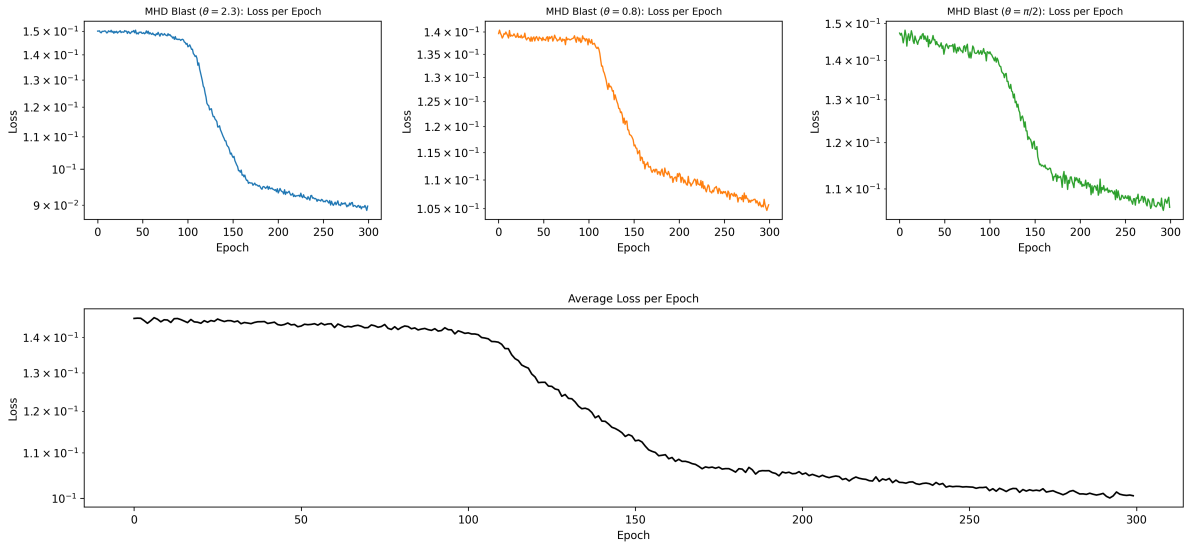


Figure 5.13: Training loss evolution of the MHD blast angle study model. The model was trained on 3 MHD Blasts with equal parameters but the azimuthal direction of the initial magnetic field (θ).

5.4.5. Scaled multiproblem training

As a final experiment, we train the model simultaneously on six problems, consisting of three MHD blasts and three OT vortices:

- **MHD blast** ($B_0 = 0.5$): With $B_0 = 0.5$, $\theta = \pi/2$, $\phi = \pi/4$, and $r_0 = 0.125$.
- **MHD blast** (B -field third quadarant): With $B_0 = 10$, $\theta = \arccos(-1/\sqrt{3}) \approx 2.186$, $\phi = -3\pi/4$, and $r_0 = 0.125$.
- **MHD blast** (Default): With $B_0 = 10$, $\theta = \pi/2$, $\phi = \pi/4$, and $r_0 = 0.125$.

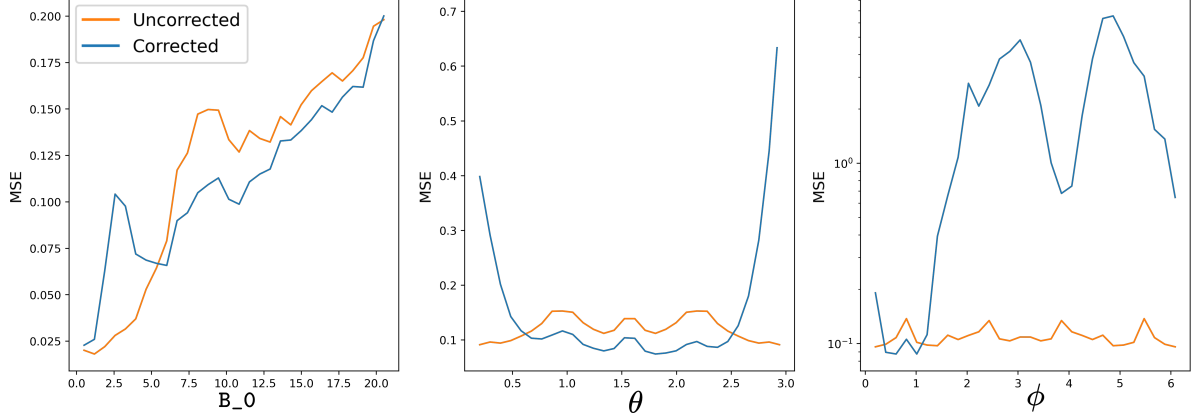


Figure 5.14: Loss parameter study of the MHD blast. The blast radius is fixed in all panels. From left to right, (i) the magnetic field strength is varied while the field direction is fixed at $\theta = \pi/2$ and $\phi = \pi/4$; (ii) the polar angle θ is varied while the magnetic field strength and azimuthal angle ($\phi = \pi/4$) are fixed; (iii) and the azimuthal angle ϕ is varied while the magnetic field strength and polar angle ($\theta = \pi/2$) are fixed, restricting the field direction to the x - y plane.

- **OT vortex** ($\epsilon = 0.6$): With $\epsilon = 0.6$, `vortex_axis` = z axis, and `parity` = -1 .
- **OT vortex** (y -axis): With $\epsilon = 0.2$, `vortex_axis` = y axis, and `parity` = -1 .
- **OT vortex** (Default): With $\epsilon = 0.2$, `vortex_axis` = z axis, and `parity` = -1 .

The training loss evolution can be seen in Figure 5.15. Overall, the model converges across most of the 6 problems, with the exception of the MHD Blast with low magnetic field strength. For this problem, the loss evolution remains noisy and does not show convergence. This may be attributed to the difficulty of finding coherent updates in relation to the other 5 problems, as they all entail strong magnetic fields, thus, just by numbers, the optimisation goes in a strong magnetic field direction. Possible solutions include augmenting the number of problems which include low magnetic field regimes or an ensemble of models trained on different regimes that could later be orchestrated.

The parameter study is presented in Figure 5.16. Looking first at the MHD Blast problem, the plots show improved performance mainly in the trained parameter range. We see improvements in the small valued B_0 region and around the two values of θ considered. We also observe improvement at the extreme $\theta = \pi$, whereas in the previous

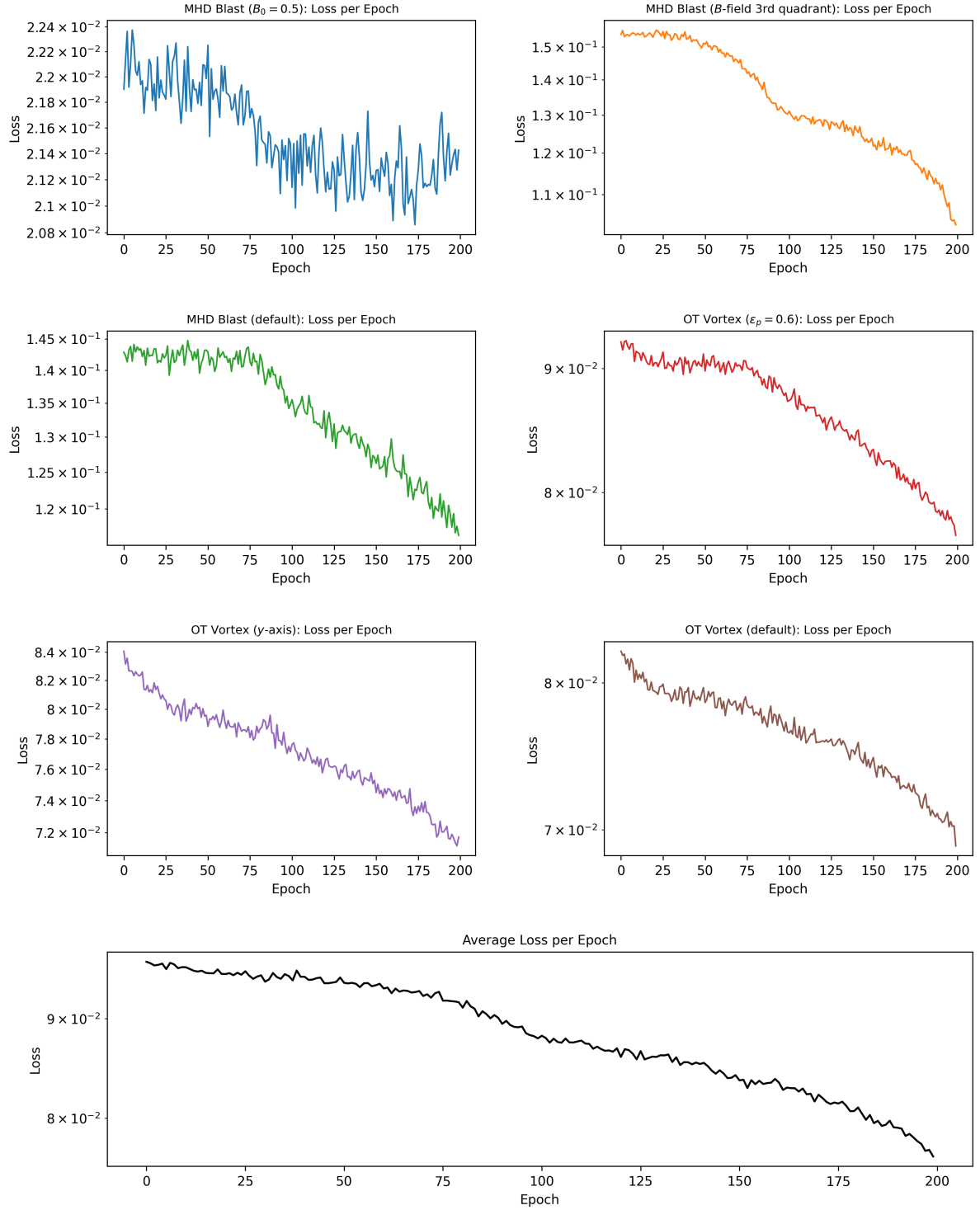
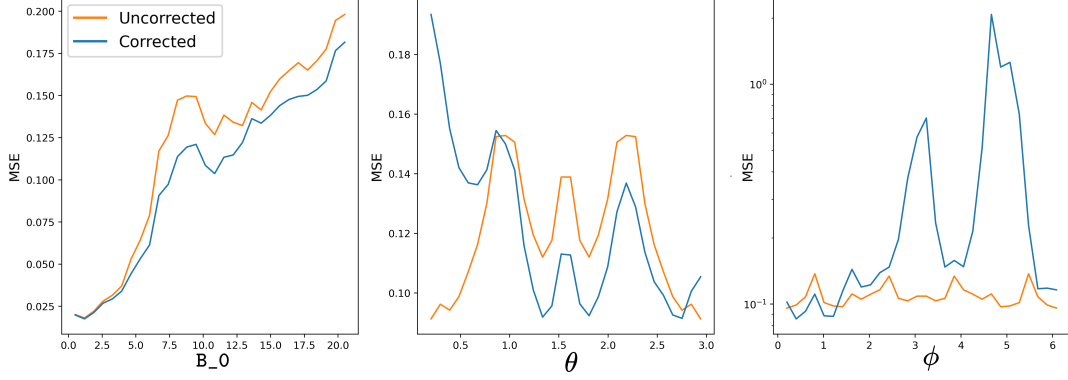
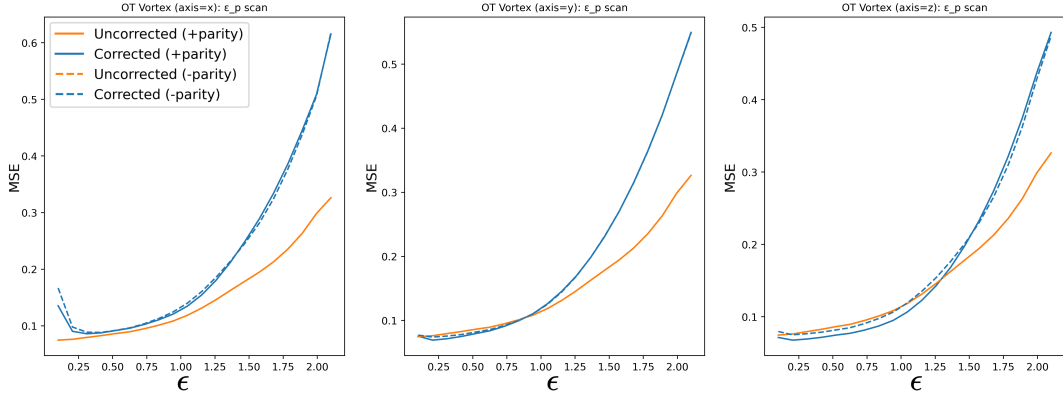


Figure 5.15: Training loss evolution of the SOL model trained on 6 different problems. The model was trained on 3 MHD Blasts and 3 OT vortices, each with different parameters.



(a) Loss parameter study of the MHD blast. The blast radius is fixed in all panels. From left to right, (i) the magnetic field strength is varied while the field direction is fixed at $\theta = \pi/2$ and $\phi = \pi/4$; (ii) the polar angle θ is varied while the magnetic field strength and azimuthal angle ($\phi = \pi/4$) are fixed; (iii) and the azimuthal angle ϕ is varied while the magnetic field strength and polar angle ($\theta = \pi/2$) are fixed, restricting the field direction to the x - y plane



(b) OT vortex parameter study. The ϵ parameter, which controls the perturbation amplitude in the plane perpendicular to the vortex axis, is varied. From left to right, different vortex orientations are studied by varying the axis perpendicular to the vortex plane, together with the effect of parity.

Figure 5.16: Generalization study performed on the model trained simultaneously on 3 MHD blasts and 3 OT vortices.

angle study it did explode in that regime. Still, we do not observe an improvement over the ϕ angle, which is to be expected as the model was only trained with 1 fixed $\phi = \pi/4$. Comparing the response over the θ range with the previous section, the model shows worse overall performance, this again can be attributed to a tradeoff in induced biases and performance.

Continuing on OT Vortex, the pronounced split between different parities observed in the 2 problem trained model is reduced. This, together with the improvements over the angle ranges in the MHD blast, may be an early sign of rotation invariance due to data augmentation. However, the evidence is not conclusive. Besides the model showcases improvement mainly for the set of trained parameters (axis y and z) whereas axis x remains worse than the low resolution reference.

5.5. Discussion

In this section solver-in-the-loop models were trained with multiproblem approaches and further studies into generalisation and data augmentation were done. It's important to remark the time and memory complexity that entails training any of these models as for low simulations the priors occupy substantial memory and strategies such as memory checkpointing have to be used. Thus, there's a high memory-time tradeoff; for instance, the last model shown, trained on 6 problems and 32^3 low resolution, was trained on 3 NVIDIA A100 GPUs with 40 GB of memory and took approximately 10 hours. Thus, SOL models might be useful only in scenarios where hundreds of simulations of 1 or 2 type problems with slightly different initial parameters are needed; still, in such a case, it is still to be compared with skipping the solver-in-the-loop and getting the simulations directly, due to the time it takes to train and finetune each of these models. Such an application could be Simulation Based Inference, where several simulations with different initial parameters are needed.

Commenting on the overall generalisation capabilities, early stages of generalisation were observed, although it is still unclear whether this was pure generalisation or interpolation. Strategies such as physically informed neural networks or gradient aggregation were used

and proven to work. In future works, this could be expanded with further inductive biases and extra losses such as vorticity-based losses for turbulence. On the topic of turbulence, due to the chaoticness of the simulation, the early experiments done were unstable and eventually gradients exploded. This signifies the need for further generalisation strategies in future work.

One reference state per problem was used together with rather large simulation rollouts. This proved to work but also allowed for overfitting in phase space and underperformance on the unseen regions of the simulation. One strategy left to explore is to, instead of training on rather large rollouts and one reference state per problem, like the ones used, training on several small segments of a larger simulation. In such fashion, the dataset of initial states would be larger without largely trading off for memory time, as the simulation and gradient computation time would be shorter. It would also allow for more stable and faster loss convergence, and later in training, these time intervals could be increased to further motivate the model’s performance in larger rollouts, allowing for both a quick and stable start and better performance on larger rollouts. An early approach of this strategy was tried with curriculum learning and the results were mixed. Here we propose taking smaller simulation times and more than once at an epoch, from different parts of the simulation.

On our multiproblem work we handpicked the problems to train on. Although this allows for quick experimentation, it does not allow for proper generalisation or big probes. The use of larger datasets with broader parameter ranges and stochastic decisions was tried, and in our initial experiments we found rather complex loss evolutions where the loss seemed noisy and nonconvergent. This approach, together with the previous, where smaller simulation segments would be used, might be fundamental in continuing the study of these models. This would allow models to be trained on several parameters and regimes without a large tradeoff in time or active memory.

In our work, only one architecture was considered; although it proved useful for the reduced experiments performed, our results and conclusions depend on it. One limitation of the model considered is its parameter agnosticism, especially with respect to simulation parameters. This makes the model unable to generalise through simulation pa-

rameters such as CFL constant, Riemann solvers or boundary conditions. Future works should explore other models in this direction. Examples of parameter-aware models are: a suite of models trained on different simulation parameters that could be orchestrated or models with specialised parameter-aware layers or blocks.

In conclusion solver-in-the-loop models are still to be further studied and expanded to be considered a proper tool in the MHD astrodynamical field. Nowadays, these models don't ensure proper generalisation nor performance across broad domains. Still, SOL architectures in theory pose as an intermediate machine learning application in the MHD field that, given sufficient proven performance, could become a functioning tool for large simulation rollouts, reducing time and memory costs on further ISM studies.

6. Conclusion

In this work, both ad-hoc and post-hoc approaches for three-dimensional MHD simulations were investigated and evaluated: superresolution and solvers-in-the-loop. The results obtained for superresolution are promising and consistent with previous work. At the same time, our work was an early test of FNOs in turbulent setups, and several avenues of research remain open for further implementation. In particular the incorporation of adversarial losses, multiscale approaches and datasets that span a larger range of parameters would overall improve the sharp peaks missing from our superresolved states. The shifted Fourier modes were introduced in this work, which in multiscale approaches would be of particular use as each different resolution increase could focus on different parts of the spectra. Overall, the SFNO and U-SFNO models were proven successful for superresolution tasks with neural operators.

Developing a superresolution methodology that could generalise over different simulation regimes, not only turbulence, that scales properly with data and is able to recover meaningful physical statistics is still challenging. Nevertheless, with the recent advancements in superresolution and NO models, it seems plausible in the near future that similar neural operator approaches could be used in complex simulation setups to provide computational speedups over high resolution simulations.

Regarding solvers-in-the-loop, different strategies that involve curriculum training or multiproblem approaches were tested. From our experiences, we can extract similar conclusions to those from the superresolution experiments; further research is needed in the direction of advanced training techniques. Specifically, adaptive multiinterval training that allows the model to avoid overfitting on the simulation and better capture the physical dynamics. Nevertheless, even within this early proof, early signs of generalisation were found.

The use of coarser approaches to the dataset in both methodologies remains to be improved. Typical approaches that involve RNNs or superresolution models patch their dataset to allow for faster training and facilitate convergence and learning. Due to time limitations and the broad scope of the work, such an approach was not implemented.

Based on previous work on patched training [Schiødt et al., 2025] and windowed time series [Eshaghi et al., 2025] that such strategies could be beneficial and potentially lead to more robust models across different simulations.

Having tested both approaches, one aspect not investigated in this work is the comparison with emulators, which are a state of the art and a rapidly developing field of hybrid ML simulation approaches. On paper, we expect both, solvers-in-the-loop and superresolution, to outperform emulators for longer rollouts where emulators still struggle to correctly predict the simulation. Nevertheless, emulators reduce bottlenecks on simulations with different time regimes occurring simultaneously; for instance, radiative transfer, when coupled with hydrodynamics, takes up a large part of the computation time as the radiation time steps are considerably smaller than the hydrodynamical ones. Examples of emulators that aim to solve this bottleneck are [Fredj et al., 2025] or [Branca et al., 2025].

In conclusion, approaches that aim to improve simulations by exploiting their differentiability were tested. Both approaches are well suited for longer rollouts and could even be combined if trained together. A new parameter for the FNO model was introduced and proven to work, and studies on multiproblem solvers-in-the-loop were implemented with an API which allows for easy experimentation. Hybrid machine learning simulations are the next frontier of computational physics. By exploiting the large amounts of available data and numerical methods methodologies, larger and larger proofs could be achieved in the future.

References

- [Abadi et al., 2015] Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., and Zheng, X. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org.
- [Abid and San, 2026] Abid, M. and San, O. (2026). Simr-no: A spectrally-informed multi-resolution neural operator for turbulent flow super-resolution.
- [Akiba et al., 2019] Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631.
- [Alam et al., 2025] Alam, M. N. U., Basava, K., Chitransh, A., and et al. (2025). Machine learning in biological research: key algorithms, applications, and future directions. *BMC Biology*, 23:324.
- [Ansel et al., 2024] Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., Hirsh, B., Huang, S., Kalambarkar, K., Kirsch, L., Lazos, M., Lezcano, M., Liang, Y., Liang, J., Lu, Y., Luk, C., Maher, B., Pan, Y., Puhersch, C., Reso, M., Saroufim, M., Siraichi, M. Y., Suk, H., Suo, M., Tillet, P., Wang, E., Wang, X., Wen, W., Zhang, S., Zhao, X., Zhou, K., Zou, R., Mathews, A., Chanan, G., Wu, P., and Chintala, S. (2024). PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM.

- [Augustine, 2024] Augustine, M. T. (2024). A survey on universal approximation theorems.
- [Baron, 2019] Baron, D. (2019). Machine learning in astronomy: a practical overview.
- [Baydin et al., 2018] Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey.
- [Bradbury et al., 2018] Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Katariya, Y., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. (2018). JAX: composable transformations of Python+NumPy programs.
- [Branca et al., 2025] Branca, L., Rost, R., and Buck, T. (2025). Ray-trax: Fast, time-dependent, and differentiable ray tracing for on-the-fly radiative transfer in turbulent astrophysical flows.
- [Butler et al., 2018] Butler, K. T. et al. (2018). Machine learning for molecular and materials science. *Nature*, 559:547–555.
- [Carraro, 2021] Carraro, G. (2021). *Astrophysics of the Interstellar Medium*. UNITEXT for Physics. Springer, Cham.
- [Cazenavette and Lucey, 2021] Cazenavette, G. and Lucey, S. (2021). On the bias against inductive biases.
- [Choi et al., 2025] Choi, H., Cho, C., Kim, M., and Park, J. (2025). Perspective on machine-learning-based large-eddy simulation. *Physical Review Fluids*, 10(11):110701.
- [Daubner et al., 2025] Daubner, S., Cohen, A. E., Dörich, B., and Cooper, S. J. (2025). evoxels: A differentiable physics framework for voxel-based microstructure simulations.
- [Daviet et al., 2025] Daviet, G., Shen, T., Sharp, N., and Levin, D. I. (2025). Neurally integrated finite elements for differentiable elasticity on evolving domains. *ACM Trans. Graph.*, 44(2).

- [Defazio et al., 2024] Defazio, A., Cutkosky, A., Mehta, H., and Mishchenko, K. (2024). Optimal linear decay learning rate schedules and further refinements.
- [Domke, 2009] Domke, J. (2009). Automatic differentiation: The most criminally underused tool in the potential machine learning toolbox? <https://justindomke.wordpress.com/2009/02/17/automatic-differentiation-the-most-criminally-underused-tool-in-the-potential-machine-learning-toolbox/>. Blog post, accessed 2026-05-11.
- [Draine, 2011] Draine, B. T. (2011). *Physics of the Interstellar and Intergalactic Medium*.
- [Drysch et al., 2025] Drysch, S., Stotko, D., and Klein, R. (2025). Differentiable XPBD for Gradient-Based Learning of Physical Parameters from Motion. In *International Symposium on Vision, Modeling, and Visualization (VMV)*.
- [Du et al., 2021] Du, T., Wu, K., Ma, P., Wah, S., Spielberg, A., Rus, D., and Matusik, W. (2021). Diffpd: Differentiable projective dynamics.
- [Duruiseaux et al., 2026] Duruiseaux, V., Kossaifi, J., and Anandkumar, A. (2026). Fourier neural operators explained: A practical perspective.
- [Dvorkin et al., 2022] Dvorkin, C., Mishra-Sharma, S., Nord, B., Villar, V. A., Avestruz, C., Bechtol, K., Ćiprijanović, A., Connolly, A. J., Garrison, L. H., Narayan, G., and Villaescusa-Navarro, F. (2022). Machine learning and cosmology.
- [Eshaghi et al., 2025] Eshaghi, M. S., Valizadeh, N., Anitescu, C., Wang, Y., Zhuang, X., and Rabczuk, T. (2025). Multi-head neural operator for modelling interfacial dynamics.
- [Eyring et al., 2024] Eyring, V., Collins, W. D., Gentine, P., and et al. (2024). Pushing the frontiers in climate modelling and analysis with machine learning. *Nature Climate Change*, 14:916–928.
- [Farea et al., 2024] Farea, A., Yli-Harja, O., and Emmert-Streib, F. (2024). Understanding physics-informed neural networks: Techniques, applications, trends, and challenges. *AI*, 5(3):1534–1557.

- [Fredj et al., 2025] Fredj, E., Gal, I. S., Lavi, N., Belkar, S., Wasserman, M., Zhaohui, D., and Delorme, Y. (2025). A lightweight neural network for accelerating radiative transfer modeling in wrf.
- [Galtier, 2016] Galtier, S. (2016). *Introduction to Modern Magnetohydrodynamics*. Cambridge University Press, Cambridge, United Kingdom.
- [Giovanetti et al., 2025] Giovanetti, C., Lisanti, M., Liu, H., Mishra-Sharma, S., and Ruderman, J. T. (2025). LINX: A fast, differentiable, and extensible big bang nucleosynthesis package. *Phys. Rev. D*, 112:063531.
- [Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>.
- [Guilmin et al., 2025] Guilmin, P., Bocquet, A., Genois, É., Weiss, D., and Gautier, R. (2025). Dynamiqs: an open-source python library for gpu-accelerated and differentiable simulation of quantum systems.
- [Hao Wu, 2026] Hao Wu, Yuan Gao, F. X. F. Z. Q. W. K. W. X. H. X. W. (2026). Differential-integral neural operator for long-term turbulence forecasting.
- [Harder et al., 2024] Harder, P., Hernandez-Garcia, A., Ramesh, V., Yang, Q., Sattigeri, P., Szwarcman, D., Watson, C., and Rolnick, D. (2024). Hard-constrained deep learning for climate downscaling.
- [Harris et al., 2020] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., and Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.
- [Hattori et al., 2024] Hattori, S., Garcia, L., Murray, C., Dong, J., Dholakia, S., Degen, D., and Foreman-Mackey, D. (2024). exoplanet-dev/jaxoplanet: Astronomical time series analysis with JAX.
- [Hayat et al., 2021] Hayat, M. A., Stein, G., Harrington, P., Lukić, Z., and Mustafa, M.

- (2021). Self-supervised representation learning for astronomical images. *The Astrophysical Journal Letters*, 911(2):L33.
- [He et al., 2015] He, K., Zhang, X., Ren, S., and Sun, J. (2015). Deep residual learning for image recognition.
- [Heek et al., 2024] Heek, J., Levskaya, A., Oliver, A., Ritter, M., Rondepierre, B., Steiner, A., and van Zee, M. (2024). Flax: A neural network library and ecosystem for JAX.
- [Helzel et al., 2011] Helzel, C., Rossmanith, J. A., and Taetz, B. (2011). An unstaggered constrained transport method for the 3d ideal magnetohydrodynamic equations. *Journal of Computational Physics*, 230(10):3803–3829.
- [Hennigan et al., 2020] Hennigan, T., Cai, T., Norman, T., Martens, L., and Babuschkin, I. (2020). Haiku: Sonnet for JAX.
- [Hirashima et al., 2025] Hirashima, K., Fujii, M. S., Saitoh, T. R., Harada, N., Nomura, K., Yoshikawa, K., Hirai, Y., Asano, T., Moriwaki, K., Iwasawa, M., Okamoto, T., and Makino, J. (2025). The first star-by-star n -body/hydrodynamics simulation of our galaxy coupling with a surrogate model. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '25, page 1859–1873. ACM.
- [Hopkins, 2016] Hopkins, P. F. (2016). A constrained-gradient method to control divergence errors in numerical mhd. *Monthly Notices of the Royal Astronomical Society*, 462(1):576–587.
- [Hornik et al., 1989] Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366.
- [Horowitz et al., 2025] Horowitz, B., Lukić, Z., Nagamine, K., and Oku, Y. (2025). diffhydro: Inverse multiphysics modeling and embedded machine learning in astrophysical flows.
- [Jumper et al., 2021] Jumper, J. et al. (2021). Highly accurate protein structure prediction with alphafold. *Nature*, 596:583–589.

- [Kaiser et al., 2024] Kaiser, J., Xu, C., Eichler, A., and Garcia, A. S. (2024). Cheetah: Bridging the gap between machine learning and particle accelerator physics with high-speed, differentiable simulations.
- [Karagiorgi et al., 2021] Karagiorgi, G., Kasieczka, G., Kravitz, S., Nachman, B., and Shih, D. (2021). Machine learning in the search for new fundamental physics.
- [Kidger and Garcia, 2021] Kidger, P. and Garcia, C. (2021). Equinox: neural networks in JAX via callable PyTrees and filtered transformations. *Differentiable Programming workshop at Neural Information Processing Systems 2021*.
- [Kingma and Ba, 2017] Kingma, D. P. and Ba, J. (2017). Adam: A method for stochastic optimization.
- [Kovachki et al., 2024] Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A., and Anandkumar, A. (2024). Neural operator: Learning maps between function spaces.
- [LeCun et al., 1989] LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., and Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541–551.
- [Leshno et al., 1993] Leshno, M., Lin, V. Y., Pinkus, A., and Schocken, S. (1993). Multi-layer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867.
- [LeVeque, 2002] LeVeque, R. J. (2002). *Finite Volume Methods for Hyperbolic Problems*, volume 31 of *Cambridge Texts in Applied Mathematics*. Cambridge University Press, Cambridge, UK.
- [Li et al., 2024] Li, Z., Huang, D. Z., Liu, B., and Anandkumar, A. (2024). Fourier neural operator with learned deformations for pdes on general geometries.
- [Li et al., 2021] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. (2021). Fourier neural operator for parametric partial differential equations.

- [Liaw et al., 2018] Liaw, R., Liang, E., Nishihara, R., Moritz, P., Gonzalez, J. E., and Stoica, I. (2018). Tune: A research platform for distributed model selection and training. *arXiv preprint arXiv:1807.05118*.
- [Lim et al., 2017] Lim, B., Son, S., Kim, H., Nah, S., and Lee, K. M. (2017). Enhanced deep residual networks for single image super-resolution.
- [List et al., 2025] List, F., Hahn, O., Flöss, T., and Winkler, L. (2025). Disco-dj ii: a differentiable particle-mesh code for cosmology.
- [Liu et al., 2025] Liu, Q., Chu, M., and Thuerey, N. (2025). Config: Towards conflict-free training of physics informed neural networks.
- [Liu and Tang, 2025] Liu, X. and Tang, H. (2025). Diffno: Diffusion fourier neural operator.
- [Liu-Schiaffini et al., 2024] Liu-Schiaffini, M., Berner, J., Bonev, B., Kurth, T., Aziz-zadenesheli, K., and Anandkumar, A. (2024). Neural operators with localized integral and differential kernels.
- [Lopes et al., 2018] Lopes, M. M., Deiterding, R., Gomes, A. K. F., Mendes, O., and Domingues, M. O. (2018). An ideal compressible magnetohydrodynamic solver with parallel block-structured adaptive mesh refinement. *Computers Fluids*, 173:293–298.
- [Loshchilov and Hutter, 2017] Loshchilov, I. and Hutter, F. (2017). Sgdr: Stochastic gradient descent with warm restarts.
- [Lu et al., 2021] Lu, L., Jin, P., Pang, G., Zhang, Z., and Karniadakis, G. E. (2021). Learning nonlinear operators via deepnet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229.
- [Mahlau et al., 2026] Mahlau, Y., Schubert, F., Berg, L., and Rosenhahn, B. (2026). Fdtdx: High-performance open-source fdtd simulation with automatic differentiation. *Journal of Open Source Software*, 11(117):8912.
- [MAYOR, 2018] MAYOR, A. (2018). *Gods and Robots: Myths, Machines, and Ancient Dreams of Technology*. Princeton University Press.

- [Mignone and Del Zanna, 2021] Mignone, A. and Del Zanna, L. (2021). Systematic construction of upwind constrained transport schemes for mhd. *Journal of Computational Physics*, 424:109748.
- [Mikhaeil et al., 2022] Mikhaeil, J. M., Monfared, Z., and Durstewitz, D. (2022). On the difficulty of learning chaotic dynamics with rnns.
- [Moser et al., 2021] Moser, R. D., Haering, S. W., and Yalla, G. R. (2021). Statistical Properties of Subgrid-Scale Turbulence Models. *Annual Review of Fluid Mechanics*, 53(Volume 53, 2021):255–286.
- [Mukund and Hof, 2018] Mukund, V. and Hof, B. (2018). The critical point of the transition to turbulence in pipe flow. *Journal of Fluid Mechanics*, 839:76–94.
- [Najafi et al., 2026] Najafi, H., Lagerwall, G. L., Obeysekera, J., and Liu, J. (2026). Machine learning in climate downscaling: A critical review of methodologies, physical consistency, and operational applications. *Water*, 18(2).
- [Newbury et al., 2024] Newbury, R., Collins, J., He, K., Pan, J., Posner, I., Howard, D., and Cosgun, A. (2024). A review of differentiable simulators.
- [Neymotin and Hazan, 2025] Neymotin, S. A. and Hazan, H. (2025). Differentiable simulation expands frontiers for biophysical neural models. *Nature Methods*, 22(12):2503–2505.
- [Oommen et al., 2025] Oommen, V., Khodakarami, S., Bora, A., Wang, Z., and Karniadakis, G. E. (2025). Learning turbulent flows with generative models: Super-resolution, forecasting, and sparse flow reconstruction.
- [Orszag and Tang, 1979] Orszag, S. A. and Tang, C.-M. (1979). Small-scale structure of two-dimensional magnetohydrodynamic turbulence. *Journal of Fluid Mechanics*, 90:129–143.
- [Pasupuleti, 2025] Pasupuleti, M. K. (2025). Vision transformers vs. convolutional neural networks: Benchmarking across tasks. *International Journal of Academic and Industrial Research Innovations(IJAIRI)*, 05:424–434.

- [Qin et al., 2024] Qin, S., Lyu, F., Peng, W., Geng, D., Wang, J., Tang, X., Leroyer, S., Gao, N., Liu, X., and Wang, L. L. (2024). Toward a better understanding of fourier neural operators from a spectral perspective.
- [Reddi et al., 2019] Reddi, S. J., Kale, S., and Kumar, S. (2019). On the convergence of adam and beyond.
- [Ronneberger et al., 2015] Ronneberger, O., Fischer, P., and Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation.
- [Schiødt et al., 2025] Schiødt, M., Mücke, N. T., and Velte, C. M. (2025). Generative super-resolution of turbulent flows via stochastic interpolants.
- [Schmidt, 2019] Schmidt, R. M. (2019). Recurrent neural networks (rnns): A gentle introduction and overview.
- [Semenov, 2025] Semenov, V. A. (2025). Capturing turbulence with numerical dissipation: A simple dynamical model for unresolved turbulence in hydrodynamic simulations. *The Astrophysical Journal Supplement Series*, 281(2):37.
- [Shi et al., 2016] Shi, W., Caballero, J., Huszár, F., Totz, J., Aitken, A. P., Bishop, R., Rueckert, D., and Wang, Z. (2016). Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network.
- [Sormani, 2022] Sormani, M. (2022). Lecture notes in astrophysical fluid dynamics. Lecture notes for a course at Heidelberg University.
- [Storcks, 2025] Storcks, L. (2025). astronomix - differentiable mhd in jax.
- [Storcks and Buck, 2024] Storcks, L. and Buck, T. (2024). Differentiable conservative radially symmetric fluid simulations and stellar winds – jfluids.
- [Tang et al., 2025] Tang, X., Zhang, H., Cao, Y., and Zou, D. (2025). Understanding the generalization of stochastic gradient adam in learning neural networks.
- [Ting, 2026] Ting, Y.-S. (2026). Deep learning in astrophysics. *Annual Review of Astronomy and Astrophysics*.

- [Toosi et al., 2021] Toosi, A., Bottino, A. G., Saboury, B., Siegel, E., and Rahmim, A. (2021). A brief history of ai: How to prevent another winter (a critical review). *PET Clinics*, 16(4):449–469.
- [Tran et al., 2023] Tran, A., Mathews, A., Xie, L., and Ong, C. S. (2023). Factorized fourier neural operators.
- [Um et al., 2021] Um, K., Brand, R., Yun, Fei, Holl, P., and Thuerey, N. (2021). Solver-in-the-loop: Learning from differentiable physics to interact with iterative pde-solvers.
- [Vermariën et al., 2025] Vermariën, G., Grassi, T., de Sande, M. V., Viti, S., Bovino, S., Lupi, A., Ruf, A., Branca, L., and Walsh, C. (2025). Carbox: an end-to-end differentiable astrochemical simulation framework.
- [Viterbo and Buck, 2025] Viterbo, G. and Buck, T. (2025). Differentiable n-body code for galactic dynamics – odiseo.
- [Wen et al., 2022] Wen, G., Li, Z., Azizzadenesheli, K., Anandkumar, A., and Benson, S. M. (2022). U-fno – an enhanced fourier neural operator-based deep-learning model for multiphase flow.
- [Wu, 2018] Wu, K. (2018). Positivity-preserving analysis of numerical schemes for ideal magnetohydrodynamics. *SIAM Journal on Numerical Analysis*, 56(4):2124–2147.
- [Yang et al., 2023] Yang, Q., Hernandez-Garcia, A., Harder, P., Ramesh, V., Sattegeri, P., Szwarcman, D., Watson, C. D., and Rolnick, D. (2023). Fourier neural operators for arbitrary resolution climate data downscaling.
- [Yang et al., 2025] Yang, X., Ji, Z., and Lai, Y.-K. (2025). Differentiable physics-based system identification for robotic manipulation of elastoplastic materials. *The International Journal of Robotics Research*, 44(13):2126–2155.
- [Zeghal et al., 2022] Zeghal, J., Lanusse, F., Boucaud, A., Remy, B., and Aubourg, E. (2022). Neural posterior estimation with differentiable simulators.
- [Zhang et al., 2018] Zhang, Y., Li, K., Li, K., Wang, L., Zhong, B., and Fu, Y. (2018). Image super-resolution using very deep residual channel attention networks.

-
- [Zhao et al., 2018] Zhao, H., Gallo, O., Frosio, I., and Kautz, J. (2018). Loss functions for neural networks for image processing.
- [Zhuang et al., 2020] Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., and He, Q. (2020). A comprehensive survey on transfer learning.

A. Training hyperparameters

Here we present the hyperparameters used to train the SFNO and EDSR in Section 4. We use a scheduler following an early stopper fashion, after a fixed amount of non improving epochs the learning rate is reduced by a predefined rate. To ensure the training stability the gradient modulus is clipped. The inputs have an extra gaussian noise to improve generalization, this is encoded by the noise standard deviation. The data is normalized to allow for a stable gradient calculation. The parameters used for each of the models are presented in 7. The SFNO and UFNO considered in this work consist of 35 million parameters whereas the EDSR consists of 5 million parameters, this difference is represented in the change of parameters used to train each of the models. Hence SFNO has a larger training time but also large initial learning rate.

Table 7: Training hyperparameters for SFNO and EDSR.

Hyperparameter	SFNO	EDSR
Epochs	500	250
Learning rate	1×10^{-3}	3×10^{-4}
Weight decay	1×10^{-4}	1×10^{-4}
Batch size	8	8
Noise standard deviation	0.01	0.01
Gradient accumulation	8	8
Early stop patience	40	20
Scheduler factor	0.8	0.5
Scheduler patience	20	10
Gradient clip norm	0.8	1.0

Final thanks

Thanks to the Astro AI group, Tobias, Leonard, Giuseppe, Lorenzo, Rune, Anna Lena and everyone else I had the pleasure of sharing the office space with this last year. It was great to do science with you, and I hope we cross paths again some time soon.

I'm thankful to have friends and family that keep my strength and motivation on hold. I don't want to give anyone preference, so I'm just going to start from South to North geographically. Firstly, my family and friends from my village, Cox: my parents, my grandparents, and my brother, who have always helped and supported my studies. My friends from school (El Chiringo) and the village's wind band La Armónica de Cox, although the distance is large, I hold you all close to my heart. To all my Bachelor's friends who, to this day, still make time every week to meet online and play some video games, thank you so much; some weeks, those were the moments I could only hold on to. To my girlfriend Tabea, who always believed in me and sees things in me I'd never say I had. To my friends in Heidelberg, Ainhoa, Leon, Cristina, Iñigo, Drishika, Oliwia, Samu, Fabi, Hanyi and William, Yolanda, Ale, Gahee, Gautham, Jhananii, Paul, Stefi and anyone I encountered along the way; it was lovely to share these two years with all of you. All those Mensa lunches, Botanik coffees, movie nights, bike trips and picnics have made these two years something I'll remember forever.

It's been a bumpy road, but at the end I'm happy to say I made it, not for myself but for the ones that care for me. These are just words and could never represent how much love I have for each of the friends and family mentioned here, so I'll try in spanish: ¡Gracias amigos, os quiero!

Erklärung

Ich versichere, dass ich diese Arbeit selbstständig verfasst habe und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Heidelberg, 01.09.2026

Ort, Datum

A handwritten signature in black ink, consisting of a large, stylized 'J' followed by several loops and a long horizontal stroke extending to the right.

Unterschrift